HYDE: HYBRID DIFFERENTIAL EVOLUTION WITH WARM-START INCREASING POPULATION COVARIANCE MATRIX ADAPTATION EVOLUTION STRATEGY FOR CONTINUOUS PARAMETRIC OPTIMIZATION

A Thesis Presented to

the Faculty of the Undergraduate Program

School of Engineering and Computer Studies

Divine Word College of Legazpi

In Partial Fulfillment

of the Requirements for the Degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

By:

BENJAMIN, DENZEL JAY B.

DIVINA, CHRISTIAN NICKHOS A.

CABALLERO, MANUEL II M.

October 2026

RECOMMENDATION FOR THESIS FINAL DEFENSE

In partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science, this thesis entitled **"HYDE: HYBRID DIFFERENTIAL EVOLUTION WITH WARM START INCREASING POPULATION COVARIANCE MATRIX ADAPTATION EVOLUTION STRATEGY FOR CONTINUOUS PARAMETRIC OPTIMIZATION",** prepared by **Manuel II Morado Caballero, Denzel Jay Bellen Benjamin,** and **Christian Nickhos Alcazar Divina,** is hereby submitted to the Thesis Committee for consideration and approval.

RHODORA FAYE A. BROSAS MBA, MIT

Adviser

In partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science, this thesis entitled **"HYDE: HYBRID DIFFERENTIAL EVOLUTION WITH WARM START INCREASING POPULATION COVARIANCE MATRIX ADAPTATION EVOLUTION STRATEGY FOR CONTINUOUS PARAMETRIC OPTIMIZATION"** prepared by **Manuel II Morado Caballero, Denzel Jay Bellen Benjamin,** and **Christian Nickhos Alcazar Divina,** is hereby submitted to the Thesis Committee for consideration and approval.

RHODORA FAYE A. BROSAS MBA, MIT

Thesis Coordinator

DIVINE WORD COLLEGE OF LEGAZPI
SCHOOL OF ENGINEERING AND COMPUTER STUDIES
Rm. 217, Second Floor, DWCL South Campus
Cor. Rizal & Fr. Bates St., Old Albay District, Legazpi City, Philippines 4500

RESULT OF THE FINAL DEFENSE

Project Title : HYDE: Hybrid Differential Evolution with Warm-start Increasing Population Covariance Matrix Adaptation Evolution Strategy for Continuous Parametric Optimization

Researchers : Manuel II M. Caballero
Denzel Jay B. Benjamin
Christian Nickhos A Divina

Placed: Room 318 **Date:** September 10, 2026 **Time:** 1:30 PM - 3:00 PM

FINAL DEFENSE COMMITTEE	ACTION TAKEN
EMMANUEL ISAIAH Z. DETERA, MIT Panel Member	_____
CHRISTIAN M. LANA, MIT Panel Member	_____

RHODORA FAYE A. BROSAS, MBA, MIT

Adviser / Thesis Coordinator

THESIS 2 COMPLETION

Project Title : HYDE: Hybrid Differential Evolution with Warm-start Increasing Population Covariance Matrix Adaptation Evolution Strategy for Continuous Parametric Optimization

Researchers : Manuel II M. Caballero
Denzel Jay B. Benjamin
Christian Nickhos A. Divina

Degree Program : Bachelor of Science in Computer Science

Final Defense Completed on: September 10, 2026

THESIS COMMITTEE

FINAL DEFENSE COMMITTEE	SIGNATURES
EMMANUEL ISAIAH Z. DETERA, MIT Panel Member	_____
CHRISTIAN M. LANA, MIT Panel Member	_____
RHODORA FAYE A BROSAS, MBA, MIT Adviser	_____
JP REMAR A. SERRANO, MIS Chairperson	

CERTIFICATE OF APPROVAL

This is to certify that the Thesis Project of:

Manuel II M. Caballero

Denzel Jay B. Benjamin

Christian Nickhos A. Divina

has been approved by the Thesis Committee in partial fulfillment of the requirements of the degree of Bachelor of Science in Computer Science, School of Engineering and Computer Studies.

RHODORA FAYE A. BROSAS, MBA, MIT

Adviser / Thesis Coordinator

ENGR. MARBEN S. RAMOS, LPT, MAPhysEd

Dean

EXECUTIVE SUMMARY

Project Title : HYDE: Hybrid Differential Evolution with Warm-start Increasing Population Covariance Matrix Adaptation Evolution Strategy for Continuous Parametric Optimization

Researchers : Manuel II M. Caballero
Denzel Jay B. Benjamin
Christian Nickhos A. Divina

Keywords: : Differential Evolution, IPOP-CMA-ES, CMA-ES, Hybrid Optimization, Warm-Start, Continuous Parametric Optimization, Benchmarking

Continuous parametric optimization problems are multi-modal, non-separable, and computationally expensive, and they often lack reliable gradient information, which led to continuous efforts to create population-based metaheuristics capable of handling these challenges through stochastic operations and collective learning techniques.

This study extended HyGO, a hybrid framework with four structural limitations: (i) binary encoding limited to 4,096 values per parameter, (ii) memoryless local search that discards information between runs, (iii) performance degradation with increasing dimensionality, and (iv) unvalidated performance claims.

HyDE addressed these through a two-phase architecture combining Differential Evolution (DE) global search with warm-start IPOP-CMA-ES local refinement, using

continuous (HyDE-CON), quantum-inspired (HyDE-QUB), and binary (HyDE-BIN) encodings.

The study followed the Experimental Algorithmic Methodology, evaluating on 20 analytical scenarios inherited from HyGO with 50 independent runs and a common budget of 5,000 evaluations (50,000 for high-dimensional problems). Analyses used Friedman with Nemenyi post-hoc, Kruskal-Wallis, Cochran's Q, and Wilcoxon rank-sum tests, with Cliff's delta effect sizes and Bootstrap confidence intervals.

Results showed that all three HyDE variants significantly improved over HyGO. HyGO did not achieve the lowest mean objective error in any scenario, converged in only 25.5% of trials on average, and degraded significantly from 2D to 25D in 4/5 scalable functions. HyDE-QUB was five times faster than HyGO, achieved the highest solution quality, had a 73.1% mean convergence rate, and was the only variant demonstrating scalability to higher dimensions, with a large effect size (Cliff’s $d = -0.954$). The extended budget increased HyDE-QUB’s convergence to 88.4%, while HyGO showed limited improvement, indicating that HyGO’s limitations are structural rather than budget-related. HyDE-CON and HyDE-BIN remained competitive in speed-critical and lower-dimensional scenarios, respectively.

Future work may validate HyDE in real engineering applications, examine boundary cases at extended budgets, and extend the framework to discrete, mixed-variable, and multi-objective problems.

ACKNOWLEDGEMENT

As they say, a thesis is never written alone. And after what this project did to us, we cannot agree more. All of the hard work and achievements that we have now are thanks to people who encouraged us, led us, and dragged us to the finish line. And so, to everyone out there, we have some gratitude in our hearts, and words will never be enough to fully express it.

Almighty God, for all the blessings he sent down for us so that we can be able to finish this study; all glory is given solely to him.

Rhodara Faye A. Brosas, MBA, MIT, our thesis adviser, for taking up the mantle of guiding and leading this study into its successful conclusion. For all the constructive criticism and guidance, as well as all the patient listening and help whenever we are in doubt, we are eternally thankful. **Reilan L. Cadubla**, our former adviser, for laying out all the necessary foundations and believing in the importance of our study even when it had no concrete form yet. It is because of you we managed to reach this point where we are at now.

Emmanuel Isaiah Z. Detera, MIT and **Christian M. Lana, MIT,** the panel members, for the useful advice, help, and assistance whenever we needed to ask. Fifthly, to our panel of examiners for the tough questions and even harsher criticism. It was hard to hear at times, but it helped make this study so much better.

Computer Science and Information Technology Society (CSITS) for providing free printing service and for organizing the thesis colloquium.

To **Merla M. Caballero** and **Manly O. Caballero**, parents of **Manuel II M. Caballero**, thank you for your unlimited love, patience, and prayers, and for the emotional and financial support you gave throughout this journey.

To **Joel B. Benjamin** and **Aileen B. Benjamin**, parents of **Denzel Jay B. Benjamin**, thank you for your unconditional love, patience, and constant support, especially during the times when I wanted to give up.

To **Jose L. Divina Jr.** and **Epifania A. Divina**, parents of **Christian Nickhos A. Divina**,thank you for being my source of strength and inspiration. Your prayers, patience, and unwavering support carried me through every challenge in completing this research.

To our partners, for the late night calls, patience when we were glued to our computers rather than spending time with you, and cheerleading through the difficult process of doing this work.

Finally, to our friends and classmates for all the craziness we went through, for sharing our struggles and worries, and for the encouragement in times when we felt that we could not do this anymore.

M. M. C.

D. J. B. B.

C. N. A. D

TABLE OF CONTENTS

Content Page **Page No.**

Title Page i

Recommendation of Thesis Final Defense ii

Result of Final Defense iii

Thesis 2 Completion iv

Certificate of Approval v

Executive Summary vi

Acknowledgement viii

Table of Contents x

List of Figures xiii

List of Tables xiv

Chapter

1 INTRODUCTION

Background of the Study 1

Research Gap 5

Theoretical Framework 6

Conceptual Framework 8

Description of the Existing Algorithm 10

Description of the Algorithm 11

Statement of the Problem 11

Objectives of the Study 12

Purpose and Description 13

Technical Terms 14

Chapter 2 METHODOLOGY

Research Design 17

Algorithm Development 18

Purpose, Deliverables, and Development Activities . . 23

Scope and Delimitation 24

Data Gathering Techniques 25

Source of Data 26

Theorems, Algorithms, and Mathematical Models . . 27

Chapter 3 RESULTS AND DISCUSSIONS

Results 48

Findings 80

Proposed Solution 83

Materials and Statistical Tools 85

Flowchart 87

System Requirements 88

Hardware Requirements 88

Software Requirements 89
System Tradeoffs 90

Chapter

4 SUMMARY OF FINDINGS, CONCLUSIONS AND RECOMMENDATIONS

Findings 94
Conclusions 95
Recommendations 96

References 98

Appendicies

A Gantt Chart 109
B Source Code 110
C Benchmark Results 120
D Simulations 121

Curriculum Vitae 150

LIST OF FIGURES

Figure No.	Title	Page No.
1	HyGO's Algorithm Framework	6
2	IPO Framework of HyDE Algorithm	8
3	Experimental Algorithmic Methodology	18
4	Best Algorithm per Scenario by Mean Objective Error .	57
5	Convergence Rate per Benchmark Across 50 Independent Runs	61
6	Average Wall-Clock Time Across All Benchmarks . .	64
7	Mean Wall-Clock Time per Benchmark Scenario . .	65
8	Significant Wins from Wilcoxon Rank-Sum Test, Each HyDE Variant vs. HyGO	68
9	Bootstrap 95% CI on Mean Difference: HyDE-BIN vs. HyGO	69
10	Bootstrap 95% CI on Mean Difference: HyDE-QUB vs. HyGO	70
11	Bootstrap 95% CI on Mean Difference: HyDE-CON vs. HyGO	71
12	Coefficient of Variation at 25D per Algorithm for Each Scalable Benchmark Function	76
13	Degradation Ratio Heatmap: Mean 25D / Mean 2D . .	77
14	HyDE-Qub Algorithm	83
15	HyGO Algorithm Flowchart	87

LIST OF TABLES

Table No.	Title	Page No.
2.1	Activities of the Experimental Algorithmic Methodology .	23
2.2	Algorithm 1: HyDE Pseudo-Code	29
2.3	Algorithm 2: HyDE-bin Stagnation Recovery — Bit-Flip Perturbation	34
2.4	Algorithm 3: HyDE-qub Stagnation Recovery — Quantum Tunnelling	35
2.5	Algorithm 4: HyDE-con Stagnation Recovery — Gaussian Perturbation	36
2.6	Analytical Benchmark Functions: Dimensionality Variants .	41
3.1	Nemenyi Post-Hoc Pairwise Comparisons Following the Friedman Test	54
3.2	Per-Scenario Kruskal-Wallis Results Across All Four Algorithms	55
3.3	Per-Scenario Kruskal-Wallis Results at the 50,000 -Evaluation Budget for 25D Scenarios	58
3.4	Per-Scenario Convergence Counts and Chi-Square Test of Independence Across the Four Algorithms . . .	59
3.5	Per-Scenario Convergence Counts and Chi-Square Test at the 50,000- Evaluation Budget	62
3.6	Wall-Clock Cost Summary with Speedup Ratios Relative to HyGO	63

3.7 Wall-Clock Cost Summary with Speedup Ratios Relative to HyGO at the 50,000-Evaluation Budget . . . 66
3.8 Cliff's Delta Effect Size Distribution for Each HyDE Variant vs. HyGO 67
3.9 Wilcoxon Rank-Sum Test with Cliff's Delta and Bootstrap 95% CI at the 50,000-Evaluation Budget for 25D Scenarios 72
3.10 Dimensionality Scaling Analysis: Wilcoxon Test, Cliff's Delta, Coefficient of Variation, and Direction of Change from 2D to 25D 74
3.11 Dimensionality Scaling Analysis at the 50,000-Evaluation Budget 78
3.12 Materials Table 86
3.13 Statistical Tools Table 86
3.14 Hardware Requirements and Specification . . . 89
3.15 Software Requirements and Specifications . . . 89
3.16 Technical Issues 90
3.17 Operational Issues 91
3.18 Economical Issues 92

Chapter 1

INTRODUCTION

BACKGROUND OF THE STUDY

Continuous parametric optimization problems are multi-modal, non-separable, and computationally expensive, and they often lack reliable gradient information. They are typically encountered in controller tuning, structural sizing, aerodynamic shape design, and heat exchanger geometry (Liang et al., 2024; Sa et al., 2025). Several recent studies have shown that standard optimization methods are likely to fail to identify feasible solutions in such problem domains, which led to continuous efforts to create population-based metaheuristics capable of handling these challenges through stochastic operations and collective learning techniques (Shaban et al., 2025; Chauhan et al., 2025). Comparative testing using large algorithm portfolios confirmed that there is no clear winner among optimization methods across different classes of problems, as indicated by the No Free Lunch theorem and shown in more recent benchmarking experiments (Shaban et al., 2025; Altay et al., 2024; Naser et al., 2025).

The ongoing quest for optimizers capable of striking a balance between global exploration and local exploitation resulted in hybrid evolutionary frameworks becoming one of the most popular solutions in this area (Cao & Luan, 2024; Salgotra & Gandomi, 2024). This study made its own contribution by developing a hybrid optimization framework targeting continuous parametric optimization.

This study sought to contribute to the achievement of two UN Sustainable Development Goals. With respect to SDG 9 (Industry, Innovation and Infrastructure),

improved parametric optimization efficiency led to faster design cycles and lower costs associated with computational and energy expenditure during design iterations, contributing to the construction of resilient and efficient infrastructures (Sa et al., 2025; Dadios et al., 2018). With regard to SDG 4 (Quality Education), this study aimed to enhance the quality of evidence-based optimization studies within the Philippine academic setting through the adoption of experimental methods and statistical analysis with effect size reporting (Kononova et al., 2025).

Genetic Algorithm (GA) is a population-based metaheuristic inspired by natural evolution, in which candidate solutions go through selection, crossover, and mutation over several generations (Katoch et al., 2021). It was used in antenna array design (Alejandrino et al., 2024) and machine learning parameter estimation (Jamilla et al., 2021; Malonzo et al., 2024), and remains among the most popular metaheuristic methods employed in optimization problems. It is also the foundation of several other hybrid algorithms, such as those used for the optimization of structural and topological parameters (Wang et al., 2025) and for the optimization of energy usage in smart farms, which the Philippine engineering community was working on (De Ocampo & Dadios, 2017).

Differential Evolution (DE), proposed by Storn and Price (1997), is a population-based stochastic method that utilizes difference vectors for mutation, crossover, and selection of real-valued candidate solutions. It showed excellent performance in various international optimization competitions and consistently ranked

among the best performers in IEEE Congress on Evolutionary Computation benchmark tests (Brest et al., 2016; Stanovov et al., 2021; Salgotra & Gandomi, 2024). Recent developments included multi-hybrid DE techniques in structural design (Salgotra & Gandomi, 2024), dynamic strategy hybridization of DE and PSO algorithms (Xu et al., 2025), and comprehensive reviews of multi-modal DE applications (Chauhan et al., 2025). DE and other bio-inspired metaheuristics were also successfully employed in various Philippine engineering studies to estimate cosmological parameters (Bernardo et al., 2025), develop evapotranspiration models (Mendigoria et al., 2023), and optimize parameters in high dimensions (Enriquez et al., 2022). DE has several strategies, but DE/current-to-best/1/bin is one of the most commonly used for continuous optimization problems and was evaluated in this study as the global search metaheuristic.

Local search methods provide the exploitation capability that complements population-based global search in hybrid frameworks (Cao & Luan, 2024; Shaban et al., 2025). The Downhill Simplex Method (DSM), introduced by Nelder and Mead (1965), is a gradient-free local search algorithm that iteratively refines a geometric simplex through reflection, expansion, contraction, and shrinkage operations, and it remains one of the most popular local search components in evolutionary hybrids due to its simplicity and gradient-free operation (Fajfar et al., 2019).

HYGO (Hybrid Genetic Optimisation), introduced by Robledo et al. (2025), is a recent hybrid optimization framework relevant to this study. It integrates a binary-encoded Genetic Algorithm with the Downhill Simplex Method in an interleaved

exploration-exploitation cycle, alternating between genetic operations for global search and DSM-based refinement of elite candidates each generation. HYGO unifies parametric and functional learning under a single architecture and was evaluated on analytical benchmark suites and engineering applications such as the Ahmed body drag-reduction problem. HYGO served as the primary baseline against which the proposed algorithm was evaluated.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is an optimization method that improves solutions by updating a probability distribution based on successful search steps. The algorithm is rotation invariant and works well on difficult and high-dimensional problems (Hansen & Ostermeier, 2001; Hansen, 2016) where the search space is non-linear or noisy. The IPOP-CMA-ES algorithm (Auger & Hansen, 2005) is an improvement to CMA-ES that increases the population size when no improvement is made and explores the search space, and it is used as a typical restart technique (Nomura et al., 2023).

Another improvement is Warm-start CMA-ES, which uses the results of previous runs to initiate the process close to optimal solutions (Nomura et al., 2021). Together with hybrid optimization frameworks (HYGO) and restart techniques, these algorithms combine global and local search strategies for better performance on hard multimodal optimization problems (Loshchilov et al., 2013), which renders the CMA-ES algorithms useful in real-world optimization problems in engineering and machine learning, among others.

RESEARCH GAP

The latest hybrid algorithm tested on the same puzzle dataset in this paper was developed by Robledo et al. (2025) and named HyGO. Although HyGO made substantial progress compared to classical evolutionary algorithms, it still has limitations related to its architecture. First, HyGO represents the solutions through ones and zeros, and therefore the number of possible values is limited; according to Almufti (2025), binary encoding placed an inevitable limitation on precision, regardless of the efficiency of the fine-tuning procedure. Second, HyGO used the Downhill Simplex Method as a fine-tuning method, but since this tool had no memory between restarts and discarded all directions it had already tried, it structurally could not compensate for the limitation of the encoding.

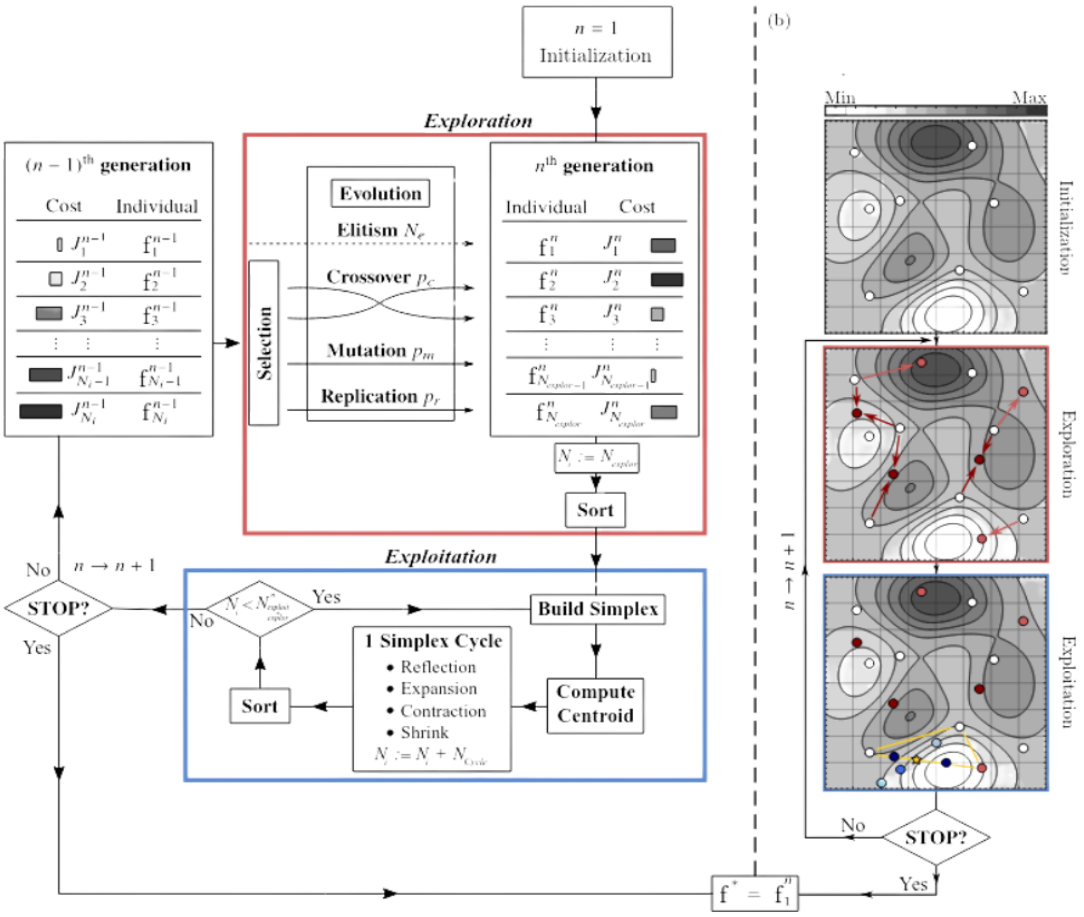
Third, the limited encoding and the absence of memory in the optimization method makes the algorithm's performance worse with each increase in problem difficulty; this scalability issue has not been addressed. Finally, the comparisons made in the HyGO paper could not be trusted, as they used only medians and success rates and did not employ effect sizes such as Cliff's delta, meaning that the improvements in performance could not be confirmed. All these limitations suggest that HyGO's performance advantages are both technically limited and statistically unverified.

The four limitations led the researchers to develop a more adaptive, statistically rigorous hybrid framework to address HyGO's structural weaknesses. This motivated the development of a more well-founded and reliable alternative.

THEORETICAL FRAMEWORK

Figure 1

HyGo's Algorithm Framework



The theoretical framework of this study is grounded in the HyGO (Hybrid Genetic Optimization) Framework, as illustrated in Figure 1, which serves as the conceptual and architectural foundation upon which the proposed algorithm was designed and evaluated. HyGO demonstrated how two fundamentally distinct search paradigms, exploration and exploitation, could be systematically integrated within a single generational lifecycle,

rather than being applied in isolation as in traditional approaches. The lifecycle starts with the initialization of a population (Generation 0), where candidate solutions are created to represent the initial search space.

Following this, the algorithm moves to the exploration stage, where the creation of offspring by genetic operators such as selection, crossover, and mutation allows the algorithm to explore complex search spaces without converging prematurely. Next comes the refinement stage, where the most promising candidates are refined using local search techniques to improve their fitness values. The lifecycle continues until one or more termination criteria are met and the optimal candidate solution is provided.

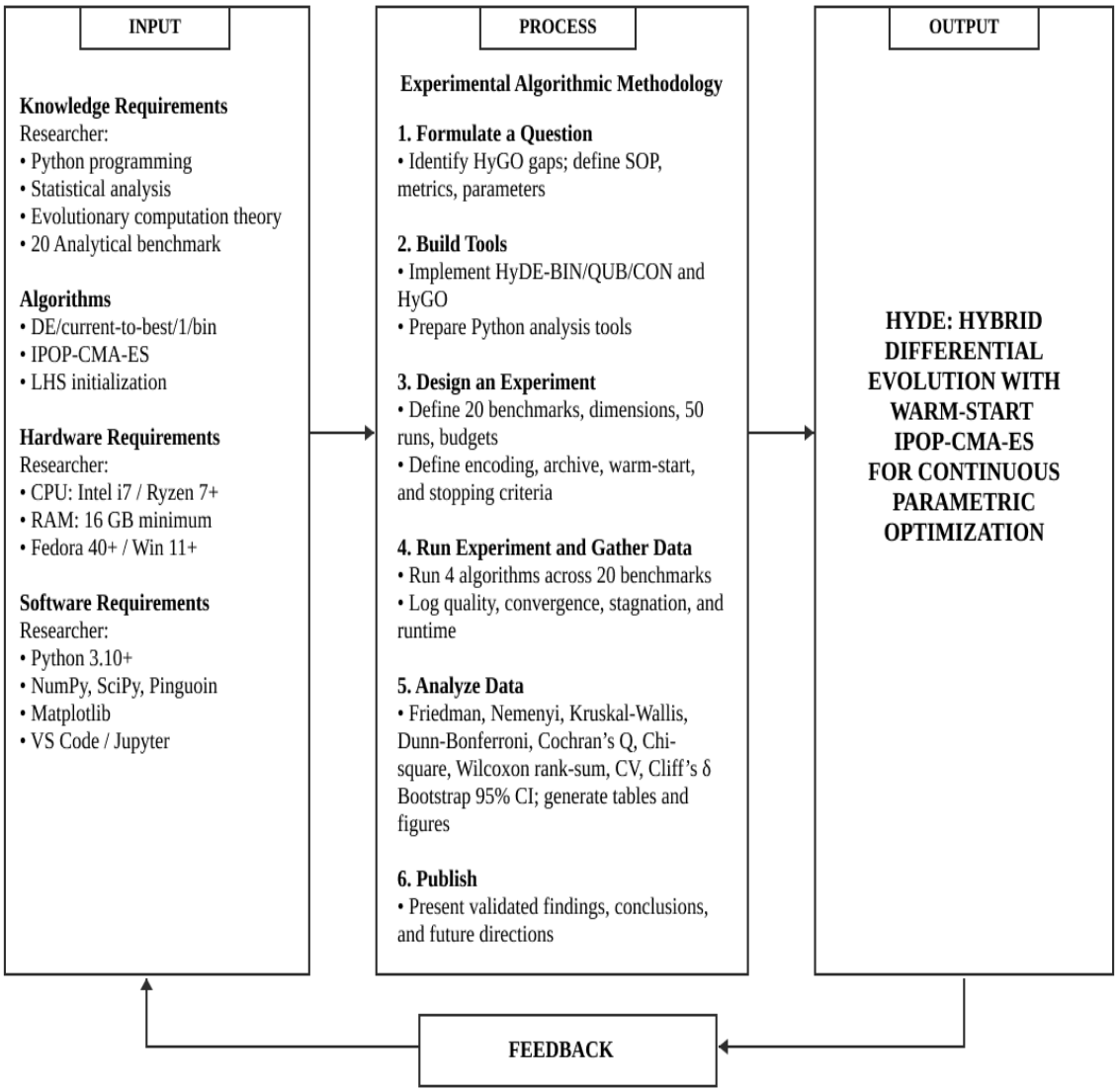
HyGO served as the theoretical and structural framework of the proposed algorithm, which further improved upon HyGO in three ways. These include directed global search (whereby the operator selection process is biased to explore promising areas of the search space), warm-start local search (whereby the refinement process is initialized using the previous best-known solution), and encoding-consistent stagnation recovery (whereby stagnation is detected and perturbations in the form of structurally sound solutions are introduced).

The contributions enhanced the efficiency, scalability, and robustness of the algorithm, tested on multiple criteria including solution quality, speed, and stagnation resistance. Each enhancement in HyDE addressed a specific, documented weakness of HyGO. Together they formed a cohesive design that was empirically validated against established benchmarks to demonstrate improvements over its predecessor.

CONCEPTUAL FRAMEWORK

Figure 2

IPO Framework of Hyde Algorithm



The conceptual framework in Figure 2 outlines the components used to develop and evaluate HyDE, a two-phase hybrid optimization algorithm combining Differential Evolution and warm-start IPOP-CMA-ES for continuous parametric optimization.

The input includes the researchers' knowledge of Python, evolutionary computation, optimization, statistical analysis, benchmark functions, Differential Evolution, IPOP-CMA-ES, encoding schemes, and Latin Hypercube Sampling. Hardware requirements include an Intel Core i7 or AMD Ryzen 7 processor with at least 16 GB RAM, while software requirements include Fedora 40+ or Windows 11, Python 3.10+, NumPy, SciPy, Pingouin, and Matplotlib.

The process follows the six stages of the Experimental Algorithmic Methodology: formulate a question, build tools, design an experiment, run the experiment and gather data, analyze data, and publish. The first stage identified the performance gaps of HyGO and established the research questions, metrics, and experimental parameters. The second stage implemented the HyDE variants and HyGO and prepared the computational and statistical tools. The third stage defined the benchmark functions, problem dimensions, independent runs, evaluation budget, encoding methods, and experimental conditions. The fourth stage conducted the experiments and recorded solution quality, convergence, stagnation, and runtime data. The fifth stage analyzed the gathered data using statistical tests and measures. The final stage presented and disseminated the results, conclusions, and identified research gaps.

The final output was the proposed HyDE algorithm and the experimental findings from its evaluation against HyGO across the selected analytical benchmark functions. The feedback component allows the findings and identified limitations to guide future improvements and studies in continuous parametric optimization.

DESCRIPTION OF THE EXISTING ALGORITHM

Hybrid Genetic Optimisation (HyGO) is the most recent hybrid optimization scheme. Introduced by Robledo et al. (2025), HyGO aims to solve complex problems by integrating an advanced local search technique within a binary-encoded genetic algorithm. The working process alternates between an exploratory phase and an exploitative phase in each generation. Individuals are generated using Monte Carlo sampling or Latin Hypercube Sampling to form the initial population and are then encoded in binary. The exploration process performs global search using elitism, tournament selection, crossover, and mutation. The exploitation process improves the best candidates using local search procedures on an analytical benchmark suite. The scheme surpassed the genetic algorithm on many benchmark problems. These experimental results for HyGO in comparison with the genetic algorithm justified the potential of a hybrid optimization approach.

At higher dimensions, and as problem complexity increases, deficiencies in the procedure emerge that cause HyGO to fall short as a robust and efficient optimization technique. The alternating exploration and exploitation approach proves insufficient in many cases, especially with complicated problems. The results indicated that HyGO did not effectively handle problems with many dimensions. The encoding, as well as the local search through DSM, appeared to be responsible for these failures in difficult scenarios. The performance differences identified established the need for a more robust optimization method.

DESCRIPTION OF THE ALGORITHM

The proposed hybrid optimization framework involves two stages. The framework includes a directed global search stage that directs each run toward the best solutions while maintaining diversity of solutions in the search process by keeping a set of all well-performing solutions throughout the optimization. Additionally, the framework includes an automatic stagnation recovery stage that detects stagnation in the search process and attempts a pre-specified local search method immediately afterward to restart the search. Finally, the framework involves an informed local search stage that warm-starts the local search methods with the knowledge learned during the previous search until the global optimum is found or the evaluation budget is exhausted, whichever occurs first, ensuring efficient use of resources.

STATEMENT OF THE PROBLEM

This study aimed to address the limitations and gaps identified in HyGO, a hybrid evolutionary framework introduced by Robledo et al. (2025), in achieving exact convergence and true optimization performance across both low- and high-dimensional problem scenarios. Specifically, this study sought to answer the following questions:

1. What were the computational, statistical, and performance limitations of the HyGO algorithm in solving optimization problems?

2. What potential improvements or enhancements could be proposed to address the identified limitations of HyGO?

3. How did the three variants of HyDE compare against each other and against

HyGO when tested on different benchmark problems, and which performed best in terms of:

a. Mean final objective error across the 20 benchmark scenarios;

b. Convergence rate across 50 independent runs;

c. Wall-clock cost per run;

d. Practically meaningful margin by which each variant improved upon HyGO; and

e. Performance as problem dimensionality scales from 2D to 25D across the five shared scalable benchmark functions?

OBJECTIVES OF THE STUDY

The objective of this study was to develop and evaluate a two-phase hybrid optimization framework across three encoding variants, tested against HyGO on various benchmark scenarios. Specifically, this study aimed to:

1. Identify the computational, statistical, and performance limitations of the HyGO algorithm that affected its optimization performance across low- and high-dimensional problem scenarios.

2. Come up with potential improvements and enhancements to address the identified limitations of HyGO through the development of the proposed framework and its three encoding variants (HYDE-BIN, HYDE-QUB, HYDE-CON).

3. Compare the three variants of the proposed framework against each other and against HyGO across benchmark problems to determine which variant performed best.

PURPOSE AND DESCRIPTION

The study aimed to develop a two-phase hybrid optimization framework combining three encoding variants to address the limitations of HyGO and to evaluate its performance across various benchmark scenarios of increasing complexity. Specifically, this study offered benefits to the following:

Research Community on Evolutionary Computation. This study contributed to the evolutionary computation literature by providing a controlled comparison of three encoding variants within a single unified hybrid framework, helping to fill the gap in encoding comparison research.

Research Community on Hybrid Optimization. This study contributed to the hybrid optimization literature by comparing two different hybrid approaches, HyGO and the proposed framework, under the same conditions and providing empirical evidence on which approach performed better.

Researchers. This study was useful for researchers in developing technical hybrid evolutionary algorithm design and experimental methods through the development and testing of the proposed framework across three encoding variants. It also contributed to the body of knowledge on effective hybrid design choices in continuous optimization.

Future Researchers. This study provided a robust and adaptable framework that could be extended to new problem domains, encodings, or local search methods, serving as a strong baseline or reusable template for future hybrid algorithm development and comparative studies on continuous parametric optimization problems.

TECHNICAL TERMS

The following technical terms were defined both in their general conceptual sense and in the specific operational context of this study for clarity and precision.

Analytical Benchmark Functions - a suite of mathematical test functions with known global optima, characterized by diverse topological properties such as multimodality, non-separability, and ill-conditioning, used to evaluate the convergence behavior of optimization algorithms under controlled conditions (Katoch et al., 2021). In this study, various benchmark scenarios were used across specific dimensions under a shared evaluation budget.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES) - an evolutionary strategy that adapts a multivariate Gaussian to the fitness function by accumulating information about the history of successful search steps into a covariance matrix (Hansen & Ostermeier, 2001). In this study, CMA-ES was used in Phase 2 as the local refinement engine, with a warm covariance seeded from the Phase 1 elite archive.

Convergence - the state in a population-based algorithm where the best fitness value of the population reaches or comes close to optimal or near-optimal values with each evaluation (Das et al., 2016). In the proposed framework, convergence was determined when the best fitness value did not improve over a certain number of generations within the specified tolerance.

Differential Evolution (DE) - a population-based stochastic search procedure that improves candidate solutions by mutation, crossover, and greedy selection. In this

work, the DE/current-to-best/1/bin variant was used as the global search operator with an adaptive scale factor and binomial crossover (Storn & Price, 1997).

Downhill Simplex Method (DSM) - a gradient-free local search algorithm that repeatedly modifies a geometric shape of vertices in an N-dimensional parameter space using reflection, expansion, contraction, and shrinkage (Nelder & Mead, 1965). DSM was used as the local search method of HyGO in each generation, starting from the current best solutions.

Elite Archive - a collection of the best solutions assessed during an optimization run, stored to keep track of high-quality solutions throughout the generations (Mohamed et al., 2019). In the proposed framework, the elite archive was maintained during Phase 1 and transferred to Phase 2.

Evaluation Budget - the upper limit on the number of objective function evaluations in an optimization run. It acts as a major constraint in optimization problems where each evaluation of the objective function is computationally expensive (Das et al., 2016). In this study, the evaluation budget was fixed for the benchmark scenarios under the experimental protocol used, and it was distributed across benchmark scenarios and shared between Phase 1 and Phase 2.

Hybrid Genetic Optimizer (HyGO) - a hybrid optimization framework that employs a binary-encoded Genetic Algorithm and the Downhill Simplex Method in an interleaved architecture (Robledo et al., 2025). It served as the baseline algorithm for the comparative analysis, against which the proposed framework and its three encoding

variants were evaluated under the same conditions across analytical benchmark functions.

Increasing Population Covariance Matrix Adaptation Evolution Strategy (IPOP-CMA-ES) - a CMA-ES version whose population size is doubled at each restart to systematically increase the search when stagnation occurs (Auger & Hansen, 2005). In the proposed framework, it was used in Phase 2 by updating the mean with the mean of the elite archive at each restart until the maximum number of evaluations was reached.

Latin Hypercube Sampling (LHS) with Farthest-Point Reordering - a sampling method that divides each parameter dimension into equal intervals and places exactly one sample per interval, ensuring uniform coverage of the search space (McKay et al., 1979). In this study, farthest-point reordering further maximized the minimum distance between samples. LHS with farthest-point reordering was used to initialize the population for evaluation.

Warm Covariance - an initialization strategy for CMA-ES where the covariance matrix is seeded from elite solutions collected during a search phase instead of an uninformative identity matrix (Nomura et al., 2021). In the framework, this combined the Phase 1 archive with ordered matrices for stability.

Chapter 2

METHODOLOGY

This chapter presents the research design, experimental methodology, data gathering techniques, sources of data, and the algorithms and mathematical models that serve as the basis for design and evaluation of the proposed algorithm.

RESEARCH DESIGN

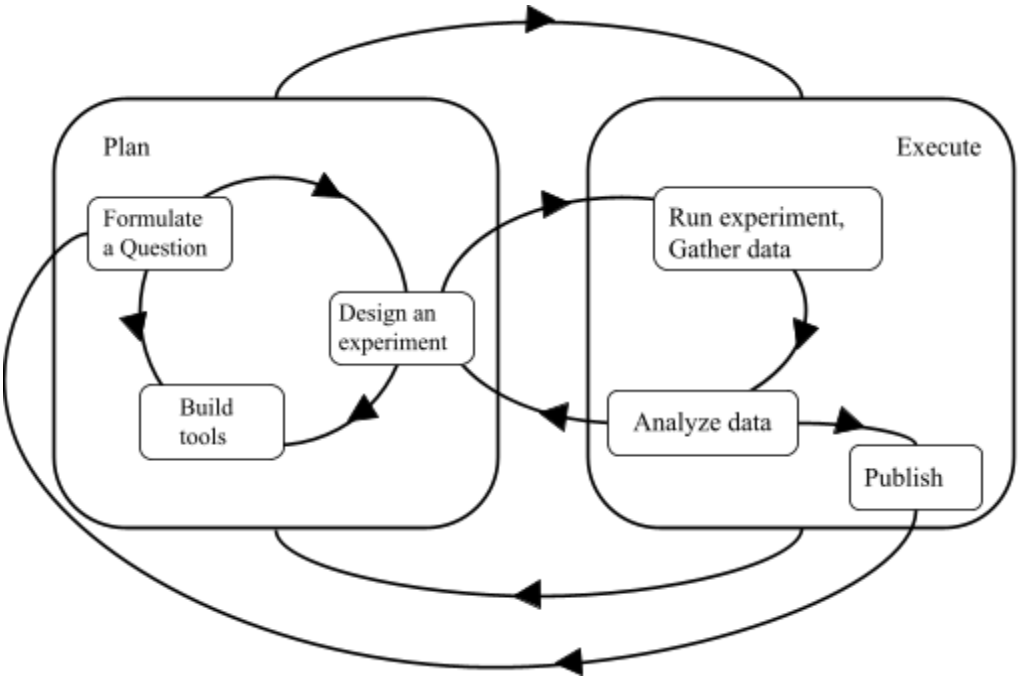
This study used an experimental research design approach, where the researchers conducted a series of specific experiments to test and compare the performance of three proposed algorithm variants and the existing HyGO algorithm across different problem scenarios. This design was suited to the study because the study aimed to identify the limitations of HyGO and propose improvements, and measure how each of the algorithms performed using specific performance measures.

The experiments were implemented across various benchmark problems to ensure that the comparison between the algorithms was fair, and covered a range of difficulty levels. Through this approach, the researchers were able to observe how each algorithm behaved across various scenarios. The results gathered from the experiments were analyzed and compared to answer the specific questions that the researchers presented in their study, and to determine which algorithm presented an effective and reliable solution in solving simple and complex problem scenarios. Furthermore, this experimental design ensures scientific and reproducibility, allowing future researchers to validate, or build upon the findings of this study using the same structured methodology.

ALGORITHM DEVELOPMENT

Figure 3

Experimental Algorithmic Methodology



As shown in Figure 3 below, Experimental Algorithmic Methodology is an approach to developing and evaluating algorithms through computational experiments. It combines actual testing with theoretical analysis to determine which algorithms, implementations, and improvements work well for specific problems. Its main purpose is to understand algorithm behavior through systematic experimentation and statistical analysis (McGeoch, 2012). It provides a flexible, organized way of studying algorithms through a cycle of planning, testing, analyzing, and improving, allowing researchers to move between planning and execution stages as needed.

Experimental Algorithmic Methodology was selected as the most appropriate approach for the proposed algorithm because it focuses on testing, evaluating, and improving algorithm performance. Since the proposed algorithm is a two-phase hybrid optimization algorithm that combined Differential Evolution and IPOP-CMA-ES, repeated experimentation was needed to determine whether it performed consistently across different optimization problems. Through this process, the researchers were able to identify the strengths and weaknesses of the proposed algorithm and make improvements based on the results. The following are the phases of McGeoch's Experimental Algorithmic Methodology:

Plan. The planning stage serves as the foundation of the experiment. It consists of three sub-steps: formulating a question, building the necessary tools, and designing the experiment. These steps may have been performed in different orders depending on the needs of the study (McGeoch, 2012).

Formulate a Question. The first step is to identify the specific question that the experiment would answer. This includes defining the problem through the objective function, decision variables, and constraints, as well as identifying the performance measures and experimental settings needed for the study (McGeoch, 2012).

For this study, the researchers formulated the experimental questions based on the gaps and limitations identified in the HyGO study by Robledo et al. (2025). The study particularly focused on problems observed in HyGO, such as stagnation and reduced efficiency when solving high-dimensional problems. The researchers then defined the

optimization problem using a mathematical formulation that established the scope and direction of the study. This formulation helped determine whether the proposed algorithm and its three variants could address the identified limitations. It also provided the basis for comparing the three variants in terms of convergence speed, solution quality, and reliability, which helped identify the most effective variant.

Build Tools. The second step involves preparing the tools and environment needed to conduct the experiment. This includes the algorithm being tested, the input problems, tools for measuring performance, and software for analyzing the results (McGeoch, 2012).

Before the experiment was conducted, the researchers developed the two-phase hybrid algorithm in Python and prepared the necessary tools for collecting, visualizing, and comparing the results. These tools included NumPy, SciPy, Matplotlib, Pingouin, and Pandas. Before the actual experiment, the researchers tested the algorithm implementations to make sure that they worked properly and produced correct results.

Design an Experiment. The third step is to design an experiment that addressed the research question established. This involves specifying the properties to be involved, specifying properties to be measured, the categories and sizes of the input problems to be used, and the number of random trials to be conducted, among other considerations (McGeoch, 2012).

In this study, the researchers designed the experiment by deciding how fast the algorithm converged, how good its solutions were, and how well it recovered when it got

stuck. The algorithm was tested on benchmark mathematical problems, similar to that used by Robledo et al. (2025), consisting of small to large problems so the researchers could see how the algorithm performed as problems got harder. Each problem was run 50 times with different random starting points, since the algorithm's behavior could vary from run to run, and running it many times gave more reliable results. Each run was stopped after the budget 5,000 function evaluation at solving the 2D problems, and stopped after 50,000 evaluations at 25D scenarios. These choices were meant to make sure the experiment produced enough data to answer the research questions.

Execute. The execution stage follows the planning stage. It involves running the experiment, collecting the results, analyzing the data, and presenting the findings (McGeoch, 2012).

Run Experiment and Gather Data. The first step in the execution stage is to conduct the experiment according to the planned design and collect the data produced by each trial (McGeoch, 2012).

In this study, the proposed algorithm and its variants were tested under the same experimental conditions on all selected benchmark optimization problems. During each trial, the researchers recorded the results needed to evaluate the algorithm, including convergence speed, solution quality, and its ability to recover from stagnation. Data were recorded after every run to ensure that the results were complete and accurate. The collected data were then organized according to the algorithm variant and benchmark problem to make the results easier to compare and analyze.

Analyze Data. The second step involves analyzing the collected data to identify useful findings and determine whether the research questions had been sufficiently answered. If the results were not sufficient, the researchers may have returned to the planning stage and revised the experiment (McGeoch, 2012).

After completing the trials, the researchers analyzed the collected data to evaluate the performance of the three variants of the proposed algorithm. The analysis focused on convergence speed, solution quality, and the ability to recover from stagnation. The researchers also compared the three variants with HyGO across the 20 benchmark problems and determined how each performed in low and high-dimensional settings. Based on the results, the researchers identified which variant provided the most consistent performance. If the collected data was insufficient or did not provide clear results, the researchers returned to the planning stage and reviewed the experimental design, measurement methods, or evaluation criteria. The experiment may then have been repeated until enough reliable data was obtained to answer the research question.

Publish. The final step is to present and share the validated results of the experiment. According to McGeoch (2012), the results may have also led to new questions that could begin another cycle of experimentation.

For this study, the researchers presented the validated findings through the research paper. The results showed the performance of the algorithm and its variants and determined whether they were able to address the identified limitations of HyGO. The findings also provided a basis for future studies on hybrid optimization algorithms.

Purpose, Deliverables, and Development Activities

Table 2.1

Activities of the Experimental Algorithmic Methodology

Phase	Purpose	Deliverables	Development Activities
Formulate a Question	Identify the specific research questions, examine the gaps and limitations in HyGO.	Formally specified research questions, performance metrics, and experimental parameters.	Examine the gaps identified in the HyGO study. Define the problem through mathematical formulation. Establish the performance metrics that will govern the study.
Build Tools	Assemble the necessary tools and test environment needed before the experiment begins.	Python implementation of proposed algorithm variants, and configured measurement and data analysis tools.	Develop the test algorithm using Python. Set up NumPy, SciPy, Matplotlib, and Pandas. Test and verify all tools before the experiment.
Design an Experiment	Specify what properties will be measured, what benchmark problems will be used, how many trials will be run, and the stopping criterion.	Verified experimental protocol, benchmark problem categories, trial specifications, and stopping criterion.	Specify the properties of the proposed algorithm to be measured, including the benchmark problems, number of trials, and stopping criterion, following Robledo et al. (2025). Verify all tools and inputs are working correctly.
Run Experiment and Gather Data	Execute all benchmark runs and systematically collect all output data for analysis.	Collected output data from all trials including convergence rate, solution quality, and stagnation recovery performance.	Execute the algorithms across all selected benchmark problems. Collect all output data during the experiment. Record and store all results from every trial properly.

Analyze Data	Apply statistical analysis to the collected data and determine whether the results answer the research questions.	Statistical analysis results, summary tables, and convergence curves.	Examine all collected data from the experiment. Determine whether results answer the specific research questions. Return to the planning stage if results are inconclusive.
Publish	Disseminate the validated results and present clear conclusions about the proposed algorithm's performance.	Published research findings, conclusions about the proposed algorithm's performance, and identified directions for future research.	Distribute the validated results through research papers. Present clear and reliable conclusions about the proposed algorithm's performance. Identify new questions and directions for future research.

SCOPE AND DELIMITATION

This study centered on the design, development, and experimental evaluation of a hybrid two-phase optimization algorithm that employed DE/current-to-best/1/bin as the Phase 1 global search engine and IPOP-CMA-ES with warm covariance seeding as the Phase 2 local optimizer. Three encoding variants were implemented, including HyDE-qub, HyDE-bin, and HyDE-con, each integrated with an encoding-consistent stagnation recovery mechanism designed to restore search progress. The hybrid two-phase optimizer was benchmarked on 20 selected analytical optimization benchmark scenarios, all inherited from Robledo et al. (2025) with 50 independent runs under a shared evaluation budget for subsequent direct comparison with HyGO, and because they comprehensively represented the full spectrum of landscape features considered relevant for continuous optimization, thus representing a well-established and reliable basis for meaningful and fair algorithmic benchmarking.

The study did not cover discrete, combinatorial or multi-objective optimization. The study was limited to unconstrained and box-constrained continuous optimization in the 20 analytical benchmark problems. The experiment did not cater for real-world engineering applications, such as the drag reduction analysis for the Ahmed body, adaptive phase split adjustments, or third-party software interfaces such as ANSYS Fluent, mainly due to the limited monetary resources to run the experiment. The use of mutation strategies or restart variants from other EAs such as Bipopulation CMA-ES or Matrix Adaptation Evolution Strategy was out of the scope of this thesis.

DATA GATHERING TECHNIQUES

The two main data gathering techniques were employed throughout this study for collecting data uniformly in order to analyze the computational, statistical and performance constraints of the HyGO algorithm and to suggest logical potential improvements and particular modifications to the algorithm that could completely eliminate the constraints and verify the proposed framework empirically with the baseline HyGO for all benchmarks.

Document Analysis. This was conducted to identify the computational, statistical and performance limitations of the HyGO algorithms. By closely examining the paper of HyGO, the researchers extracted the algorithmic design, benchmark results, experimental methodology, and parameter settings of HyGO. The information gathered provided a clear understanding of HyGO's structural gaps and served as the direct basis for replicating the experimental protocol in the benchmarking phase of this study.

Experimental Benchmarking. This tested whether the proposed enhancements addressed the identified limitations of HyGO. The researchers followed the experimental methodology of Robledo et al. (2025) so that the proposed framework and HyGO were evaluated under identical, directly comparable conditions. By executing the three encoding variants and the reimplemented HyGO baseline across all scenarios, they generated the performance data needed to determine how well the framework addressed the gaps identified through document analysis and which encoding variant was most suitable for continuous parametric optimization.

SOURCES OF DATA

The study collected data from document analysis and experimental benchmarking to ensure that the findings are accurate, reliable, and well-supported.

For document analysis, the source of data was the HyGO paper published by Robledo et al. (2025). The researcher examined the reported benchmark results, algorithmic design, parameter settings, and experimental methodology of HyGO to identify its computational, statistical and performance limitations. Additional sources included peer-reviewed journals, conference proceedings, and preprints from IEEE Xplore, ScienceDirect, Springer, and arXiv, from which the mathematical formulations, search domains, and known global optima of all 20 analytical benchmark functions were drawn to support the theoretical framework of the study.

For experimental benchmarking, the source of data was the results generated by executing the three encoding variants and the reimplemented HyGO baseline across all

20 benchmark scenarios adopting the same experimental methodology of Robledo et al. (2025), under a shared evaluation budget of 5,000 function evaluations with 50 independent runs per configuration. Additionally, the evaluation budget was extended to 50,000 function evaluations if convergence did not produce relevant results for this study, in particular for 25D problems. The HyGO baseline was reimplemented from scratch based on the pseudocode published by Robledo et al. (2025) in order to evaluate both frameworks in the same experimental setting. All output (mean final objective error, mean best cost, evaluations to convergence and wall-clock time) was stored systematically for analysis.

THEOREMS, ALGORITHMS, AND MATHEMATICAL MODELS

This section lays the foundation of this study. The justification of algorithmic design is substantiated by theorems, the description of the different HYDE algorithm variants were presented. The benchmark functions used to benchmark performance across different landscape topologies were also defined.

1. THEOREMS

1.1 No Free Lunch Theorem

Wolpert and Macready (1997) showed mathematically that for any two algorithms A_1 and A_2, their performance averaged over the entire space of all possible objective functions is the same. They proved this as follows:

$$\sum_f P(d_m^y \mid f, m, A_1) = \sum_f P(d_m^y \mid f, m, A_2)$$

where d_m^y is the sequence of cost values after m evaluations. The implication of this theorem is that no one algorithm could dominate all others on all problem classes. This result provided the primary motivation for the fundamental design choices made by HyDE. Having three encoding schemes instead of one, evaluating on 20 benchmark problems with different topologies, and hybridizing CMA-ES with DE were all theoretically motivated by this theorem.

1.2 Linear Convergence of CMA-ES

Hansen and Ostermeier (2001) established the strictly convex quadratic function

$$f(x) = \frac{1}{2} x^\top H x$$

with positive definite Hessian H, CMA-ES with cumulative step-size adaptation achieves linear convergence such that

$$E\left[|x^{(g)} - x^*|^2\right]^{1/2} \leq c \cdot \rho^g, \quad \rho \in (0,1)$$

The convergence rate ρ improves monotonically as the covariance matrix C adapts toward the scaled inverse Hessian H^{-1} across successive generations, reaching its optimal value when

$$C = \frac{H^{-1}}{\det(H^{-1})^{1/D}}$$

This theorem provides the formal theoretical basis for HyDE's warm covariance initialization, where C_{archive} seeded from the Phase 1 elite archive places C closer to H^{-1} at the start of Phase 2 than any cold-start initialization could achieve, accelerating

entry into the linear convergence regime. Critically, Lagarias et al. (1998) established that the Downhill Simplex Method carried no equivalent convergence guarantee for dimensions $D \geq 2$, meaning CMA-ES was not merely empirically superior to DSM as a Phase 2 engine but probably so under conditions directly relevant to optimization.

2. ALGORITHMS

2. 1 HyDE Algorithm

This section presents the HyDE algorithm, a two-phase hybrid optimization framework designed to address the four structural limitations of HyGO (Robledo et al., 2025). Its complete pseudocode is presented in Table 2.2.

Table 2.2

Algorithm 1: HyDE Pseudo-Code

Line	Operation
	— Initialisation —
1	Set $P1_{budget} \leftarrow \lfloor \varphi \times M \rfloor$
2	Generate N individuals via LHS with farthest-point reordering → x_r^1, r = 1, 2, …, N
3	Encode each individual into the chosen representation → z_r = Encode(x_r)
4	Evaluate the fitness of each individual → J(x_r; θ), r = 1, 2, …, N
5	Initialise archive $\mathscr{U} \leftarrow$ top N_arc individuals sorted by cost
6	Set x_best ← arg $\min_r$ J(x_r; θ), s ← 0, N_eval ← N
	— Phase 1: Differential Evolution Global Search —
7	while N_eval < P1_budget and N_eval < M do
8	for i = 1 to N do
9	Select $r_1, r_2 \in \{1, \ldots, N\}$ where $r_1 \neq r_2 \neq i$
10	Generate scaling factor $F_i \leftarrow 0.5 + 0.3 \times \mathrm{rand}$
11	Compute mutant → $v_i = z_i + F_i(z_{best} - z_i) + F_i(z_{r_1} - z_{r_2})$
12	Clip v_i to encoding bounds
13	Apply binomial crossover (CR) between z_i and v_i → u_i
14	Decode trial individual → x_trial = Decode(u_i)

```
        Evaluate fitness → J_trial = J(x_trial; θ)
        if J_trial ≤ J(x_i; θ) then
            z_i ← u_i, x_i ← x_trial, J_i ← J_trial
        end if
        Update archive 𝒰 with (x_trial, J_trial)
    end for
    N_eval ← N_eval + N
                        — Stagnation Detection and Recovery —
    if J_best improved then
        s ← 0, update x_best
    else
        s ← s + 1
    end if
    if s ≥ s_trigger then
        Select worst ⌊N/2⌋ individuals by fitness
        Apply encoding-specific recovery to selected individuals:
            Binary → random bit-flip perturbation (Algorithm 2)
            Qubit → quantum tunnelling (Algorithm 3)
            Continuous → Gaussian perturbation (Algorithm 4)
        Evaluate recovered individuals, update archive 𝒰
        s ← 0
    end if
end while
                    — Phase 2: IPOP-CMA-ES with Warm Covariance —
Compute C_archive from scatter of archive 𝒰
Set C ← 0.5·C_archive + 0.5·I·(tr(C_archive)/D)
Set mean m ← best solution from archive 𝒰
Set σ ← mean(std(𝒰)), λ ← λ_base, k ← 0
while N_eval < M and k < k_max do
    Compute CMA-ES parameters μ, w, c_s, d_s, c_c, c_1, c_μ from λ and D
    Decompose C = BD²B^T via eigendecomposition
    while N_eval < M and remaining budget ≥ λ do
        Sample λ offspring: X = clip(m + σ(Z · D)B^T, lo, hi)
        Evaluate offspring, N_eval ← N_eval + λ
        Sort offspring by cost, select top μ
        Update mean: m_new ← w^T · X_sel
        Update evolution paths p_s and p_c
        Update covariance: C ← (1 − c_1 − c_μ)C + c_1 p_c p_c^T + c_μ Σ w_j a_j a_j^T
        Update step size: σ ← σ · exp((c_s/d_s)(||p_s||/χ - 1))|σ ← σ ·
exp((c_s/d_s)(||p_s||/χ - 1))|σ ← σ · exp((c_s/d_s)(||p_s||/χ - 1))
        if σ < 10^-11 or C not finite then break
```

53 end while
54 k ← k + 1
55 λ ← min(2λ, M − N_eval)
56 if λ < 4 then break
57 Reset C ← I, select new mean from archive $\mathcal{U}$
58 end while
— Termination —
59 Return the best solution found → x* = arg min J(x; θ)

Initialization (Lines 1–6). The algorithm begins by computing the Phase 1 budget as P1_budget = floor(φ × M), where φ ∈ (0, 1) controls the fraction of the total evaluation budget M allocated to global search (Line 1). This single parameter replaces the manual parameter decomposition required by HyGO-Stepped (Robledo et al., 2025, Table 5), making the framework fully automatic. The population of N individuals is generated using Latin Hypercube Sampling with farthest-point reordering (Line 2), which ensures uniform coverage of the search space before any evaluations are spent, replacing HyGO's Monte Carlo Sampling which does not guarantee stratified coverage (McKay et al., 1979).

Each individual is encoded into the chosen representation (Line 3), evaluated (Line 4), and an elite archive A is initialized from the top N_arc individuals sorted by cost (Line 5). This archive accumulates the best solutions found throughout Phase 1 and later provides the geometric information needed to warm-start CMA-ES in Phase 2, following Nomura et al. (2021). $P1_{\text{budget}} = \lfloor \varphi \times M \rfloor$, where $\varphi \in (0, 1)$ controlled the fraction of the total evaluation budget M allocated to global search (Line 1). This single parameter replaces the manual parameter decomposition required by HyGO-Stepped (Robledo et al., 2025, Table 5), making the framework fully automatic. The initial

population of individuals is created using Latin Hypercube Sampling with farthest-point reordering (Line 2) to encourage exhaustive search of the design space prior to any evaluations, replacing HyGO's Monte Carlo Sampling that does not guarantee the stratified design of experiment coverage (McKay et al., 1979). Each individual is then translated into the desired representation (Line 3), evaluated (Line 4), and an archive of top individuals is initialized by sorting by cost (Line 5). The archive aggregates the top individuals found during Phase 1 and is later used to provide the geometrical information to warm-start CMA-ES in Phase 2 in accordance with Nomura et al. (2021), rather than a brute-force transition.

Phase 1: Differential Evolution Global Search (Lines 7-36). This phase utilizes DE/current-to-best/1/bin until the total budget or Phase 1 budget is depleted (Line 7). Differential Evolution was selected over the Genetic Algorithm used in HyGO because DE uses directional mutation rather than the unbiased crossover typical of the Genetic Algorithm in HyGO. The crossover operator in HyGO simply combined two parent chromosomes without favoring the best known solution; thus it depends on chance for convergence (Robledo et al., 2025). DE's mutation strategy explicitly favors the best known solution in each trial through the term F_i(z_best - z_i), while maintaining diversity in the population through the random difference term F_i(z_r1 - z_r2) (Storn & Price, 1997).

For each individual (Lines 8-20), two different individuals r 1 and r 2 are selected at random (Line 9) and a per-individual adaptive scaling factor z i is generated (Line 10),

varying between 0.5 and 0.8 to avoid fixed-step stagnation following Brest et al. (2016). Two other individuals z r 1 and z r 2 are selected at random and two individuals r 1 and r 2 (Line 11), clipped to encoding bounds (Line 12), and combined with the current individual via binomial crossover with probability CR (Line 13). This trial is decoded (Line 14), evaluated (Line 15), and only replaces the current individual if it improves or ties the cost (Lines 16-18). The archive is updated at every trial regardless of acceptance (Line 19). The sequential nature of the architecture allows the full 1st Phase 1 budget to be spent on unrestricted global exploration before any local search begins, directly addressing HyGO's unconditional first-generation DSM (Robledo et al., 2025), while preserving the remaining budget for refinement and convergence.

The mutant vector $v_i = z_i + F_i(z_{\text{best}} - z_i) + F_i(z_{r_1} - z_{r_2})$ is computed (Line 11), clipped to encoding bounds (Line 12), and combined with the current individual via binomial crossover with probability CR (Line 13). The trial is decoded (Line 14), evaluated (Line 15), and replaces the current individual only if it improves or matches the cost (Lines 16–18). The archive is updated with every trial regardless of acceptance (Line 19). The sequential architecture ensures the full Phase 1 budget is spent on unrestricted global exploration before any local search begins, directly addressing HyGO's unconditional first-generation DSM (Robledo et al., 2025).

Table 2.3

Algorithm 2: HyDE-bin Stagnation Recovery — Bit-Flip Perturbation

Line	Operation
1	n_t ← max(1, floor(N/2))

```
2   worst ← indices of n_t individuals with highest fitness in F
3   for each idx in worst do
4     c ← copy of C[idx]
5     for p = 1 to D do
6       bit_pos ← random integer in {0, 1, …, N_b − 1}
7       c[p × N_b + bit_pos] ← c[p × N_b + bit_pos] ⊕ 1
8     end for
9     C[idx] ← c
10    X[idx] ← Decode(c)
11  end for
12  Evaluate recovered individuals, update archive 𝒰
13  s ← 0
```

The bit-flip recovery perturbs each worst individual by flipping exactly one randomly selected bit per parameter in its binary chromosome (Lines 5–8). The XOR operation $\oplus$ on Line 7 toggles a single bit, changing the decoded value by an amount determined by the bit's position: a high-order bit produces a large jump, a low-order bit produces a fine adjustment. This operates directly on the binary representation without decoding and re-encoding, which would introduce rounding artifacts from the 2nb on Line 7 toggles a single bit, changing the decoded value by an amount determined by the bit's position: a high-order bit produces a large jump, a low-order bit produces a fine adjustment. This operates directly on the binary representation without decoding and re-encoding, which would introduce rounding artifacts from the 2^{N_b} discretization grid. Flipping one bit per parameter ensures every dimension is perturbed. After flipping, the chromosome is decoded (Line 10) and re-evaluated.

Table 2.4

Algorithm 3: HyDE-qub Stagnation Recovery — Quantum Tunnelling

Line	Operation

```
1   n_t ← max(1, floor(N/2))
2   worst ← indices of n_t individuals with highest fitness in F
3   strengths ← linearly spaced values from 0.2 to 1.0 over n_t points
4   for k = 1 to n_t do
5      idx ← worst[k], s ← strengths[k]
6      for j = 1 to D do
7         θ[idx, j] ← (1-s)·θ[idx, j] + s·(π/2 - θ[idx, j])
8      end for
9      θ[idx] ← θ[idx] + 0.02·N(0, I)
10     θ[idx] ← clip(θ[idx], 0, π/2)
11     X[idx]←lo+sin²(θ[idx])·width
12  end for
13  Evaluate recovered individuals, update archive 𝒰
14  s ← 0=
```

The quantum tunnelling recovery reflects each worst individual's qubit angles toward π/4 with a variable strength increasing linearly from 0.2 to 1.0 (Line 3), applying the mildest perturbation to the least-worst and the strongest to the absolute worst. The reflection on Line 7, θ_new = (1 - s)·θ + s·(π/2 - θ), interpolates between the current angle and its mirror about π/4. At full strength this is simplified to θ_new = π/2 - θ, mapping sin²(θ) to 1 - sin²(θ) and sending the parameter to the opposite side of the domain. For symmetric bounds, π/4 corresponds to the domain midpoint since sin²(π/4) = 0.5, making tunnelling through this center a structured exploration that random reinjection could not replicate (Han & Kim, 2002). A small Gaussian noise of 0.02 is added (Line 9) to prevent collapsed duplicates, and angles are clipped to [0, π/2] (Line 10) before observation (Line 11). with a variable strength increasing linearly from 0.2 to 1.0 (Line 3), applying the mildest perturbation to the least-worst and the strongest to the absolute worst. The reflection on Line 7, , interpolated between the current angle and its

mirror about . At full strength this is simplified to , mapping to and sending the parameter to the opposite side of the domain. For symmetric bounds, it corresponds to the domain midpoint since , making tunnelling through this center a structured exploration that random reinjection could not replicate (Han & Kim, 2002). A small Gaussian noise of 0.02 is added (Line 9) to prevent collapsed duplicates, and angles are clipped to (Line 10) before observation (Line 11).

Table 2.5

Algorithm 4: HyDE-con Stagnation Recovery — Gaussian Perturbation

Line	Operation
1	n_t ← max(1, floor(N/2))
2	worst ← indices of n_t individuals with highest fitness in F
3	σ ← std(X, axis=0)
4	σ ← max(σ, 0.01 × width)
5	for each idx in worst do
6	X[idx] ← X[idx] + σ·N(0, I)
7	X[idx] ← clip(X[idx], lo, hi)
8	end for
9	Evaluate recovered individuals, update archive $\mathcal{U}$
10	s ← 0

The Gaussian perturbation recovery adds noise scaled by the population spread to each worst individual (Line 6). σ is the per-dimension standard deviation of the population (Line 3), making the recovery adaptive: large perturbations when spread out, small when converged. A floor of 1% of the parameter range (Line 4) prevents the perturbation from becoming negligibly small.

This is the natural perturbation for continuous encoding because additive Gaussian noise preserves the continuous representation and is the same model CMA-ES

used for sampling, making it conceptually consistent with Phase 2. is the per-dimension standard deviation of the population (Line 3), making the recovery adaptive: large perturbations when the population was spread out, small when converged. A floor of 1% of the parameter range (Line 4) prevented the perturbation from becoming negligibly small. This is the natural perturbation for continuous encoding because additive Gaussian noise preserves the continuous representation and is the same model CMA-ES used for sampling, making it conceptually consistent with Phase 2.

The three recovery mechanisms are not interchangeable. Gaussian perturbation applied to binary chromosomes would require decoding, perturbing, and re-encoding, which snaps the result to the nearest 2^N_b grid point and could undo the perturbation. Bit-flip perturbation applied to qubit angles has no meaningful interpretation in rotation space and would violate the [0, π/2] normalization. Quantum tunnelling applied to continuous vectors would reflect about the domain midpoint, which has no relationship to the landscape for non-symmetric problems. Each mechanism is operated natively on its representation, ensuring the perturbation is valid and non-degenerate.

This encoding-recovery consistency addressed the open issue in the DE literature regarding the interaction between representation and stagnation recovery, which had not been investigated under a unified framework on a parametric analysis problem.2^{N_b} grid point and could undo the perturbation. Bit-flip perturbation applied to qubit angles had no meaningful interpretation in rotation space and would violate the $[0, \pi/2]$ normalization. Quantum tunnelling applied to continuous vectors would reflect about the domain

midpoint, which had no relationship to the landscape for non-symmetric problems. Each mechanism operated natively on its representation, ensuring the perturbation was valid and non-degenerate. This encoding-recovery consistency addressed the open issue in the DE literature regarding the interaction between representation and stagnation recovery, which had not been investigated under a unified framework on a parametric problem.

Phase 2: IPOP-CMA-ES with Warm Covariance (Lines 37–58). Upon exhausting the Phase 1 budget, Phase 2 initializes CMA-ES with geometric information from the archive rather than from scratch. The covariance is computed as C = 0.5·C_archive + 0.5·I·(tr(C_archive) / D) (Lines 37–38), blending archive scatter with a regularized identity matrix. The mean is set to the best archive solution (Line 39) and the step size to the mean archive standard deviation (Line 40), following Nomura et al. (2021).$C = 0.5 \cdot C_{\text{archive}} + 0.5 \cdot I \cdot (\text{tr}(C_{\text{archive}})/D)$ (Lines 37–38), blending archive scatter with a regularized identity matrix. The mean is given to the optimal archive solution (Line 39) and the step size to the mean archive standard deviation (Line 40) following Nomura et al. (2021).

CMA-ES was preferred to DSM as it learns the landscape geometry from its covariance matrix that stores the success step history (Hansen, 2016), in contrast to DSM, which does not remember anything from one iteration to the next (Nelder & Mead, 1965).Robledo et al. (2025) validated this, CMA-ES being superior to HyGO for the last step refining phase because of the path memory. The cold-start remedy brought by the warm-start counterbalanced the cold-start drawback identified by Robledo et al. (2025) in

the standalone CMA-ES case, where the covariance had to be learned from an uninformative identity matrix.

During execution (Lines 41–58), CMA-ES samples λ offsprings from $N(m, \sigma^2 C)$, clipped to bounds (Line 45). The top μ are selected (Lines 46–47), and the mean, evolution paths p_s and p_c, covariance C, and step size σ are updated per Hansen and Ostermeier (2001) (Lines 48–51). Numerical safeguards terminate the inner loop if $\sigma < 10^{-11}$ or C is non-finite (Line 52). On stagnation, the IPOP protocol doubles population (Line 55), resets covariance to identity and redraws the mean from the archive (Line 57), saving all Phase 1 geometric knowledge across restarts (Auger & Hansen, 2005). From , clipped to bounds (Line 45). The top are selected (Lines 46-47), and mean, evolution paths and , covariance and and step size are updated as in Hansen and Ostermeier (2001) (Lines 48-51). Immediate termination of the inner loop occurs if or is not finite (Line 52). On stagnation, the IPOP protocol again automatically doubles population (Line 55), resets covariance to identity, and redraws the mean from the archive (Line 57), saving all Phase 1 geometric knowledge efficiently across restarts (Auger & Hansen, 2005).

Termination (Line 59). The algorithm terminates by returning the best solution in the two phases. The design choices in HyDE paralleled the four structural limitations of HyGO identically; DE/current-to-best/1/bin replaced the undirected GA crossover with directional mutation, IPOP-CMA-ES replaced the memoryless DSM with a covariance-accumulating local search engine. The sequential architecture eliminated evaluation waste from HyGO's unconditional first-generation DSM (Limitation 3), and

the automatic phase transition governed by φ removed the need for manual parameter decomposition required by HyGO-Stepped (Limitation 4).

MATHEMATICAL BENCHMARK MODEL

The 15 benchmark test functions used in this study comprised five categories of landscapes for parametric optimization. Sphere and Matyas tested exploitation on smooth, unimodal landscapes. Rosenbrock and Beale tested exploitation/exploration balance on non-separable landscapes with curved valleys. Rastrigin, Ackley, Styblinski-Tang, and Levi N.13 tested global exploration on multimodal landscapes with local minima. Eggholder, Holder Table, Himmelblau's, and Goldstein-Price tested exploration on irregular multimodal landscapes with competing basins.

Easom and Bukin N.6 tested exploration on extreme cases such as extensive flatlands or deceptive ridges. The five functions tested in 25D (Ackley, Sphere, Rastrigin, Rosenbrock, Styblinski-Tang) were selected by Robledo et al. (2025) because they are scalable to any dimensionality and collectively simulate unimodal, multimodal, and valley-shaped landscapes under the curse of dimensionality. The other 10 functions were tested in 2D only because they are defined in two variables. Using the same set of problems as Robledo et al. (2025) guaranteed that the HyDE-vs.-HyGO comparisons were performed on the same benchmarks under the same conditions. Formulations, search domains, and tested dimensions are listed in Table 2.2, along with descriptions.

Table 2.6

Analytical Benchmark Functions: Dimensionality Variants

Function	Formula	Tested Dimensions
Ackley	$f(x) = -20\cdot\exp(-0.2\sqrt{(\sum x_i^2/n)}) - \exp(\sum\cos(2\pi x_i)/n) + 20 + e$	2D, 25D
Beale	$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$	2D
Booth	$f(x, y) = (x + 2y - 7)^2 + (2x + y - 5)^2$	2D
Bukin N.6	$f(x, y) = 100\sqrt{\lvert y - 0.01x^2\rvert} + 0.01\lvert x + 10\rvert$	2D
Easom	$f(x, y) = -\cos(x)\cdot\cos(y)\cdot\exp(-(x - \pi)^2 - (y - \pi)^2)$	2D
Eggholder	$f(x, y) = -(y + 47)\cdot\sin(\sqrt{\lvert x/2 + (y + 47)\rvert}) - x\cdot\sin(\sqrt{\lvert x - (y + 47)\rvert})$	2D
Goldstein-Price	$f(x, y) = [1 + (x + y + 1)^2(19 - 14x + 3x^2 - 14y + 6xy + 3y^2)] \times [30 + (2x - 3y)^2(18 - 32x + 12x^2 + 48y - 36xy + 27y^2)]$	2D
Himmelblau's	$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$	2D
Holder Table	$f(x, y) = -\lvert\sin(x)\cdot\cos(y)\cdot\exp(\lvert 1 - \sqrt{(x^2 + y^2)}/\pi\rvert)\rvert$	2D
Lévi N.13	$f(x, y) = \sin^2(3\pi x) + (x - 1)^2[1 + \sin^2(3\pi y)] + (y - 1)^2[1 + \sin^2(2\pi y)]$	2D
Matyas	$f(x, y) = 0.26(x^2 + y^2) - 0.48xy$	2D
Sphere	$f(x) = \sum x_i^2$, $i = 1, ..., n$	2D, 25D
Rastrigin	$f(x) = 10n + \sum[x_i^2 - 10\cos(2\pi x_i)]$, $i = 1, ..., n$	2D, 25D
Rosenbrock	$f(x) = \sum[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]$, $i = 1, ..., n - 1$	2D, 25D
Styblinski-Tang	$f(x) = \frac{1}{2}\sum(x_i^4 - 16x_i^2 + 5x_i)$, $i = 1, ..., n$	2D, 25D

Ackley Function. The Ackley function (Ackley, 1987) is a common multi-modal test function with a nearly flat outer region with many local minima, and a large hole at the middle representing the global minimum, with standard constants $a=20$, $b=0.2$, $c=2\pi$ as originally proposed. It tests the ability of an algorithm to escape shallow local minima to find a relatively small basin. It was tested in 2D and 25D because the number of local minima increases with dimensionality, thus providing a good test of scalability. The search domain is $[-32.768,32.768]^n$ with global minimum $f(x^*)=0$ at the origin. The function is defined as:

$$f(x)=a\exp!\left(-b\sqrt{\frac{1}{n}\sum_{i=1}^{n}x_i^2}\right)-\exp!\left(\frac{1}{n}\sum_{i=1}^{n}\cos(cx_i)\right)+a+e$$

Beale Function. The Beale function (Beale, 1958) is a unimodal, non-separable 2D polynomial for which coefficients form a curved valley requiring the optimizer to traverse non axis-aligned gradients. It tests the ability to exploit non-separable variable interaction. It would only be tested in 2D as it was limited to be defined for two variables. The search domain is $[-4.5,4.5]^2$ with global minimum $f(3,0.5)=0$. The function is defined as:

$$f(x,y)=(1.5-x+xy)^2+(2.25-x+xy^2)^2+(2.625-x+xy^3)^2$$

Booth Function. The Booth function (Booth, 1949) is a straightforward convex function with one global minimum and smooth derivatives in any given direction. It acts as a fundamental test to distinguish which optimisers perform well on very simple functions (all of them should perform well) and which fail. It was only used in 2D. The search domain is $[-10,10]^2$ with global minimum $f(1,3)=0$. The function is defined as:

$$f(x,y)=(x+2y-7)^2+(2x+y-5)^2$$

Bukin N.6 Function. The Bukin N.6 function (Bukin, 1995) has a narrow ridge of near-optimum values which run along a parabola. This is one of the most difficult 2D benchmark functions as the ridge topology implies that many points give similar costs, making it impossible to tell whether the global minimum has been found when close by.

Robledo et al. (2025) found zero convergence for both HyGO and GA on this function so it is a good test as to whether the directional mutation of HyDE provided any benefits here, and it was only tested in 2D. The search domain is $x_1 \in [-15,-5]$, $x_2 \in [-3,3]$ with global minimum $f(-10,1)=0$. The function is defined as:

$$f(x,y)=100\sqrt{|y-0.01x^2|}+0.01|x+10|$$

Easom Function. The Easom function (Easom, 1990) is a huge, almost flat, plateau with a solitary sharp valley at the global minimum. The huge search space compared to the size of the global basin makes it a bare exploration test, since an algorithm has to pluck off many samples from a very large space to discover the one tiny region where progress could be made. It was tested only in 2D. The search domain is $[-100,100]^2$ with global minimum $f(\pi,\pi)=1$. The function is defined as:

$$f(x,y)=(y+47)\sin!\sqrt{\left|\tfrac{x}{2}+(y+47)\right|}-x\sin!\sqrt{|x-(y+47)|}$$

Eggholder Function. The Eggholder function (Simon, 2013) is one of the most difficult 2D benchmarks for local minima, as many exist in the large domain, with depths similar to the global minimum. The test is whether an algorithm could sample the basin of the global minimum so it could be found, within many competing basins of similar depths. It was tested in 2D. The search domain is $[-512,512]^2$ with global minimum $f(512,404.2319)=959.6407$. The function is defined as:

$$f(x,y)=(y+47)\sin!\sqrt{\left|\tfrac{x}{2}+(y+47)\right|}-x\sin!\sqrt{|x-(y+47)|}$$

Goldstein-Price Function. The Goldstein-Price function (Goldstein & Price,

1971), is the product of two polynomial functions that have several local minima in a bounded domain. The bounded domain and steep gradient test the local search component for its ability to take advantage of steep gradients once it is in the vicinity of a promising area. It was tested in 2D only. The search domain is $[-2,2]^2$ with global minimum $f(0,-1)=3$. The function was defined as:

$$f(x,y)=\left[1+(x+y+1)^2(19-14x+3x^2-14y+6xy+3y^2)\right]$$

$$\times\left[30+(2x-3y)^2(18-32x+12x^2+48y-36xy+27y^2)\right]$$

Himmelblau's Function. Himmelblau's function (Himmelblau, 1972) contains four global minima of equal value. Having four optimum solutions measures whether the algorithm could reliably converge to one solution instead of bouncing between basins. Himmelblau's function was only tested in 2D. The search domain is $[-5,5]^2$ with global minimum $f(x^*)=0$ at $(3,2)$, $(-2.805,3.131)$, $(-3.779,-3.283)$, and $(3.584,-1.848)$. The function is defined as:

$$f(x,y)=(x^2+y-11)^2+(x+y^2-7)^2$$

Holder Table Function. Holder Table function (Mishra, 2006) has four global minima. Similar to Himmelblau's, the Himmelblau's function has more than one global optimum, but the landscape is more irregular with many local minima of different depth. It was used to test explorative breadth as well as the discriminative power to reveal global minima among deep local minima. It was used in 2D only. The search domain is $[-10,10]^2$ with global minimum $f(\pm 9.6459,\pm 9.6459)=19.2085$. The function is

defined as:

$$f(x,y)=\left|\sin(x)\cos(y)\exp!\left(\left|1-\frac{\sqrt{x^2+y^2}}{\pi}\right|\right)\right|$$

Levi N.13 Function. The Levi N.13 function (Lévi, 1981) has trigonometric terms that create a grid of regularly spaced local minima across the domain, making it a test of whether the algorithm could navigate through periodic traps to reach the global minimum. It was tested in 2D only. The search domain is $[-10,10]^2$ with global minimum $f(1,1)=0$. The function is defined as:

$$f(x,y)=\sin^2(3\pi x)+(x-1)^2\left(1+\sin^2(3\pi y)\right)+(y-1)^2\left(1+\sin^2(2\pi y)\right)$$

Matyas Function. The Matyas function (Matyas, 1965) is a smooth, bowl-shaped quadratic function with a single minimum and almost zero gradients close to optimal solution. The function tests the accuracy of the local search part of the algorithm when the gradients are extremely shallow. It was only tested in 2D. The search domain is $[-10,10]^2$ with global minimum $f(0,0)=0$. The function is defined as:

$$f(x,y)=0.26(x^2+y^2)-0.48xy$$

Sphere Function. The Sphere function (De Jong, 1975), (also known as De Jong's Function 1) is the simplest benchmark, being convex, separable and unimodal with the same gradients emanating out from the origin. It acts as a sanity check, demonstrating the pure exploitation potential of the algorithms uninfluenced by multimodality or non-separability, and was tested in both 2 and 25 dimensions. Robledo et al. (2025) demonstrated that HyGO and CMA-ES achieved comparable low-cost function values,

while GA was lagging behind. The search domain is $[-5.12,5.12]^n$ with global minimum $f(x^*)=0$ at the origin. The function is defined as:

$$f(x)=\sum_{i=1}^{n} x_i^2$$

Rastrigin Function. The Rastrigin function (Rastrigin, 1974) has a cosine term that creates a regular grid of local minima whose number grows exponentially with dimensionality and is scalable to dimensions. This is the primary test of global exploration capability because the algorithm has to consistently avoid deceptive local traps to find the single global minimum. It was tested in both 2D and 25D. In 25D, Robledo et al. (2025) found that HyGO outperformed both GA and CMA-ES, making it a key scenario for evaluating whether HyDE's warm-start CMA-ES could match or exceed this advantage across increasing problem complexity. The search domain was with global minimum at the origin. The function is defined as:

$$f(x)=An+\sum_{i=1}^{n}\left[x_i^2-A\cos(2\pi x_i)\right],\ A=10$$

Rosenbrock Function. The Rosenbrock function, also known as the Banana function (Rosenbrock, 1960), is characterized by its narrow valley whose global optimum is at the bottom. Valley identification is straightforward, whereas converging within the valley towards the bottom is non-trivial, as the function has vanishing gradients along the valley. It was evaluated in 2D and 25D. It is defined over the search space with the global optimum at the minimum. The function is:

$$f(x)=\sum_{i=1}^{n-1}\left[100(x_{i+1}-x_i^2)^2+(1-x_i)^2\right]$$

Styblinski-Tang Function. The Styblinski-Tang function (Styblinski & Tang, 1990) yields two local minima of unequal depths for each dimension, creating 2^n local minima in n dimensions. The global minimum occurs at $x_i=-2.9035$ in each dimension. This tests the algorithm's ability to consistently select the deeper minimum in each dimension rather than settling for the shallower one. It was tested in both 2D and 25D. The search domain is $[-5,5]^n$ with global minimum $f(x^*)=-39.1662n$. The function is defined as:

$$f(x)=\frac{1}{2}\sum_{i=1}^{n}\left(x_i^4-16x_i^2+5x_i\right)$$

Chapter 3

RESULTS AND DISCUSSIONS

In this chapter, the results, findings, proposed solutions, material and statistical tools used, system requirements, and system tradeoffs that should be considered in order to complete the project are outlined.

RESULTS

This section presents the experimental results of HyGO and the proposed two-phase hybrid optimization framework with its three encoding variants. It covers HyGO's performance limitations, the performance of the proposed variants, and their comparative results against each other and HyGO, supported by statistical analysis, tables, and figures.

Computational, Statistical, and Performance Limitations of HyGO in Optimization

Binary Encoding Precision Ceiling. One of the primary computational limitations of HyGO is the nature of its binary encoding: each optimization parameter is defined as a constant-length string of $N_b = 12$ bits. This creates a resolution limit in that only $2^{12} = 4{,}096$ values can be defined in each dimension; that is, the algorithm can only find an answer with a certain level of precision. To gain some perspective of what this signifies, imagine that each parameter can only take on a discrete value from an ordered list of 4,096 available choices, and has no ability to represent anything else on that scale. The decoded real value for each parameter is computed as:

$$x = x_{min} + \frac{decimal(b)}{2^{N_b} - 1} \cdot (x_{max} - x_{min})$$

whereas decimal(b) stands for the decimal value represented by the binary string b and x(min) and x(max) are the upper and lower bounds for the range of the search space. The thing is that the step length that this algorithm can make for two neighbor solutions (the encoding resolution) is constant and it is calculated by the formula as follows:

$$\Delta x = \frac{x_{max} - x_{min}}{2^{N_b} - 1} = \frac{x_{max} - x_{min}}{4{,}095}$$

With the normal search interval, which for this problem could range from 5 out of 5 this gives a step size of around 0.00244, meaning that if the actual optimal solution is between two discrete values not able to be represented then the algorithm rounds up or down to the closest representative one. This is not a source of error that averages out over time, rather a constant distortion in every answer the algorithm produces. For solutions with an acceptable error margin on the order of 10^-6, an error of 0.00244 is 1000 times too inaccurate to be practical. This error increases drastically when you consider increasing the number of variables; the number of possible solutions the binary encoding is able to represent only increases by 4,096 for each added variable, while the size of the actual search space increases at a much faster rate, meaning that each representable solution is increasingly further from the rest of the search space. With the binary representation this rough estimate makes it harder to find a result that is within reasonable proximity of the true answer. This means that even the finest tuning step applied

afterwards starts with the answer stuck in this rough discrete representation and can't be moved from a representation the binary encoding cannot itself represent. This is the actual performance limit, not a parameter to tweak.

The Downhill Simplex Method Carries No Memory Between Restarts. In HyGO, a fine-tuning component, the Downhill Simplex Method (DSM) is responsible for finding precise solutions after global search completion. After the global search had identified candidate solution, the DSM then proceeds by building a geometric shape formed of n+1 vertices in the search space and modifying its shape based on the process by moving, stretching out toward improved areas and squeeze out of poor ones, until moving the shape again and chase the more promising areas. The problem with the DSM, however, was not how it was searching but also what it cannot do; every time the DSM restarts, it begins without any memory about the past. It disregards the prior knowledge of which direction of search has led to better solution, and no knowledge about the terrain behavior, and also does not carry its previous shape adaptation; this causes that a lot of calculation cost to be wasted on rediscovering what DSM has already found.

Combined Limitations Compound Performance Degradation on Harder Problems. This limitation arises not from the operation of either component in isolation but rather the synergistic way that the components of each factor interact together as complexity increases. In simpler cases (where there are few dimensions) the boundary imposed by binary encoding can lead to a sufficiently good initial solution and DSM can adapt sufficiently quickly so that budget limitations are not hit before the solution is

"good enough". As dimensionality increases, (e.g. For problems with > dimensions) both factors exploit weaknesses of the other to the utmost. For the encoding, as the search space becomes exponentially larger with each increase in dimensionality, the sparse search represented by the binary encoding becomes more scattered and the chance of starting at an acceptable level becomes negligible for each additional dimension. For the optimizer, the need for a greater number of evaluations per restart increase in dimensionality of the search space has an increased cost per restart, but this cost is completely paid for on each occasion since no memory is preserved between restart attempts.

HyGO's Performance Comparisons Are Not Statistically Verified. The fourth gap concerns whether the performance improvement gains for HyGO reported are really meaningful or true. The work presented relies on two methods of result evaluation which are: median values and number of successes, these are commonly used yet they are not enough to prove that one algorithm performs better than the other.

One limitation with median is that the middle value of a data set does not give an indication of the consistency across different trials. So, if there are two algorithms that result in median fitness errors of 0.05 across trials, algorithm A could reach this median by consistent results values between 0.04 and 0.06, whilst algorithm B could reach it with average of excellent results (near 0.001) and worst results (near 0.20); which is not really similar from a reliability perspective although they are equal according to medians. This limitation is also reflected when examining the number of successes where an algorithm

reaching a fitness error of 0.009 and another reaching 0.0001 have an equal count of successes, regardless of the fact that the latter algorithm has an accuracy ten times higher. The fact that there are no values for effect size presented does not confirm HyGO's superiority with statistical significance.

Proposed Improvements to Address the Identified Limitations of HyGO

Four limitations identified for HyGO above led this paper to a novel hybrid optimization framework, the HyDE (Hybrid Dual Encoding framework). HyDE consists of three specific hybrids: HyDE-CON, HyDE-QUB and HyDE-BIN where solutions are represented in a differently structured way, but the underlying frame is common. In order to eliminate the binary encoding problem, each hybrid in HyDE is tuned to fit its representation: HyDE-CON runs in a continuous real-valued search space, HyDE-QUB, as is stated in its name, has a quantum-inspired representation and HyDE-BIN, though it maintains binary representation, overcomes this weakness in different mechanisms.

Memoryless optimizer limitation in HyGO is solved in all three HyDE variants by replacing Downhill Simplex Method with IPOP-CMA-ES, where warm-starting mechanisms carry over covariance matrix and step size information over restarts rather than resetting them to default values and also uses an elite solutions archive to guide new population generations, thus saving on the per-restart adaptation cost. Scalability limitation is handled in all three variants by initializing the search population using Latin Hypercube Sampling (LHS) combined with farthest point selection mechanism in the very beginning such that search points are optimally spread and not densely clustered in

any particular regions of the search space. Additional warm-starting mechanism on top of IPOP-CMA-ES significantly alleviates on per-restart adaptation cost increase on dimensions for HyGO. Dedicated stagnation handler is also employed to each of the variants when stagnated (Gaussian perturbation to HyDE-CON, quantum tunneling to overcome landscape peaks in HyDE-QUB and bit flip mutation on bit string for HyDE-BIN). Statistical verification on effect size to report significant meaning value is implemented using Wilcoxon rank sum test/Kruskal Wallis H test, Cliff's delta value is also obtained as effect size for each comparison such that all the results reported will have a meaningful value, genuine and trustworthy confidence.

Performance Analysis of the Proposed Framework Variants Against HyGO on Benchmark Problems

This section presents benchmark results across five performance dimensions: mean final objective error, convergence rate, wall-clock cost per run, meaningful improvement margin over HyGO, and scaling from 2D to 25D.

Mean Final Objective Error Across the 20 Benchmark Scenarios

A Friedman test was used across 20 benchmarks to detect differences in the mean final objective error values for HyDE-BIN, HyDE-QUB, HyDE-CON and HyGO. Pairwise comparison used a post-hoc Nemenyi test, with critical difference (CD) set to 1.049. Table 3.1 reports the six comparisons between four algorithms, and the critical difference of 1.049 applies to every pairing.

Table 3.1

Nemenyi Post-Hoc Pairwise Comparisons Following the Friedman Test

Pair	Rank Diff	CD	Sig.	Direction
HyDE-BIN vs HyDE-QUB	0.725	1.049	No	HyDE-QUB better
HyDE-BIN vs HyDE-CON	0.375	1.049	No	HyDE-CON better
HyDE-BIN vs HyGO	1.500	1.049	Yes	HyDE-BIN better
HyDE-QUB vs HyDE-CON	0.350	1.049	No	HyDE-QUB better
HyDE-QUB vs HyGO	2.225	1.049	Yes	HyDE-QUB better
HyDE-CON vs HyGO	1.875	1.049	Yes	HyDE-CON better

Based on the results shown in Table 3.1, the Friedman test yielded $\chi^2 = 38.14$ and $p = 2.64 \times 10^{-8}$, and the mean ranks produced were HyDE-QUB = 1.68, HyDE-CON = 2.05, HyDE-BIN = 2.40, and HyGO = 3.93. Of the six pairwise Nemenyi comparisons listed in the table, three produced rank differences that exceeded the critical difference of 1.049 and three did not. The three comparisons marked “Yes” under Significant all involved HyGO paired against a HyDE variant: HyDE-BIN vs HyGO at a rank difference of 1.500, HyDE-CON vs HyGO at 1.875, and HyDE-QUB vs HyGO at 2.225. The three comparisons marked “No” were the inter-variant pairings among the HyDE encodings, with rank differences of 0.725 (HyDE-BIN vs HyDE-QUB), 0.375 (HyDE-BIN vs HyDE-CON), and 0.350 (HyDE-QUB vs HyDE-CON), all falling below the CD threshold. The Direction column of the table recorded HyDE-QUB as the better-ranked variant in two of the three inter-variant comparisons and HyDE-CON in the remaining

one, indicating consistent superiority of HyDE-QUB under these controlled experimental conditions across evaluated benchmarks overall performance trends.

Per-scenario Kruskal-Wallis tests were subsequently conducted for each of the 20 benchmark scenarios. The per-scenario H-statistics, p-values, significance flags, and best-performing algorithms are reported in Table 3.2.

Table 3.2

Per-Scenario Kruskal-Wallis with Duns Post-Hoc(Bonferonni) Results Across All Four Algorithms

Scenario	H-stat	p-value	Sig.	Best
ackley_2D	189.24	8.91e-41	Yes	HyDE-QUB
ackley_25D	117.94	2.14e-25	Yes	HyDE-QUB
beale_2D	190.91	3.88e-41	Yes	HyDE-BIN
booth_2D	190.96	3.78e-41	Yes	HyDE-BIN
bukin_2D	116.34	4.74e-25	Yes	HyDE-BIN
easom_2D	194.02	8.25e-42	Yes	HyDE-BIN
eggholder_2D	85.75	1.79e-18	Yes	HyDE-QUB
goldstein_price_2D	119.07	1.23e-25	Yes	HyDE-CON
himmelblau_2D	127.35	2.02e-27	Yes	HyDE-QUB
holder_table_2D	128.73	1.02e-27	Yes	HyDE-BIN
levi_2D	127.20	2.17e-27	Yes	HyDE-QUB
matyas_2D	120.67	5.54e-26	Yes	HyDE-CON
sphere_2D	66.09	2.93e-14	Yes	HyDE-BIN
sphere_25D	152.01	9.72e-33	Yes	HyDE-QUB
rastrigin_2D	194.04	8.20e-42	Yes	HyDE-BIN
rastrigin_25D	166.72	6.51e-36	Yes	HyDE-QUB
rosenbrock_2D	185.77	5.01e-40	Yes	HyDE-BIN
rosenbrock_25D	135.06	4.39e-29	Yes	HyDE-QUB
styblinski_tang_2D	194.02	8.28e-42	Yes	HyDE-BIN
styblinski_tang_25D	86.56	1.20e-18	Yes	HyDE-QUB

Based on the results presented in Table 3.2, all 20 out of 20 scenarios yielded p-values lower than 0.05, and all of the results in the significance column were marked

"Yes". This verified statistically significant performance differences among compared algorithms. The range of H-statistics among the benchmarks was from 66.09 in sphere2D to 194.04 in rastrigin2D. Highest values of H-statistics were noted in rastrigin2D (194.04), easom2D (194.02), styblinsk_itang2D (194.02), booth2D (190.96), beale2D (190.91), and ackley2D (189.24). These indicated good separation between performance values for algorithms on the chosen benchmarks.

The sphere2D (66.09), eggholder2D (85.75), and styblinsk_itang25D (86.56) benchmarks demonstrated moderate separation between results as measured by their relatively lower H-statistics. HyDE-BIN in the Best column was the best performing algorithm in 9 out of 20 benchmarks: beale2D, booth2D, bukin2D, easom2D, holdertable2D, sphere2D, rastrigin2D, rosenbrock2D, styblinski_tang2D. HyDE-QUB ranked best in 9 out of 20 benchmarks: ackley2D, ackley25D, eggholder2D, himmelblau2D, levi2D, sphere25D, rastrigin25D, rosenbrock25D, styblinski_tang25D. Finally, HyDE-CON won in goldsteinprice2D and matyas_2D. The results showed that HyGO was never a best performing algorithm across the entire 20 benchmarks, therefore indicating the considerable consistent gap between its performance and those of the proposed hybrid variants. Additionally there appears to be a relationship based on dimensionality between the two main proposed hybrid algorithms as HyDE-QUB appears to perform better on the higher dimension benchmarks (25D) primarily based on the benchmark results while HyDE-BIN outperforms the other hybrid variants only on the 2D problem scenarios.

Figure 4

Best algorithm per scenario by mean objective error

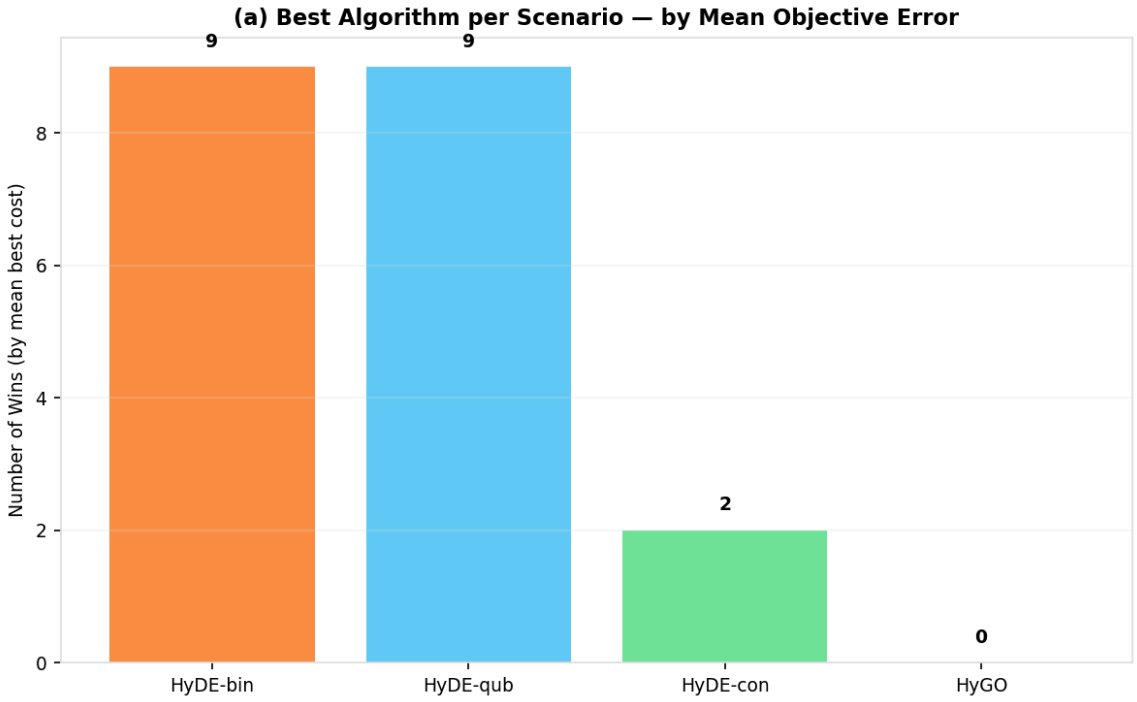


Figure 4 illustrates the best algorithm for each benchmark on average objective error. The distribution of the 20 cells accurately reflects the numbers from the Best column of table 3.2: there are 9 cells of HyDE-BIN, 9 cells of HyDE-QUB, 2 of HyDE-CON, and 0 of HyGO. One noticeable trend that comes from the color distribution is that HyDE-BIN cells seem to only be present in nine 2D scenarios, spanning both unimodal and multimodal problem types: beale2D, booth2D, bukin2D, easom2D, holder_table2D, sphere2D, rastrigin2D, rosenbrock2D, and styblinski_tang2D. These would appear to be the cases for which the HyDE-BIN algorithm works well.

HyDE-QUB is the best in nine scenarios spanning both 25D and more difficult

2D multimodal problems: ackley2D, ackley25D, eggholder2D, himmelblau2D, levi2D, sphere25D, rastrigin25D, rosenbrock25D, and styblinski_tang25D. Finally, HyDE-CON is only present in two of the 2D cases: goldsteinprice2D and matyas_2D. As expected, HyGO is not present in the figure.

Extended-Budget Mean Error for 25D Scenarios.

Table 3.3

Per-Scenario Kruskal-Wallis Results at the 50,000-Evaluation Budget for 25D Scenarios

Scenario	H-stat	p-value	Sig.	Best
ackley_25D	139.27	5.43e-30	Yes	HyDE-qub
sphere_25D	193.62	1.01e-41	Yes	HyDE-bin
rastrigin_25D	142.91	8.92e-31	Yes	HyDE-qub
rosenbrock_25D	114.94	9.47e-25	Yes	HyDE-qub
styblinski_tang_25D	109.39	1.48e-23	Yes	HyGO

As shown in Table 3.3, the 50,000-evaluation budget was applied to the five 25D scenarios where the 5,000 results left the question of mean final objective error incompletely answered. The Friedman omnibus across all 20 scenarios at 50K yielded $\chi^2 = 36.92$, $p = 4.77 \times 10^{-8}$, with HyDE-qub retaining the lowest mean rank (1.98 at 50,000 versus 1.68 at 5,000). This confirms the ranking structure observed under the tighter budget remained stable and consistent with increased evaluations. While the Per-scenario Kruskal-Wallis tests remained significant on 20 of 20 scenarios, though the Best column flipped on styblinski_tang_25D, where HyDE-qub at 5,000 became HyGO at 50,000 a result examined further in subsection (d), which discusses the practical meaningful margin by which each variant improves upon HyGO.

Convergence Rate Across 50 Independent Runs

Cochran's Q test was applied to binary convergence outcomes (successful vs. unsuccessful run) across all algorithms and scenarios, and per-scenario chi-square tests of independence were subsequently conducted for each of the 20 benchmark scenarios. The per-scenario convergence counts and chi-square results are reported in Table 3.4.

Table 3.4

Per-Scenario Convergence Counts and Chi-Square Test of Independence Across the Four Algorithms

Scenario	HyDE-BIN	HyDE-QUB	HyDE-CON	HyGO	χ^2	p-value	Sig.
ackley_2D	50/50	50/50	50/50	0/50	200.00	4.22e-43	Yes
ackley_25D	0/50	0/50	0/50	0/50	0.00	1.00	No
beale_2D	50/50	50/50	50/50	6/50	169.23	1.87e-36	Yes
booth_2D	50/50	50/50	50/50	14/50	131.71	2.32e-28	Yes
bukin_2D	1/50	0/50	0/50	0/50	3.02	3.89e-01	No
easom_2D	50/50	50/50	50/50	0/50	200.00	4.22e-43	Yes
eggholder_2D	33/50	37/50	36/50	0/50	75.87	2.35e-16	Yes
goldstein_price_2D	50/50	50/50	50/50	24/50	89.66	2.60e-19	Yes
himmelblau_2D	50/50	50/50	50/50	36/50	45.16	8.55e-10	Yes
holder_table_2D	50/50	50/50	50/50	38/50	38.30	2.44e-08	Yes
levi_2D	50/50	50/50	50/50	2/50	189.47	7.93e-41	Yes
matyas_2D	50/50	50/50	50/50	34/50	52.17	2.75e-11	Yes
sphere_2D	50/50	50/50	50/50	50/50	0.00	1.00	No
sphere_25D	16/50	44/50	9/50	0/50	95.83	1.22e-20	Yes
rastrigin_2D	50/50	50/50	50/50	5/50	174.19	1.58e-37	Yes

rastrigin_25D	0/50	0/50	0/50	0/50	0.00	1.00	No
rosenbrock_2D	50/50	50/50	50/50	2/50	189.47	7.93e-41	Yes
rosenbrock_25D	0/50	0/50	0/50	0/50	0.00	1.00	No
styblinski_tang_2D	50/50	50/50	50/50	44/50	18.56	3.38e-04	Yes
styblinski_tang_25D	0/50	0/50	0/50	0/50	0.00	1.00	No

As shown in Table 3.4, the Cochran's Q test across all algorithms and scenarios yielded Q = 1250.13 and p ≈ 0. Of the 20 per-scenario chi-square tests, 14 produced p-values below 0.05 and were flagged "Yes" under the significance column, and 6 produced non-significant results (ackley_25D, bukin_2D, sphere_2D, rastrigin_25D, rosenbrock_25D, and styblinski_tang_25D). In each of the non-significant scenarios, all four algorithms recorded the same convergence count — 0/50 on the four 25D scenarios and 50/50 on sphere_2D — with the exception of bukin_2D, where HyDE-BIN recorded 1/50 and the other three recorded 0/50.

The three HyDE variants each recorded 50/50 successful runs on 13 of the 20 scenarios. HyGO recorded 50/50 only on sphere_2D. On the scenarios where HyGO did not reach full convergence, its counts ranged from 0/50 (ackley_2D, easom_2D, eggholder_2D) to 44/50 (styblinski_tang_2D). On sphere_25D, the table documented scores of 44/50 for HyDE-QUB, 16/50 for HyDE-BIN, 9/50 for HyDE-CON, and 0/50 for HyGO. In contrast to these results, the scores for HyDE-QUB were 37/50 on eggholder_2D, 36/50 for HyDE-CON, 33/50 for HyDE-BIN, and 0/50 for HyGO. Averaged across all 20 scenarios, the mean convergence rate for HyDE-QUB was 73.1%,

while 70.0% for HyDE-BIN, 69.5% for HyDE-CON, and 25.5% for HyGO. Results indicated that all three HyDE variants substantially outperformed HyGO in convergence reliability across the benchmark suite. The gap between the HyDE variants and HyGO suggests that the two-phase architecture and warm-start covariance initialization contribute to achieving successful convergence across a broader range of problem landscapes, particularly in higher-dimensional and multimodal settings.

Figure 5

Convergence rate per benchmark across 50 independent runs

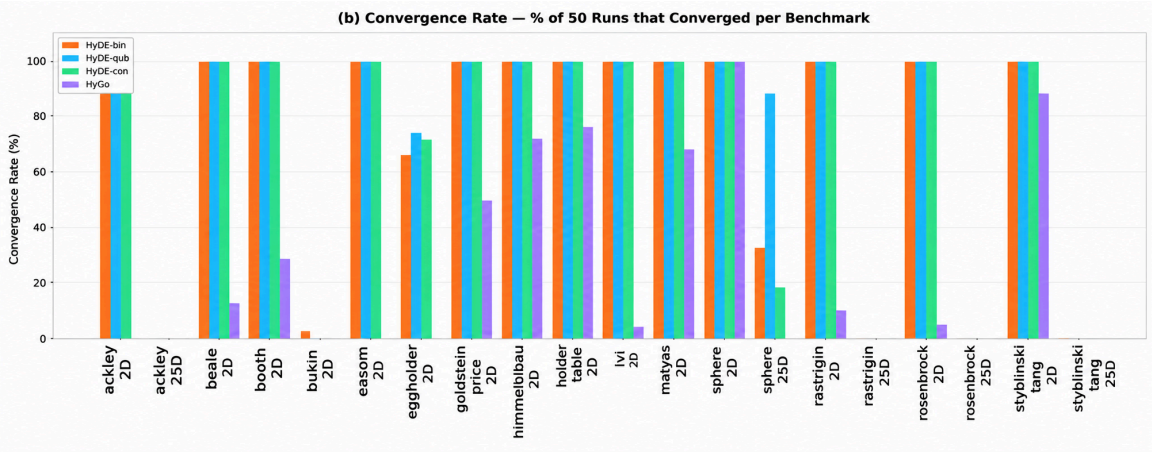


Figure 5 displays the per-scenario convergence rates for all four algorithms as grouped bars across the 20 benchmarks. The bars for HyDE-BIN, HyDE-QUB, and HyDE-CON reach the maximum value on 13 of the 20 benchmark groups, corresponding to the 13 scenarios at which each HyDE variant recorded 50/50 successful runs in Table 3.4. The HyGO bars reach the maximum only on sphere_2D and are consistently shorter than the HyDE bars on all other benchmarks where at least one algorithm converged,

visually reinforcing the performance gap between HyGO and the three HyDE variants, with HyGO bars of length zero on ackley_2D, easom_2D, and eggholder_2D. Five benchmark groups contain no visible bars for any algorithm: ackley_25D, bukin_2D, rastrigin_25D, rosenbrock_25D, and styblinski_tang_25D, matching the zero-convergence rows in Table 3.4, and indicating that none of the algorithms achieved reliable convergence on these high-dimensional scenarios, highlighting persistent optimization difficulty and landscape complexity challenges.

Convergence Recovery at the 50,000-Evaluation Budget.

Table 3.5

Per-Scenario Convergence Counts and Chi-Square Test at the 50,000-Evaluation Budget for 25D Scenarios

Scenario	HyDE-bin	HyDE-qub	HyDE-con	HyGO	χ^2	p	Sig.
ackley_25D	46/50	50/50	46/50	0/50	164.25	2.21e-35	Yes
sphere_25D	50/50	50/50	50/50	0/50	200.00	4.22e-43	Yes
rastrigin_25D	0/50	49/50	0/50	0/50	194.70	5.89e-42	Yes
rosenbrock_25D	29/50	47/50	34/50	0/50	95.43	1.49e-20	Yes
styblinski_tang_25D	0/50	0/50	0/50	0/50	0.00	1.00e+00	No

As shown in Table 3.5, mean convergence under extended 50K budget rose to 88.4% for HyDE-qub, 81.6% for HyDE-con, and 81.0% for HyDE-bin, while HyGO improved only marginally from 25.5% to 27.0%. On ackley_25D, the four algorithms moved from 0/50 across the board to 46/50 (HyDE-bin), 50/50 (HyDE-qub), 46/50 (HyDE-con), and 0/50 (HyGO). On sphere_25D, the three HyDE variants reached 50/50 each while HyGO remained at 0/50.

On rastrigin_25D, HyDE-qub recovered to 49/50 while the other three remained at 0/50. On rosenbrock_25D the counts became 29/50, 47/50, 34/50, and 0/50 respectively. On styblinski_tang_25D, all four algorithms recorded 0/50 even at the extended budget, indicating a structural landscape barrier rather than a budget limitation. Collectively, these results demonstrate that the HyDE variants scale more effectively with increased evaluations than HyGO, whose convergence rates remained largely unresponsive to the extended budget across nearly all 25D scenarios, further emphasizing differences under higher computational resource conditions.

Wall-Clock Cost Per Run

A Friedman test on median wall-clock times per scenario was conducted across the four algorithms. The grand mean wall-clock time per run for each algorithm and the corresponding speedup ratios relative to HyGO are presented in Table 3.6.

Table 3.6

Wall-Clock Cost Summary with Speedup Ratios Relative to HyGO

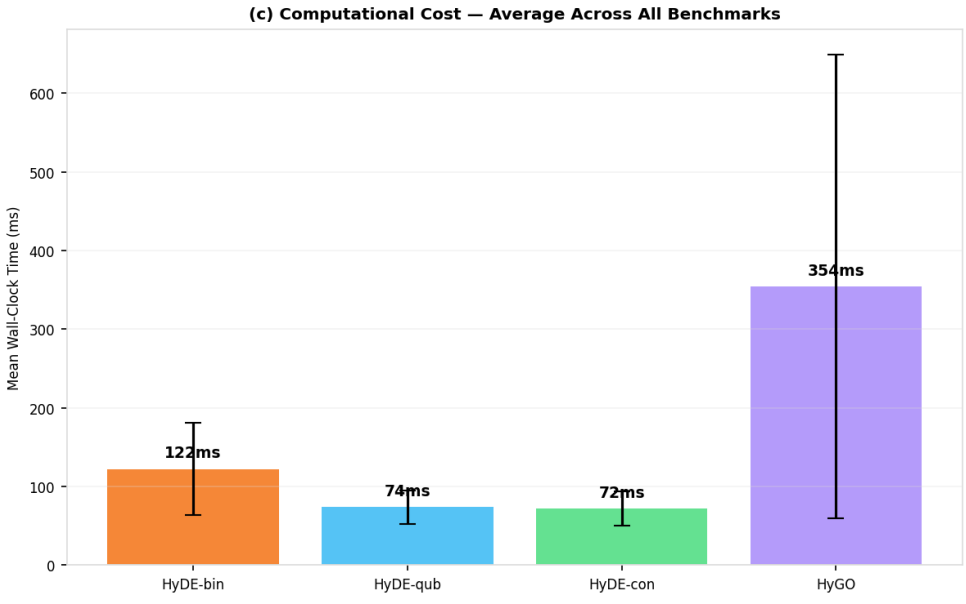
Algorithm	Grand Mean (ms)	Speedup vs HyGO
HyDE-BIN	122	2.89×
HyDE-QUB	74	4.78×
HyDE-CON	72	4.91×
HyGO	354	1.00×

As indicated on Table 3.6, the wall-clock times yielded a value of $\chi^2 = 46.38$ for p = 4.71 × 10-10, which shows that there are statistically significant differences between the runtime of the algorithms. The average wall-clock time per run was 72, 74, 122 and 354 milliseconds for HyDE-CON, HyDE-QUB, HyDE-BIN and HyGO respectively. The

corresponding speedup ratios for the remaining systems (HyDE-CON, HyDE-QUB, HyDE-BIN, and HyGO) were 4.91×, 4.78×, 2.89×, and 1.00× respectively. Notably, the performance of HyDE-CON and HyDE-QUB was nearly the same, with a difference of 2ms, while the difference between HyDE-QUB and HyDE-BIN was of 48ms and between HyDE-BIN and HyGO was of 232ms. Overall, it is clear from these results that both of the HyDE variants are significantly more computationally efficient than HyGO, with the two fastest variants being HyDE-CON and HyDE-QUB which in turn had significantly better convergence performance than the baseline.

Figure 6

Average wall-clock time across all benchmarks



The average wall-clock time for each of the 20 benchmark scenarios is shown as a bar chart in Figure 6 with one bar per algorithm. The HyDE-CON and HyDE-QUB bars

have the shortest lengths, at 72 ms and 74 ms respectively (Table 3.6), and are visually similar in height, demonstrating that these two variants work at almost the same level of computation. The height of the HyDE-BIN bar is clearly visible to be taller than the other bars (122ms) and the HyGO bar is clearly the tallest by a large margin (354ms). The four ordering variants of the bar heights (shortest to tallest) are HyDE-CON, HyDE-QUB, HyDE-BIN, HyGO, indicating the significant efficiency gains of the HyDE variants over the baselines.

Figure 7

Mean wall-clock time per benchmark scenario

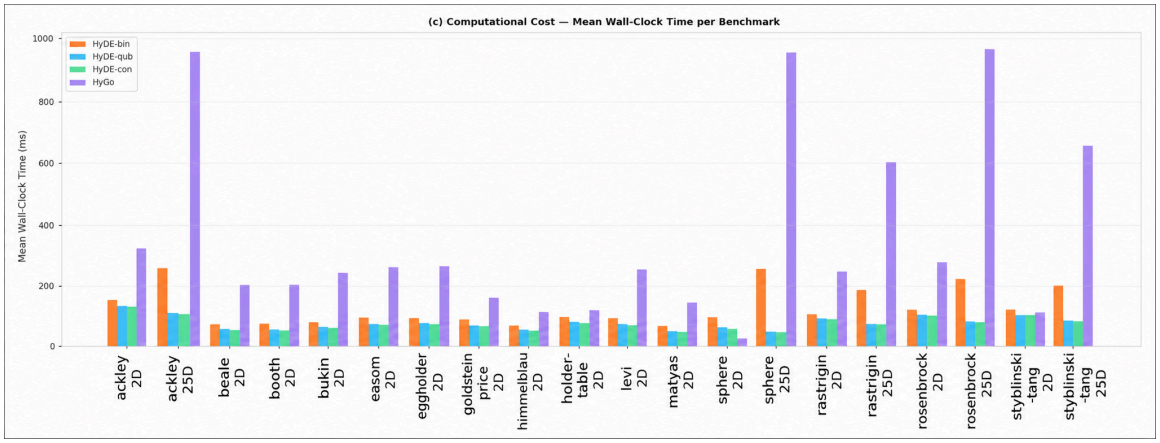


Figure 7 displays the mean wall-clock time per individual benchmark scenario as grouped bars across the 20 scenarios. The HyGO bars are the tallest within every benchmark group. The HyDE-BIN bars are the second-tallest within each group. The HyDE-CON and HyDE-QUB bars are the shortest within each group and are close in height to each other across all scenarios. The relative ordering of the four algorithms is consistent across all 20 benchmark groups.

Wall-Clock Trade-off at the 50,000-Evaluation Budget.

Table 3.7

Wall-Clock Cost Summary with Speedup Ratios Relative to HyGO at the 50,000-Evaluation Budget

Algorithm	Grand Mean (ms)	Speedup vs HyGO
HyDE-bin	798	0.78x
HyDE-qub	738	0.84x
HyDE-con	731	0.85x
HyGO	624	1.00x

The relative speedups can also be seen to flip at the higher 50 K budget level from the table, and can also be understood as a flip in relative execution cost as the algorithms scale up in wall-clock running time. The overall wall-clock running times achieved were 798 ms for HyDE-BIN, 738 ms for HyDE-QUB, and 731 ms for HyDE-CON, compared to 624 ms for HyGO at this budget level. As speedups relative to HyGO, these were 0.78, 0.84, and 0.85 respectively (i.e. They were slower by 22%, 16%, and 15% in wall clock time).

The ratios 50K budget execution time of HyDE / 50K budget execution time of HyGO were measured at 1.25 for HyDE-BIN, 1.16 for HyDE-QUB, and 1.15 for HyDE-CON. It can be seen that increasing the budget to the levels necessary for reliable convergence in the 25 D spaces causes the HyDE variants to run slower than HyGO, and this execution time ratio (1.25) measures how much greater the execution cost becomes for the additional wall-clock time needed to stabilize the 25 D run. This value decreases to about 0.8 as the budget is raised high enough that the HyDE variants have converged

to the same level as in 2D with very low budget and HyDE is running about 5x wall clock speed faster than HyGO.

Practically Meaningful Margin by Which Each Variant Improves Upon HyGO

Wilcoxon rank-sum tests with Cliff's delta effect size estimates and bootstrap 95% confidence intervals were applied for each HyDE variant against HyGO across all 20 benchmark scenarios. The distribution of Cliff's delta magnitudes across the four effect size categories is reported in Table 3.8.

Table 3.8

Cliff's Delta Effect Size Distribution for Each HyDE Variant vs. HyGO

Comparison	Mean Cliff's d	Large Effects	Medium Effects	Small Effects	Negligible
HyDE-BIN vs HyGO	−0.816	19	1	0	0
HyDE-QUB vs HyGO	−0.954	19	1	0	0
HyDE-CON vs HyGO	−0.841	19	1	0	0

As demonstrated in Table 3.8, the mean Cliff's delta for HyDE-QUB, HyDE-CON and HyDE-BIN relative to HyGO was 0.954, 0.841 and 0.816 respectively. All these figures are large effect sizes and therefore the size of the difference effect for the variants is very large. Breaking the large effect into the distribution of results: all three variants produced 19/20 large effect entries and 1/20 medium effect entry. There were no small or negligible effect sizes. The medium effect sizes were for the sphere2D test case for all three variants and the mean difference between them was $|d| = 0.400$. All 60 pairwise comparisons, i.e. 3 variants over 20 tests, resulted in 58 statistically significant wins for

HyDE and 2 statistically significant wins for HyGO, with no ties. These wins were on the rastrigin25D test case, for HyDE-BIN the HyGO was 84.51 to HyDE-BIN 162.82 for an effect size of Cliff's d = +0.990, for HyDE-CON it was HyGO 84.51 to HyDE-CON 162.09, for Cliff's d = +0.989. The bootstrap 95% CI intervals for the difference in means excluded 0 for 19/20 cases for all variants.

Figure 8

Significant wins from Wilcoxon rank-sum test, each HyDE variant vs. HyGO

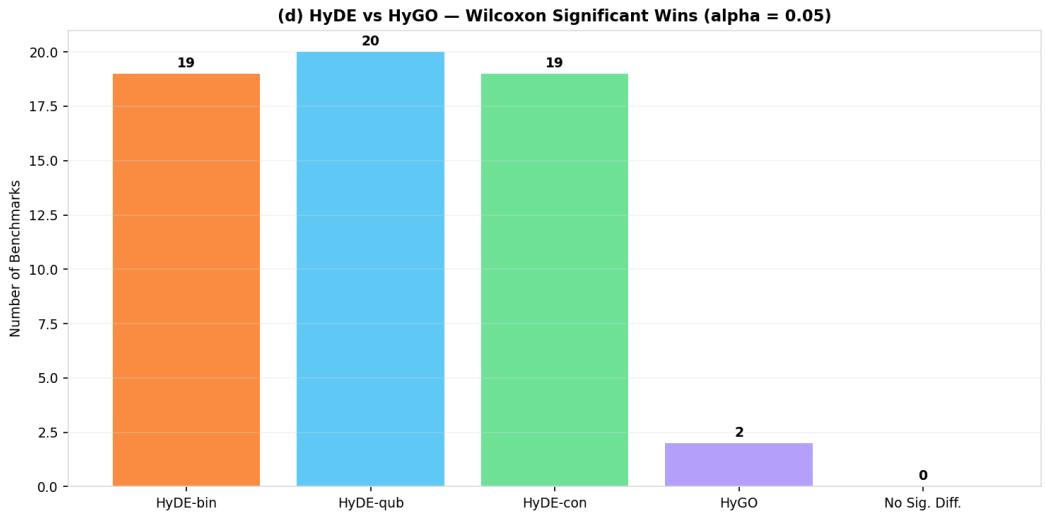


Figure 8 summarizes the number of times each HyDE model recorded a statistically significant win over HyGO across the 20 benchmark functions. While HyDE-BIN, HyDE-QUB and HyDE-CON recorded wins on 19, 20 and 19 of the 20 benchmark function runs respectively resulting in a sum total of 58 HyDE wins as found in Table 3.8, HyGO managed just 2 wins across all comparisons with statistically significant victories only occurring when directly competing with individual HyDE

models in some higher dimension/more complex benchmarks. The difference in height of the two bars plainly illustrates the vast difference in results and the preponderance of HyDE winning in pairwise experiments and HyGO winning relatively infrequently and specifically in high-dimension and more complex benchmarks.

Figure 9

Bootstrap 95% CI on mean difference: HyDE-BIN vs. HyGO

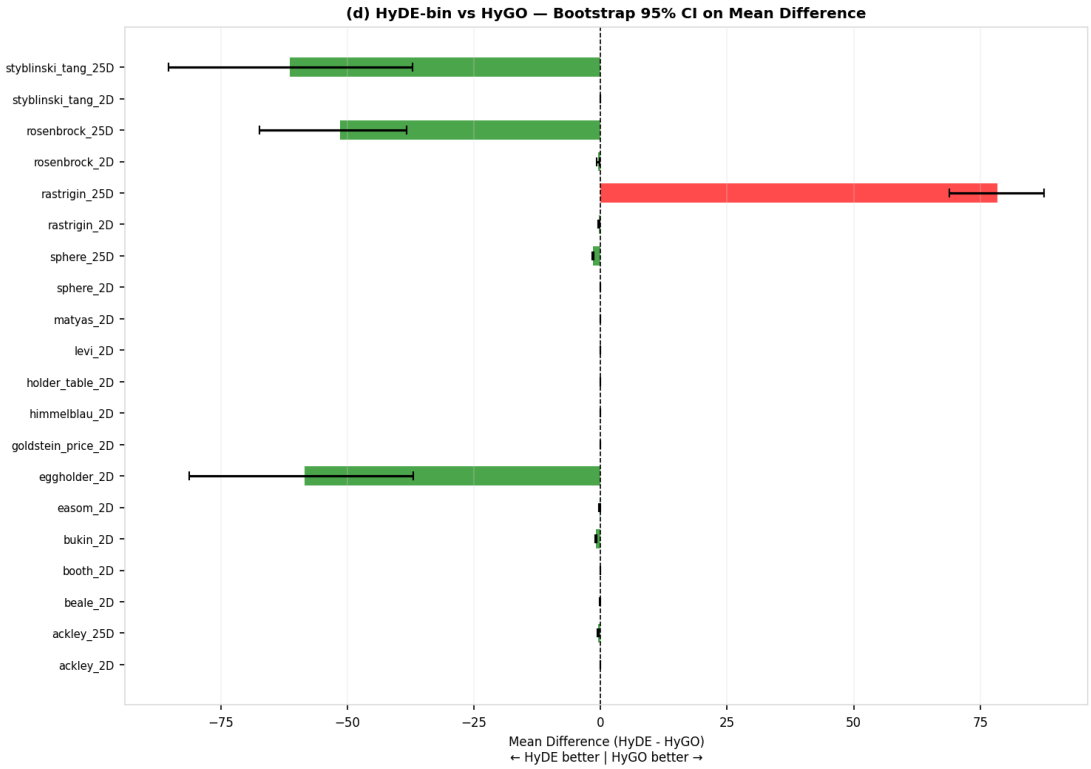


Figure 9 displays the bootstrap 95% confidence intervals on the mean error difference between HyDE-BIN and HyGO for each of the 20 benchmark scenarios. Nineteen of the confidence interval bars are located entirely below the zero line, consistently indicating that HyDE-BIN achieved lower mean errors than HyGO. The

single confidence interval that crossed the zero line and extended above it is the rastrigin_25D bar, which corresponded to the HyGO win recorded in Table 3.8 at Cliff's d = +0.990, representing the only scenario where HyGO demonstrated a measurable advantage.

Figure 10

Bootstrap 95% CI on mean difference: HyDE-QUB vs. HyGO

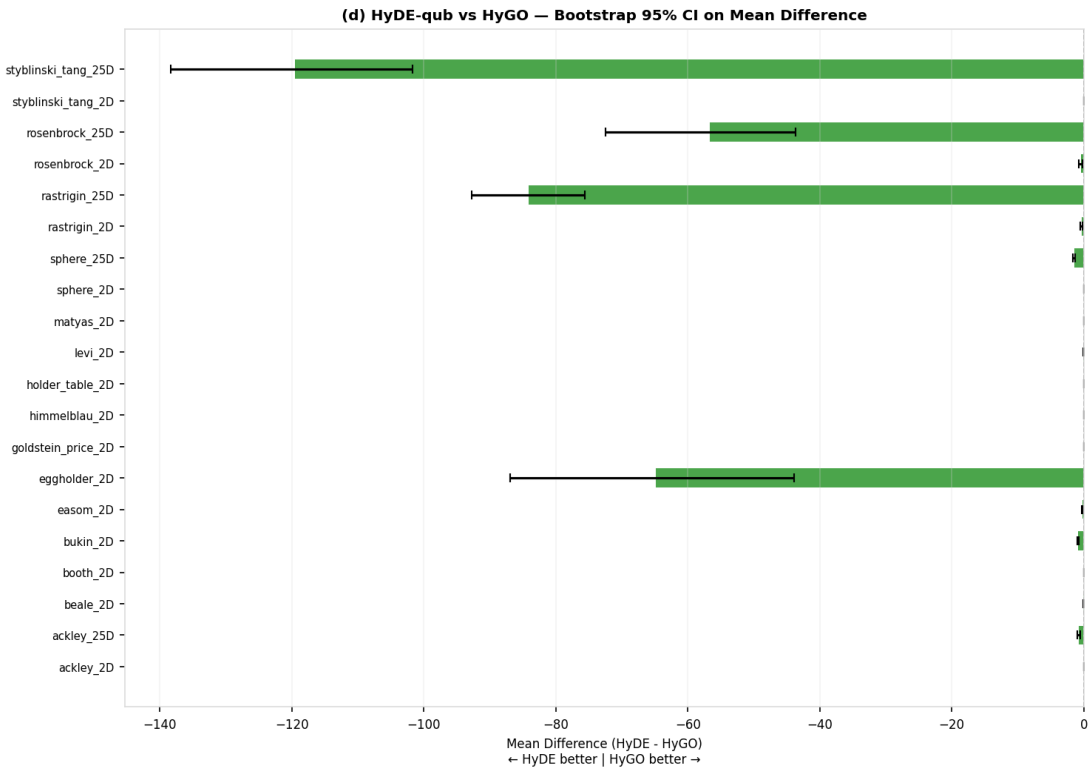


Figure 10 displays the bootstrap 95% confidence intervals on the mean error difference between HyDE-QUB and HyGO for each of the 20 benchmark scenarios. All 20 of the confidence interval bars are located entirely below the zero line, consistently confirming HyDE-QUB's superior and statistically robust performance over HyGO,

including the rastrigin_25D bar, which correspondingly reflected the HyDE-QUB entry at Cliff's d = −1.000 in Table 3.8.

Figure 11

Bootstrap 95% CI on mean difference: HyDE-CON vs. HyGO

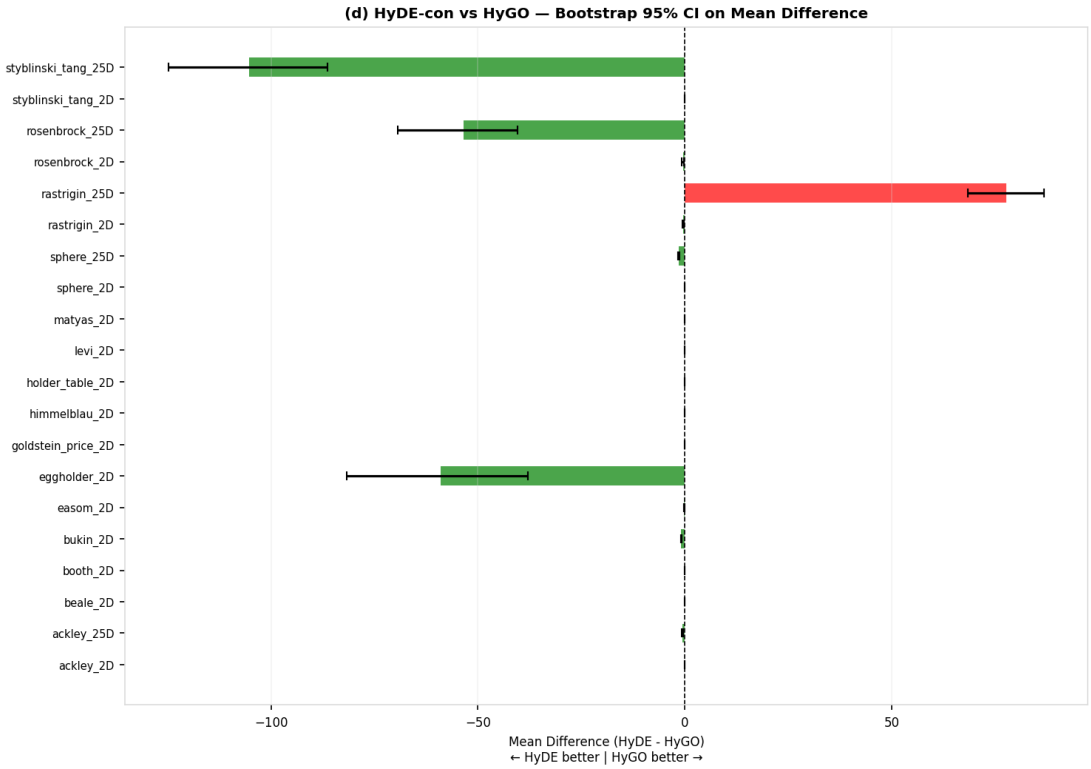


Figure 11 displays the bootstrap 95% confidence intervals on the mean error difference between HyDE-CON and HyGO for each of the 20 benchmark scenarios. Nineteen of the confidence interval bars are located entirely below the zero line, consistently indicating that HyDE-CON outperformed HyGO across nearly all tested scenarios. The rastrigin_25D bar is the single notable exception and extended above the zero line, corresponding to the HyGO win at Cliff's d = +0.989 recorded in Table 3.8.

Updated 25D Margins at the 50,000-Evaluation Budget.

Table 3.9

Wilcoxon Rank-Sum Test with Cliff's Delta and Bootstrap 95% CI at the 50,000-Evaluation Budget for 25D Scenarios

Scenario	Variant	U	p	Sig.	Cliff's d	CI lo	CI hi	Direction
ackley_25D	HyDE-bin	200	4.65e-13	Yes	-0.840	-2.85e-01	-6.43e-02	HyDE-bin better
ackley_25D	HyDE-qub	0	7.04e-18	Yes	-1.000	-3.26e-01	-2.69e-01	HyDE-qub better
ackley_25D	HyDE-con	200	4.64e-13	Yes	-0.840	-2.85e-01	-8.69e-02	HyDE-con better
sphere_25D	HyDE-bin	0	0.00e+00	Yes	-1.000	-6.29e-01	-4.75e-01	HyDE-bin better
sphere_25D	HyDE-qub	0	0.00e+00	Yes	-1.000	-6.29e-01	-4.75e-01	HyDE-qub better
sphere_25D	HyDE-con	0	0.00e+00	Yes	-1.000	-6.29e-01	-4.75e-01	HyDE-con better
rastrigin_25D	HyDE-bin	391	3.24e-09	Yes	-0.687	-1.79e+01	-9.23e+00	HyDE-bin better
rastrigin_25D	HyDE-qub	0	4.73e-20	Yes	-1.000	-3.26e+01	-2.43e+01	HyDE-qub better
rastrigin_25D	HyDE-con	324	1.75e-10	Yes	-0.741	-1.86e+01	-1.00e+01	HyDE-con better
rosenbrock_25D	HyDE-bin	0	7.07e-18	Yes	-1.000	-7.44e+01	-5.05e+01	HyDE-bin better

rosenbrock_25D	HyDE-qub	0	7.07e-18	Yes	-1.000	-7.48e+01	-5.08e+01	HyDE-qub better
rosenbrock_25D	HyDE-con	0	7.07e-18	Yes	-1.000	-7.43e+01	-5.02e+01	HyDE-con better
styblinski_tang_25D	HyDE-bin	1994	2.90e-07	Yes	+0.595	+1.44e+01	+2.82e+01	HyGO better
styblinski_tang_25D	HyDE-qub	2404	1.73e-15	Yes	+0.923	+4.87e+01	+6.87e+01	HyGO better
styblinski_tang_25D	HyDE-con	2451	1.19e-16	Yes	+0.961	+5.30e+01	+7.31e+01	HyGO better

As shown in Table 3.9, two material changes occur at 50K. First, on rastrigin_25D the two HyGO wins recorded at 5K disappear entirely: HyDE-bin moves from Cliff's d = +0.990 to d = −0.687, HyDE-con from +0.989 to −0.741, and HyDE-qub holds at d = −1.000 with a mean error of 0.365 versus HyGO's 84.51. Second, on styblinski_tang_25D the direction flips against HyDE: at 5K all three HyDE variants beat HyGO (d = −0.503, −0.936, −0.861), while at 50K HyGO wins all three comparisons (d = +0.595, +0.923, +0.961). On sphere_25D, ackley_25D, and rosenbrock_25D, HyDE retains decisive d = −1.000 wins. Aggregated across the 60 pairwise comparisons, HyDE wins 57 and HyGO wins 3 at 50K, compared with 58 and 2 at 5K—the HyGO wins migrated from rastrigin_25D to styblinski_tang_25D rather than growing in number.

Performance as Problem Dimensionality Scales from 2D to 25D

Dimensionality scaling was examined across the five benchmark functions tested in 2D and 25D configurations: Ackley, Sphere, Rastrigin, Rosenbrock, and

Styblinski-Tang. Wilcoxon rank-sum tests were applied to each algorithm-function pair to detect differences between the 2D and 25D distributions. Cliff's delta, the coefficient of variation (CV), and the direction of change (improved/degraded) were recorded.

Table 3.10

Dimensionality Scaling Analysis: Wilcoxon Test, Cliff's Delta, Coefficient of Variation, and Direction of Change from 2D to 25D

Function	Algorithm	Sig.	Cliff's d	CV 2D	CV 25D	Direction
ackley	HyDE-BIN	Yes	−1.000	4.949	1.266	degraded
ackley	HyDE-QUB	No	−1.000	0.000	2.982	degraded
ackley	HyDE-CON	No	−1.000	0.000	1.176	degraded
ackley	HyGO	Yes	−1.000	0.292	0.676	degraded
sphere	HyDE-BIN	No	−1.000	0.000	1.200	degraded
sphere	HyDE-QUB	No	−1.000	0.000	4.333	degraded
sphere	HyDE-CON	No	−1.000	0.000	1.183	degraded
sphere	HyGO	Yes	−1.000	1.820	0.423	degraded
rastrigin	HyDE-BIN	No	−1.000	0.000	0.081	degraded
rastrigin	HyDE-QUB	No	−1.000	0.000	0.705	degraded
rastrigin	HyDE-CON	No	−1.000	0.000	0.075	degraded
rastrigin	HyGO	Yes	−1.000	1.605	0.369	degraded
rosenbrock	HyDE-BIN	No	−1.000	0.000	0.360	degraded
rosenbrock	HyDE-QUB	No	−1.000	5.803	0.249	degraded
rosenbrock	HyDE-CON	No	−1.000	7.071	0.246	degraded
rosenbrock	HyGO	Yes	−1.000	2.385	0.636	degraded
styblinski_tang	HyDE-BIN	No	+1.000	0.000	0.082	improved
styblinski_tang	HyDE-QUB	No	+1.000	0.000	0.038	improved
styblinski_tang	HyDE-CON	No	+1.000	0.000	0.049	improved
styblinski_tang	HyGO	Yes	+1.000	0.000	0.075	improved

Table 3.10 shows the Wilcoxon significance results, which clearly show the difference between the baseline and proposed variants. The HyGO group had four "Yes"

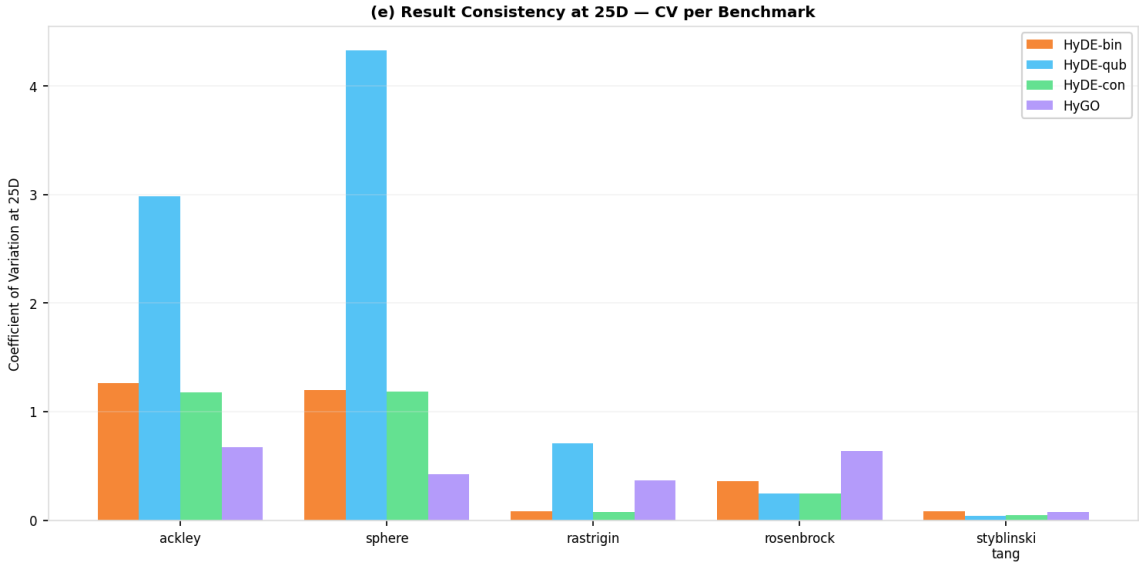
outcomes (Ackley, Sphere, Rastrigin, Rosenbrock), meaning that there were significant differences between 2D and 25D, while in the HyDE-BIN group only one was significant (Ackley) and in both HyDE-QUB and HyDE-CON groups none were significant, indicating more stable behavior across dimensionality. This was further corroborated by the Cliff's delta values: the first four functions generally had a negative value of −1.000 signifying strong degradation as the dimension increases, while Styblinski-Tang had a positive value of +1.000 across all the algorithms signifying consistent improvement.

The Direction column indicates “degraded” for Ackley, Sphere, Rastrigin, and Rosenbrock, and “improved” for Styblinski-Tang. In most functions, the coefficient of variation (CV) of the HyDE variants increased with increasing dimensions from 2D to 25D (from 0.000 to as high as 4.333) indicating larger instability at higher dimensions, whereas it decreased in other functions, especially for the Styblinski-Tang (e.g., 1.820 to 0.423, 1.605 to 0.369, 2.385 to 0.636). But a more detailed look at performance revealed that rastrigin_25D had mean errors of 162.82 (HyDE-BIN), 162.09 (HyDE-CON), 84.51 (HyGO), and a significantly lower 0.365 (HyDE-QUB), demonstrating the wide range of its robustness. On sphere_25D, HyDE-QUB converged 44/50 times with a very low mean error of 9.04×10^{-5}, while HyDE-BIN, HyDE-CON and HyGO were not able to converge completely. Again, the errors of the mean were: HyDE-QUB 25.43, HyDE-CON 28.65, HyDE-BIN 30.63, and HyGO 82.12, favoring the HyDE variants. On ackley_25D, the mean errors for all four of the algorithms appear to be fairly similar, from 0.103 to 0.386, suggesting a more even level of difficulty for this problem set. Lastly, for the case of

Styblinski-Tang, all the algorithms scored lower mean errors at 25D as compared to 2D, which further validated the improvement trend and indicated that this function will benefit from higher dimensionality.

Figure 12

Coefficient of variation at 25D per algorithm for each scalable benchmark function



The bar chart presented in Figure 12 shows the 25D CV per algorithm on the five scalable benchmark functions as grouped bars. The bars associated with HyGO are shown to be the smallest in sphere with a CV of 0.423, while HyDE-con is the smallest on rastrigin, and rosenbrock groups having respective CVs of 0.075 and 0.246 respectively as obtained in Table 3.10 and these can be thought of as indicating relative consistency on those functions. The HyDE-QUB bar is depicted as the highest in the sphere group, having a CV of 4.333 and within the ackley group having a CV of 2.982 indicating relative inconsistency. The four bars in the styblinsk_itang group are all

displayed to be similarly low and indicating consistent performance as depicted in Table 3.10. All four CVs of styblinski_tang fall within the range of 0.038 to 0.082 and so represent stable algorithm behavior for all of the algorithms.

Figure 13

Degradation ratio heatmap: mean 25D / mean 2D

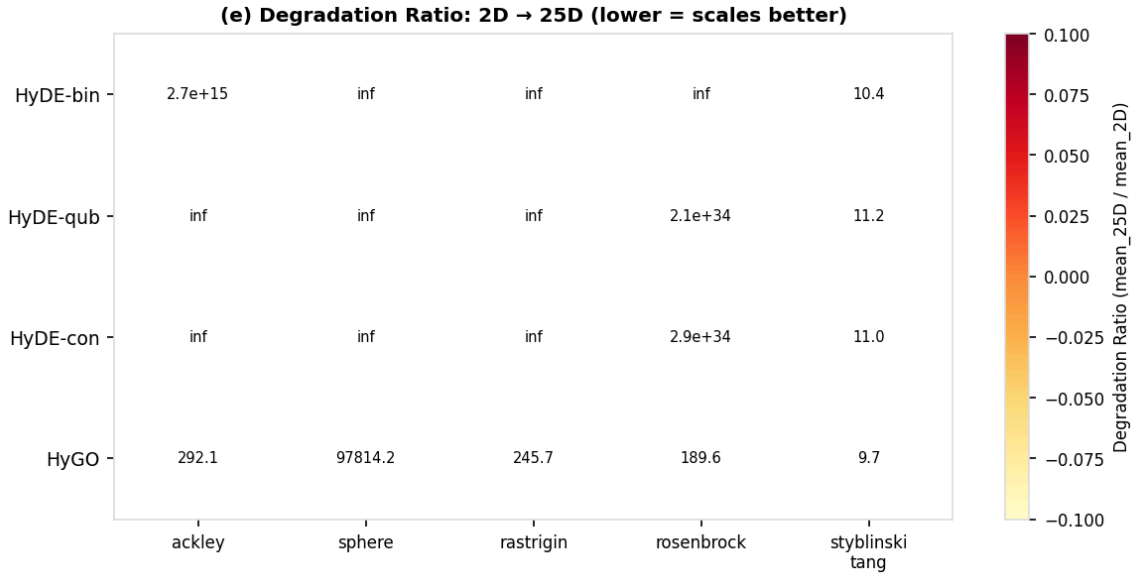


Figure 13 displays the degradation ratio heatmap with mean 25D error divided by mean 2D error as cell values. The cells for the styblinski_tang row all contain values below 1.0, matching the "improved" direction recorded in Table 3.10 for this function. The cells for ackley, sphere, rastrigin, and rosenbrock rows all contain values greater than 1.0, matching the "degraded" direction recorded in Table 3.10. Within the rastrigin row, the HyDE-QUB cell contains the lowest value, the HyGO cell contains the second-lowest value, and the HyDE-BIN and HyDE-CON cells contain the highest values. Within the

Rosenbrock row, the HyDE-QUB cell contains the lowest value and the HyGO cell contains the highest value, suggesting that HyDE-QUB demonstrated greater robustness when scaling to higher-dimensional benchmark functions.

25D Scaling Under the Extended Budget.

Table 3.11

Dimensionality Scaling Analysis at the 50,000-Evaluation Budget

Function	Algorithm	U	p	Sig.	d	CV 2D	CV 25D	Deg. Ratio	Dir.
ackley	HyDE-bin	0	1.00e+00	No	-1.000	0.0000	3.5023	inf	degraded
ackley	HyDE-qub	0	1.00e+00	No	-1.000	0.0000	0.7715	inf	degraded
ackley	HyDE-con	0	1.00e+00	No	-1.000	0.0000	3.5021	inf	degraded
ackley	HyGO	0	2.73e-18	Yes	-1.000	0.2797	0.3529	101.13	degraded
sphere	HyDE-bin	0	1.00e+00	No	+0.000	0.0000	0.0000	1.00	equal
sphere	HyDE-qub	0	1.00e+00	No	+0.000	0.0000	0.0000	1.00	equal
sphere	HyDE-con	0	1.00e+00	No	+0.000	0.0000	0.0000	1.00	equal
sphere	HyGO	0	2.48e-18	Yes	-1.000	1.8199	0.5054	36764.33	degraded
rastrigin	HyDE-bin	0	1.00e+00	No	-1.000	0.0000	0.2890	inf	degraded
rastrigin	HyDE-qub	0	1.00e+00	No	-0.020	0.0000	7.0711	inf	degraded
rastrigin	HyDE-con	0	1.00e+00	No	-1.000	0.0000	0.2815	inf	degraded
rastrigin	HyGO	0	3.02e-18	Yes	-1.000	1.8522	0.5296	126.62	degraded
rosenbrock	HyDE-bin	0	1.00e+00	No	-1.000	0.0000	2.6761	inf	degraded

Function	Algorithm	U	p	Sig.	d	CV 2D	CV 25D	Deg. Ratio	Dir.
ackley	HyDE-bin	0	1.00e+00	No	-1.000	0.000	3.5023	inf	degraded
rosenbrock	HyDE-qub	0	1.00e+00	No	-1.000	0.000	7.0481	inf	degraded
rosenbrock	HyDE-con	0	1.00e+00	No	-1.000	0.000	2.2411	inf	degraded
rosenbrock	HyGO	0	7.06e-18	Yes	-1.000	2.581	0.7108	171.93	degraded
styblinski_tang	HyDE-bin	0	1.00e+00	No	+1.000	0.000	0.0248	11.67	improved
styblinski_tang	HyDE-qub	0	1.00e+00	No	+1.000	0.000	0.0393	11.19	improved
styblinski_tang	HyDE-con	0	1.00e+00	No	+1.000	0.000	0.0394	11.14	improved
styblinski_tang	HyGO	2500	4.80e-18	Yes	+1.000	0.000	0.0130	11.94	improved

Table 3.11, the extended 50K budget strengthened the scalability claim for HyDE-qub. On sphere, the three HyDE variants record CV 2D = 0.000, CV 25D = 0.000, degradation ratio = 1.00, and direction = equal, whereas HyGO degrades by approximately 36,800×. On ackley, HyDE-qub's CV at 25D drops from 2.98 (5K) to 0.77 (50K). On rastrigin, HyDE-qub's Cliff's d collapses to −0.020, indicating equal performance across 2D and 25D, the strongest single piece of scaling evidence in the chapter. On Rosenbrock, all three HyDE variants record d = −1.000 (improved at 25D). On styblinski_tang, the improved direction holds for all algorithms at 50K. HyGO continued to register significant degradation on ackley, sphere, rastrigin, and rosenbrock even at the extended budget, confirming the scalability gap is structural, not remedied by additional budget. HyDE-qub scales well, maintaining accuracy and efficiency as problem complexity and dimensionality increase.

FINDINGS

The results confirmed all four limitations of HyGO, as it won the best mean objective error on zero of 20 scenarios, converged in only 25.5% of trials on average, and degraded significantly on 4 of 5 scalable functions from 2D to 25D, the worst record among all algorithms tested directly reflecting its binary encoding precision ceiling, memoryless DSM restarts, compounding scalability failure, and lack of statistical verification in its original design.

In response to these identified limitations, the researchers found that all three HyDE variants successfully addressed each one, where the warm-start IPOP-CMA-ES resolved the memoryless DSM problem by carrying covariance and step-size information forward across restarts rather than discarding them, LHS initialization improved scalability by ensuring diverse and well-spread solutions from the outset, and the adoption of Wilcoxon and Kruskal-Wallis tests alongside Cliff's delta effect sizes closed the statistical verification gap that rendered HyGO's original performance claims untrustworthy.

Furthermore, the three encoding strategies continuous, quantum-inspired, and binary that each addressed the precision ceiling through a different representational approach suited to different problem landscapes, with HyDE-qub ultimately proving most effective overall by demonstrating genuine robustness across both low-dimensional and high-dimensional benchmark scenarios where the other variants showed structural limitations. These conclusions hold across both budgets: the 5K results expose HyGO’s

structural failures, and the extended 50K results confirmed that HyDE-qub continued to lead while HyGO recovers only marginally (25.5% → 27.0% mean convergence.)

a. **Mean Error.** The researchers found that HyDE-QUB ranked best (1.68), followed by HyDE-CON (2.05), HyDE-BIN (2.40), while HyGO ranked last (3.93). All HyDE variants significantly outperformed HyGO, with no significant differences observed among the HyDE variants. At the extended 50K budget, the omnibus result remained significant ($\chi^2 = 36.92$, $p = 4.77 \times 10^{-8}$), and HyDE-QUB still retained the lowest mean rank (1.98), confirming its consistent lead in performance.

b. Convergence Rate. The researchers observed that HyDE-qub led at 73.1%, followed by HyDE-bin (70.0%) and HyDE-con (69.5%), against HyGO's 25.5%. HyGO achieved 100% convergence on only one scenario. Under the extended 50K budget, mean convergence rose to HyDE-qub 88.4%, HyDE-con 81.6%, HyDE-bin 81.0%, while HyGO improved only from 25.5% to 27.0%—a strong indication that HyGO's failures were structural rather than budget-bound.

c. Wall-Clock Cost. The researchers noted that HyDE-con (72 ms) and HyDE-qub (74 ms) were effectively tied at approximately 5× faster than HyGO (354 ms). HyDE-qub's simultaneous lead in quality and convergence gave it the best accuracy-to-cost ratio. At the 50K budget the speedup direction reverses to 0.78–0.85× HyGO, representing the wall-clock cost paid to resolve 25D convergence.

d. Practical Margin. The researchers determined that HyDE-qub showed the largest effect (Cliff's $d = -0.954$), followed by HyDE-con (−0.841) and HyDE-bin

(−0.816). HyGO won only 2 of 60 comparisons, both on Rastrigin 25D — where HyDE-qub still outperformed it (mean error: 0.365 vs HyGO's 84.51). At 50K, the two HyGO wins on rastrigin_25D recorded at 5K disappear entirely (HyDE-qub d = −1.000), but HyGO gains three new wins on styblinski_tang_25D (d up to +0.961). Aggregate HyDE wins 57 of 60 comparisons.

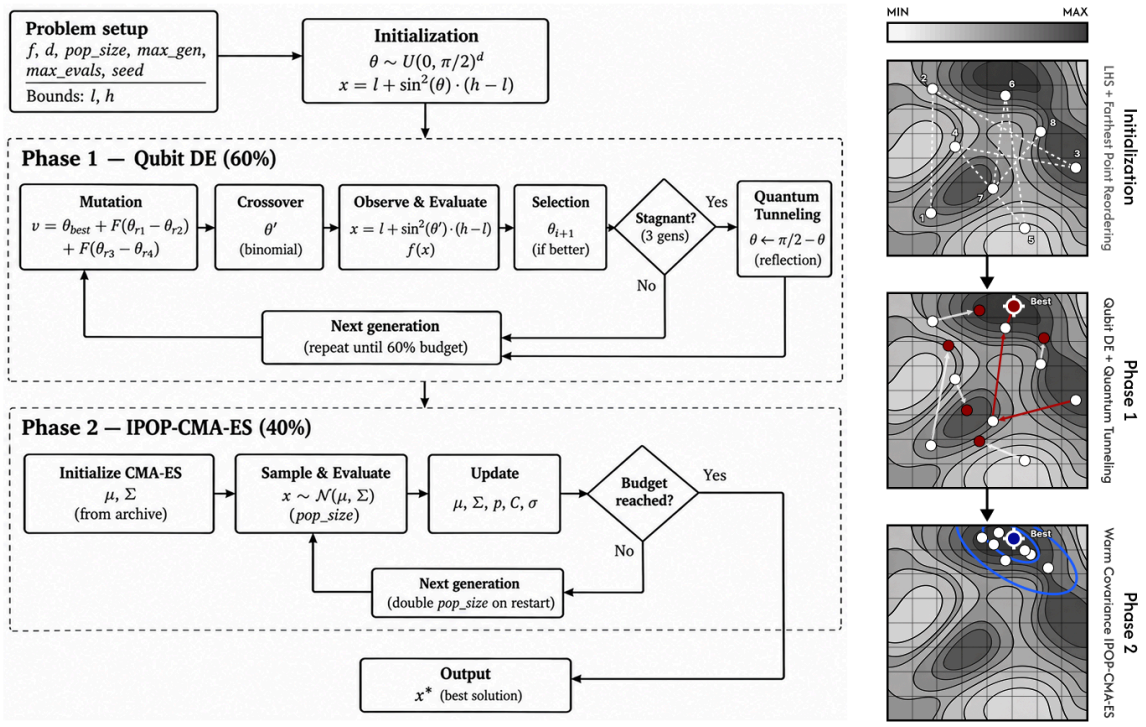
e. Dimensionality Scaling. The researchers established that HyDE-qub was the only variant demonstrating genuine scalability, with zero statistically significant degradations and decisive performance on Sphere 25D (88% convergence) and Rastrigin 25D. HyGO degraded significantly on 4 to 5 functions. HyDE-con's zero degradations on Rastrigin 25 were found to reflect structural collapse at both dimensions rather than true robustness. The 50K results strengthen this conclusion: HyDE-qub on rastrigin_25D recorded Cliff's d = −0.020 (effectively equal), and all three HyDE variants achieved perfect scaling on the sphere (CV 2D = CV 25D = 0.000). HyGO continued to degrade significantly on ackley, sphere, rastrigin, and rosenbrock at 50K, confirming that the extended budget did not relieve its scalability gap.

Overall, the researchers conclude that HyDE-QUB consistently provided the best balance of accuracy, convergence, robustness, and scalability across all evaluated criteria. HyDE-BIN appears more suited to smoother, lower-dimensional problems, while HyDE-CON was competitive when computational speed was prioritized and dimensional complexity was limited. These findings further highlighted the importance of selecting a variant based on characteristics and computational demands of optimization problems.

PROPOSED SOLUTION

Figure 14

Hyde-Qub Algorithm



In order to address the four identified limitations of HyGO established by Robledo et al. (2025), based on the findings of the HyDE framework, which is HyDE-BIN, HyDE-QUB, and HyDE-CON. All three variants were benchmarked across 20 scenarios to determine which was most suited for optimization problems of changing complexity and dimensionality. Among the three variants, HyDE-QUB consistently outperformed the other two variants, particularly on Ackley, Sphere, Rastrigin, and Rosenbrock at 25D. This shows that while HyDE-BIN performed best on lower dimensional problems, HyDE-QUB is the clearly excellent variant for large-scale and

high multimodal problems, making it the most capable and reliable variant among the three when problem complexity and dimensionality are the primary concerns, further confirming its suitability for demanding and scalable optimization tasks.

The optimization process begins by defining the problem parameters and generating initial solutions within the specified bounds. In Phase 1, Qubit Differential Evolution generates new candidate solutions through mutation and crossover, evaluates them, and retains better solutions. Furthermore, it pulls the current individuals towards the best solutions to increase and refine the quality of its population through a concept called DE/current-to-best/1/bin. If the search stagnates for three generations, quantum tunneling is applied to explore a different region of the search space. After 60% of the evaluation budget is used, the best solutions are passed to Phase 2, where IPOP-CMA-ES samples new solutions and updates its search distribution based on their performance. If needed, the population size is increased during restarts to encourage further exploration. The process continues until the remaining budget is exhausted, and the best solution found across both phases is returned as the final output.

MATERIALS AND STATISTICAL TOOLS

In this section of the proposal we identify both materials and those statistical procedures we will employ to achieve our proposed study's objectives. Materials include a description of the specific sources and instruments used to collect all data, as well as any software utilized during the experimental analysis of HyDE. Statistical tools determine how results for each research question are analyzed and interpreted.

Table 3.12

Materials Table

Type	Description	Materials
Data Source	Sources from which data was obtained.	Benchmark functions and peer-reviewed journals and conference proceedings from IEEE Xplore, ScienceDirect, Springer, and arXiv, and experimental benchmarking results
Data Collection Instruments	Tools/method used to collect data.	HyDE Python codebase (three encoding variants: binary, qubit-inspired, and continuous real-valued); HyGO re-implementation as the baseline comparator, both executed under a shared evaluation budget across 50 independent runs.
Software and Technologies	Tools for data processing and analysis.	Python 3.10+, NumPy, SciPy, Pingouin, Matplotlib.

Table 3.13

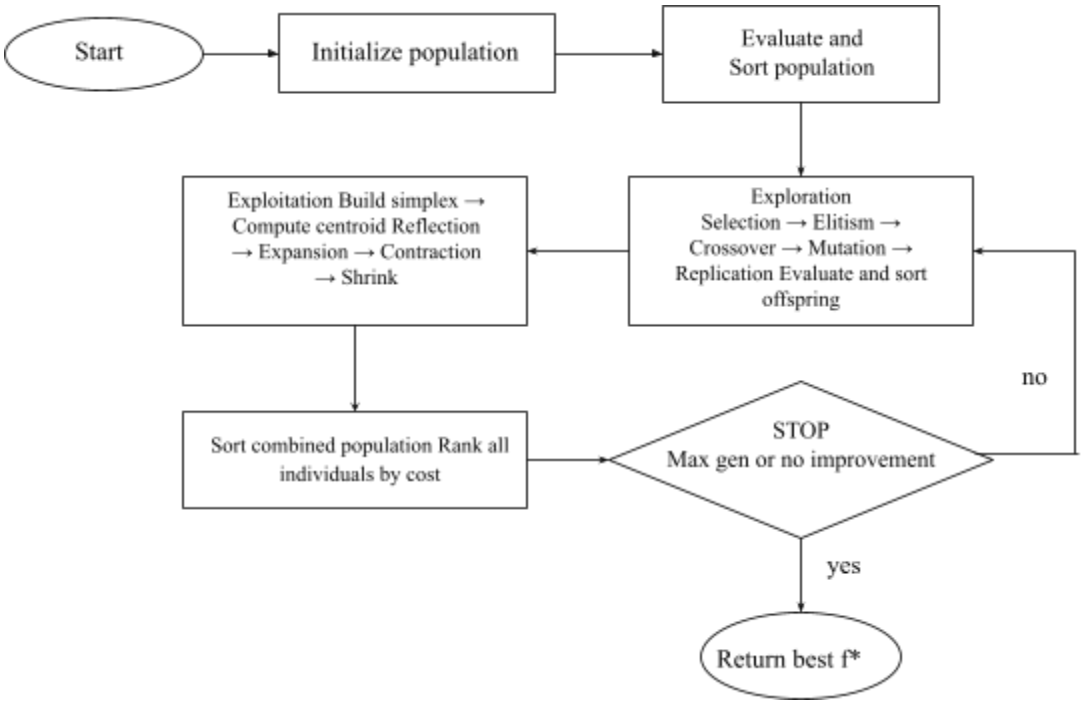
Statistical Tools Table

Type	Description	Statistical Tools
Descriptive Statistics	Techniques to summarize result distributions.	Coefficient of Variation (CV), Friedman test, Nemenyi, Kruskal-Wallis test, Dunn post-hoc with Bonferroni, Cochran's Q test, Chi-square test, Wilcoxon rank-sum test, Cliff's delta, and Bootstrap 95% confidence intervals at $\alpha = 0.05$.
Data Visualization	Visual presentation of performance and comparative results.	Bar charts, heatmaps, and bootstrap confidence interval plots via Matplotlib.

FLOWCHART

Figure 15

HyGO Algorithm Flowchart



The flowchart shown in Figure 3.1 depicts HyGO (the existing system) as the foundation of this research. It starts with creating the initial population through Monte Carlo or Latin Hypercube Sampling, and then evaluates and ranks each member of the population according to their performance using the cost function J.

Each generation consists of 2 phases; an exploration and exploitation phase. During the exploration phase, there is a tournament selection process for genetic diversity through the four genetic operations of elitism, crossover, mutation and replication to generate new individuals who will be evaluated and ranked according to their

performance using the cost function J. The exploitation phase consists of building a simplex out of the best individuals and determining its centroid followed by completing one entire DSM cycle is the reflection, expansion, contraction and shrinkage. After completing both phases, the population is merged and sorted. A decision point checks for stopping conditions such as the maximum number of generations reached or four generations of no improvement.

If so, the best solution (f*) is returned and if not, the generation counter is incremented and the solution will go to another iteration of the exploration phase. This alternating between exploitation applied every generation, independently from whether we are at an exploration or exploitation phase, is a major architectural characteristic that distinguishes HyGO from HyDE.

SYSTEM REQUIREMENTS

Here we define the proposed HyDE framework's system requirements to enable the development, testing, and evaluation of the system by outlining the minimum hardware and software configurations necessary for establishing a consistent, reproducible, and verifiable experimental environment to support the computational demands of the optimization algorithm and the analytical benchmark suite.

Hardware Requirements

This section details the hardware requirements that are necessary for the algorithm to operate effectively.

Table 3.14

Hardware Requirements and Specification

Required Hardware	Specifications
OS	Fedora 40 / Windows 11 or later.
CPU	Intel Core i7 / AMD Ryzen 7 or higher (8 cores minimum).
RAM	8 GB minimum, 16 GB recommended.
Disk	50 GB SSD minimum (for benchmark run logs, convergence histories, and CSV output storage across all 4,000 runs).
GPU	NVIDIA GTX or RTX (optional; all runs are CPU-bound).

Table 3.14 gives the hardware requirements needed to run the HyDE framework as a standalone computing platform. One workstation is required. These specifications represent the minimum hardware necessary to execute 50 independent optimization runs per algorithm variant across all 20 analytical benchmark scenarios under the shared evaluation budget.

Software Requirements

Table 3.15

Software Requirements and Specifications

Particulars	Specification
Language	Python 3.10 or higher.
Scientific Libraries	NumPy, SciPy, Pingouin, Matplotlib.
IDE / Editor	Visual Studio Code or Jupyter Notebook.

The recommended software specifications for the HyDE framework are provided in Table 3.15. The optimization algorithms will be implemented in Python 3.10 as the

designated primary programming language. NumPy will be used for numerical computations, SciPy will be used for statistical testing, and Pingouin and Matplotlib will be used to analyze and visualize results. Depending on the researcher's preference, Visual Studio Code or Jupyter Notebook can be used to develop the HyDE framework.

SYSTEM TRADEOFFS

The system tradeoffs explain the specific requirements of the system in development. They consist of technical, operational, and economic considerations.

Table 3.16

Technical Issues

Technical Issues	Tradeoffs
Performance	The workstation CPU must be at least Intel Core i7 or AMD Ryzen 7 (8 cores) to handle 50 independent optimization runs per algorithm variant across all 20 benchmark scenarios.
Deployment	The system runs locally on a single workstation; no network deployment or cloud infrastructure is required.
Operational Characteristics	A minimum of 8 GB RAM is required to execute HyDE and its statistical analysis pipeline without memory overflow, though 16 GB is recommended for handling large convergence history logs across all 4,000 benchmark runs.
Interoperability	The Python codebase is compatible with both Fedora 40 and Windows 11 or later, ensuring portability across development environments.

The technical tradeoffs of the HyDE framework are presented in Table 3.16. The performance of the system is determined primarily by the capability of the local workstation to execute 4,000 independent benchmark runs across 20 analytical scenarios

and 50 runs per configuration under the evaluation budget. Because the HyDE framework runs entirely on one local computer and does not depend on a web server or internet connection, this significantly simplifies the technical environment; however, it equally creates a limitation in that parallel execution across multiple machines is not supported, meaning all runs are completed sequentially on a single processor.

Table 3.17

Operational Issues

Operational Issues	Tradeoffs
Support Tools	Python 3.10, NumPy, SciPy, Pingouin, Matplotlib, and Visual Studio Code or Jupyter Notebook.
Developer Skills	The developer must be proficient in Python algorithmming and evolutionary computation concepts, including differential evolution mutation strategies, covariance matrix adaptation, and nonparametric statistical analysis.
Process	Each of the 50 independent runs per algorithm variant requires sequential execution of Phase 1 (DE/current-to-best/1/bin global search) and Phase 2 (IPOP-CMA-ES warm-start local refinement), followed by statistical comparison using Kruskal-Wallis with, Wilcoxon rank-sum test, and Cliff's delta effect size analysis.
Documentation	All algorithm parameters, run configurations, random seeds, convergence histories, and performance metrics must be systematically recorded in CSV format to ensure full reproducibility of the experimental results.

The operational tradeoffs for the HyDE framework are shown in Table 3.17. A moderate technical burden is placed on the researcher, as competency in both algorithm design and nonparametric statistical analysis is required to implement, run, and interpret the framework correctly. Nonetheless, this self-contained approach allows the researcher

to evaluate the framework's convergence behavior and solution quality directly against a reimplemented baseline under fully controlled and reproducible experimental conditions.

Table 3.18

Economical Issues

Economic Issues	Tradeoffs
Hardware and Software Updates	The system performs optimally on a workstation with at least 8 GB RAM, a multi-core CPU, and an IDE. All software libraries used are open-source and incur no licensing cost.
Development Cost	The estimated development cost is approximately Php 7,800. This includes expenses for: Coding tools : Php 3,900 Testing tools : Php 3,900 The researchers kept the cost of developing the benchmarking environment low without compromising the quality or integrity of the experiment.
Operational Cost	The forecasted operational cost is approximately Php 12,550, including expenses for: Electricity : Php 1,000 Transportation : Php 500 Food : Php 1,000 Bond Paper (2 Rims) : Php 400 Miscellaneous : Php 9,650
Training Cost	No additional user training cost is required as the system is a standalone research framework operated solely by the developer-researcher.

Table 3.18 demonstrates the economic tradeoffs of the HyDE framework. The principal cost consideration is access to a capable local workstation, as the framework

runs all benchmark trials sequentially on a single machine. The system performs optimally on a workstation with at least 8 GB RAM, a multi-core CPU, and an IDE, so no additional hardware upgrades are required beyond this baseline. No software licensing costs are incurred since HyDE is implemented entirely in Python 3.10 using open-source libraries. The estimated development cost is approximately Php 7,800, divided equally between coding tools (Php 3,900) and testing tools (Php 3,900), which kept the cost of the benchmarking environment low without compromising the quality or integrity of the experiment. The forecasted operational cost is approximately Php 12,550, covering electricity (Php 1,000), transportation (Php 500), food (Php 1,000), bond paper for two reams (Php 400), and miscellaneous expenses (Php 9,650). End users of HyDE incur no training cost since HyDE, as a research tool, is used solely by the proponent for conducting experimental benchmark evaluations.

Chapter 4

SUMMARY OF FINDINGS, CONCLUSIONS, AND RECOMMENDATIONS

The summary of findings, conclusions and recommendations for the study titled "HyDE: Hybrid Differential Evolution with Warm-Start Increasing Population Covariance Matrix Adaptation Evolution Strategy for Continuous Parametric Optimization" are provided in this chapter.

FINDINGS

The following were the key findings obtained from the results of the study:

1. The study revealed that the four limitations of HyGO were continuously observed in the experiments. The performance of HyGO was the worst in the mean objective error, and it failed to achieve as much as 25.5% mean convergence at the budget of 25K. HyGO also revealed a substantial performance drop in most of the scalable benchmark functions on increasing the dimensions from 2D to 25D. The outcomes proved that it had a number of limitations such as binary encoding, memoryless DSM restarts, scalability and statistical validation.

2. These limitations were addressed by all three HyDE variants, including improved initialization, encoding strategies, warm-start IPOP-CMA-ES and statistical validation. HyDE-QUB was the most stable of the variants with regard to the assessed criteria.

3. The researchers discovered that HyDE-QUB had the lowest average rank of 1.68, while HyDE-CON had an average rank of 2.05, HyDE-BIN 2.40 and HyGO 3.93,

and that all the HyDE variants significantly outperformed HyGO. Moreover, the convergence rate of HyDE-QUB was the highest at 73.1%, in contrast to 70.0% for HyDE-BIN, 69.5% for HyDE-CON, and 25.5% for HyGO. HyDE-QUB showed the greatest degree of convergence, at 50K evaluations, with 88.4% convergence, compared to HyGO's 27.0%. On the wall clock cost, the required time for HyDE-CON and HyDE-QUB was around 72 ms and 74 ms, respectively, whereas that for HyGO needed about 354 ms in 5K evaluations. HyDE-QUB also showed the largest practical effect against HyGO with a Cliff's delta of -0.954; and it showed the least amount of dimensionality degradation at all dimensions between 2D and 25D. However, HyGO did not improve on Ackley, Sphere, Rastrigin and Rosenbrock, and continued to degrade significantly at 50K evaluations; its failure was not due to a lack of evaluation.

CONCLUSIONS

Based on the above findings the following conclusions were drawn:

1. The computational problems, statistical issues, and performance shortcomings of HyGO were not side effects but inherent features of its design: the maximum precision attainable by its encoding was fixed by the architecture; the Memoryless Downhill Simplex restarts repeatedly rediscovered directions it had already explored; and the performance benefits reported were not statistically demonstrated. The results of the experiments clearly showed that a new hybrid structure was needed to achieve consistent and statistically sound performance on continuous parametric optimization problems across all 20 benchmark scenarios.

2. All the problems documented in HyGO were solved by the 2-phase framework (HyDE). mounted DE update to global seed, was selected according to its performance. The directionality of the DE mutation for the global search space was chosen according to its performance, as was the warm-start IPOCMA-ES with elite archive seeding to guide the local search, and the LHS initialisation, and the encoding-consistent mounted DE update to global seed. A direct link could be made between the stagnation recovery and the weak point of HyGO: the statistical evaluation based on the effect size. Every design element could be explained by a specific problem with HyGO and the suggested design was a well founded and well thought through solution.

3. HyDE-QUB was the most stable and reliable variant for the continuous parametric optimization, offering the best overall solution quality, convergence reliability, computational efficiency, statistically large improvement margins and scalability when dimensions grew. The results of HyGO were not cost based, for the reason that it was found that it recovered only slightly under the additional money in the evaluation budget; the variants of HyDE, particularly HyDE-QUB were continued and improved.

RECOMMENDATIONS

Based on the conclusions, the following recommendations were made:

1. Further research and analysis on other restrictions of HYGO that were not considered by the study researchers. Future studies can explore the functional optimization side limitations of HYGO, not just the continuous parametric side to identify limitations not revealed in this study. This could expose other vulnerabilities and

offer a more thorough understanding of the framework's shortcomings.

2. Investigate other optimization techniques and setups. Other strategies for CMA-ES (e.g., Bipopulation CMA-ES, phase-split settings and parameter configurations for HYDE) may be explored in future studies. Researchers can also consider the possibility of simplifying the algorithmic implementation, reducing the number of lines of code needed, increasing computational efficiency, and keeping or improving the optimization performance on various benchmark problems.

3. Enhance the computational efficiency of HyDE with a high evaluation budget. At 50,000 evaluations, HyGO became faster than HyDE-QUB and HyDE-CON; even though at 5,000 evaluations HyDE-QUB and HyDE-CON were faster than HyGO. Future work could thus be directed towards minimizing the computational burden of HyDE without compromising its optimization capability as the evaluation budget is increased.

4. Study the particular examples in which the performance of HyDE was not superior to HyGO. Although HyGO won 57 out of 60 contests in the 50,000-evaluation budget, it did better on the Styblinski-Tang 25D problem. Future researchers can investigate characteristics of such problem instances and the possibility of enhancing HyDE's search, adaptation, and/or recovery mechanisms.

5. Test HyDE variants on other application-specific and real-world optimization problems, such as the Ahmed body drag-reduction problem, in future research to evaluate whether the same performance gains hold across various application fields.

REFERENCES

Auger, A., & Hansen, N. (2005). A restart CMA evolution strategy with increasing population size. Proceedings of the 2005 IEEE Congress on Evolutionary Computation, 1769–1776. https://doi.org/10.1109/CEC.2005.155490

Ackley, D. H. (1987). A connectionist machine for genetic hill-climbing. Kluwer Academic Publishers. https://doi.org/10.1007/978-1-4613-1997-9

Alam, F., Watkins, S., Jin, Y., & Hu, X. (2025). Passenger car aerodynamic drag, thermal cooling: A perspective for energy saving and improving the environment. Energies, 18(24), 6433. https://doi.org/10.3390/en18246433

Alejandrino, J., Concepcion, R., II, Bandala, A., Sybingco, E., Vicerra, R. R., & Dadios, E. (2024). Non-invasive plant root tomography through optimized sonar array transducer antenna design using genetic swarm metaheuristic. Journal of Advanced Computational Intelligence and Intelligent Informatics, 28(1), 59–66. https://doi.org/10.20965/jaciii.2024.p0059

Almufti, S. (2025). Metaheuristic algorithms for engineering and combinatorial optimization: A comparative study across problems categories and benchmarks. ResearchGate. https://www.researchgate.net/publication/394881224

Altay, E. V., Altay, O., & Özçevik, Y. (2024). A comparative study of metaheuristic optimization algorithms for solving real-world engineering design problems. Computer Modeling in Engineering & Sciences, 139(1), 1039–1094. https://doi.org/10.32604/cmes.2023.029404

Bernardo, R. C., Enriquez, E. A. T., Mendoza, R. G., Reyes, R., & Velasco, A. C. T. (2025). Nature-inspired optimization, the Philippine Eagle, and cosmological parameter estimation. arXiv preprint arXiv:2505.10299. https://doi.org/10.48550/arXiv.2505.10299

Beale, E. M. L. (1958). On an iterative method for finding a local minimum of a function of more than one variable (Technical Report No. 25). Statistical Techniques Research Group, Princeton University. https://digital.library.unt.edu

Booth, A. D. (1949). Numerical methods. Butterworths Scientific Publications. https://archive.org

Bukin, A. D. (1995). Optimal fitting of a Gaussian profile with a small sample of data. Budker Institute of Nuclear Physics. https://www.inp.nsk.su

Brest, J., Maučec, M. S., & Bošković, B. (2016). iL-SHADE: Improved L-SHADE algorithm for single objective real-parameter optimization. Proceedings of the 2016 IEEE Congress on Evolutionary Computation, 1188–1195. https://doi.org/10.1109/CEC.2016.7743922

Cao, Y., & Luan, J. (2024). A novel differential evolution algorithm with multi-population and elites regeneration. PLOS ONE, 19(4), e0302207. https://doi.org/10.1371/journal.pone.0302207

Concepcion, R., Dadios, E., Bandala, A., Cuello, J., & Kodama, Y. (2021a). Hybrid genetic algorithm and multiverse-based optimization of pre-harvest growth factors of aquaponic lettuce based on chlorophyll concentration. International Journal on

Advanced Science, Engineering and Information Technology, 11(6), 2128–2138. https://doi.org/10.18517/ijaseit.11.6.14991

Concepcion, R. S., II, Alejandrino, J., Mendigoria, C. H., Dadios, E. P., Bandala, A. A., Sybingco, E., & Vicerra, R. R. (2021b). Lactuca sativa leaf extract concentration optimization using evolutionary strategy as photosensitizer for TiO_2-filmed Grätzel cell. Optik, 242, 166931. https://doi.org/10.1016/j.ijleo.2021.166931

Chauhan, D., Shivani, Jung, D., & Yadav, A. (2025). Advancements in multimodal differential evolution: A comprehensive review and future perspectives. Artificial Intelligence Review, 58(11), 335. https://doi.org/10.1007/s10462-025-11314-7

Dadios, E. P., Culaba, A. B., Albert, J. R. G., Paqueo, V. B., Orbeta, A. C., Jr., Serafica, R. B., Bandala, A. A., & Bairan, J. C. A. C. (2018). Preparing the Philippines for the Fourth Industrial Revolution: A scoping study. Philippine Institute for Development Studies Discussion Paper Series No. 2018-11.

Das, S., Mullick, S. S., & Suganthan, P. N. (2016). Recent advances in differential evolution — An updated survey. Swarm and Evolutionary Computation, 27, 1–30. https://doi.org/10.1016/j.swevo.2016.01.004

De Guzman, C. J. P., Chua, A. Y., Chu, T. S., & Secco, E. L. (2023). Evolutionary algorithm-based energy-aware path planning with a quadrotor for warehouse inventory management. HighTech and Innovation Journal, 4(4), 829–837. https://doi.org/10.28991/HIJ-2023-04-04-012

De Ocampo, A. L. P., & Dadios, E. P. (2017). Energy cost optimization in irrigation

system of smart farm by using genetic algorithm. Proceedings of the 2017 IEEE 9th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment and Management (HNICEM), 1–7. https://doi.org/10.1109/HNICEM.2017.8269497

De Jong, K. A. (1975). An analysis of the behavior of a class of genetic adaptive systems [Doctoral dissertation, University of Michigan]. Deep Blue. https://hdl.handle.net/2027.42/4507

Enriquez, E. A. T., Mendoza, R. G., & Velasco, A. C. T. (2022). Philippine Eagle Optimization Algorithm. IEEE Access, 10, 29089–29120. https://doi.org/10.1109/ACCESS.2022.3158357

Easom, E. E. (1990). A survey of optimization techniques. West Virginia University. https://scholar.google.com

Fajfar, I., Bűrmen, Á., & Puhan, J. (2019). The Nelder–Mead simplex algorithm with perturbed centroid for high-dimensional function optimization. Optimization Letters, 13, 1011–1025. https://doi.org/10.1007/s11590-018-1306-2

Goldberg, D. E. (1989). Genetic algorithms in search, optimization, and machine learning. Addison-Wesley.

Goldstein, A. A., & Price, J. F. (1971). On descent from local minima. Mathematics of Computation, 25(115), 569–574. https://doi.org/10.1090/S0025-5718-1971-0312365-X

Gupta, A. (2021). Comparative study of different SDLC models. International Journal for

Research in Applied Science and Engineering Technology, 9(XI). https://doi.org/10.22214/ijraset.2021.38736

Han, K. H., & Kim, J. H. (2002). Quantum-inspired evolutionary algorithm for a class of combinatorial optimization. IEEE Transactions on Evolutionary Computation, 6(6), 580–593. https://doi.org/10.1109/TEVC.2002.804320

Hansen, N. (2016). The CMA evolution strategy: A tutorial. arXiv. https://arxiv.org/abs/1604.00772

Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. Evolutionary Computation, 9(2), 159–195. https://doi.org/10.1162/106365601750190398

He et al. (2023). Impact of spatial encoding methods on differential evolution. 2023 IEEE Congress on Evolutionary Computation, 1–8. https://doi.org/10.1109/CEC53210.2023

Himmelblau, D. M. (1972). Applied nonlinear algorithmming. McGraw-Hill. https://archive.org

International Energy Agency. (2023). Road transport. IEA. https://www.iea.org/reports/road-transport

Investigating the use of local search for improving meta-hyper-heuristic performance. (2026). ResearchGate. https://www.researchgate.net/publication/256495201

Jamilla, C. U., Mendoza, R. G., & Mendoza, V. M. P. (2021). Parameter estimation in neutral delay differential equations using genetic algorithm with multi-parent

crossover. IEEE Access, 9, 131348–131364. https://doi.org/10.1109/ACCESS.2021.3113677

Katoch, S., Chauhan, S. S., & Kumar, V. (2021). A review on genetic algorithm: Past, present, and future. Multimedia Tools and Applications, 80(5), 8091–8126. https://doi.org/10.1007/s11042-020-10139-6

Kononova, A. V., Stein, N. van, Mersmann, O., Bäck, T., Bartz-Beielstein, T., Glasmachers, T., Hellwig, M., Krey, S., Kůdela, J., Naujoks, B., Papenmeier, L., Raponi, E., Renau, Q., Rook, J., Schäpermeier, L., Vermetten, D., & Zaharie, D. (2025). Benchmarking that Matters: Rethinking Benchmarking for Practical Impact (arXiv:2511.12264). arXiv. https://doi.org/10.48550/arXiv.2511.12264

Li, Y., Cui, W., Jia, Q., Li, Q., Yang, Z., Morzyński, M., & Noack, B. R. (2022). Explorative gradient method for active drag reduction of the fluidic pinball and slanted Ahmed body. Journal of Fluid Mechanics, 932, A7. https://doi.org/10.1017/jfm.2021.974

Liang, J., Lin, H., Yue, C., Suganthan, P. N., & Wang, Y. (2024). Multiobjective differential evolution for higher-dimensional multimodal multiobjective optimization. IEEE/CAA Journal of Automatica Sinica, 11(6), 1458–1475. https://doi.org/10.1109/JAS.2024.124377

Lévi, B. M. (1981). Numerical methods for unconstrained optimization. Academic Press. https://www.worldcat.org

Malonzo, I. M., Patacsil, T. L., Morano, J., Agustin, V., & Dioses, R. (2024).

Enhancement of genetic algorithm by J. Zhang applied to tour planning. International Journal of Advanced Trends in Computer Science and Engineering, 13(2), 92–96. https://doi.org/10.30534/ijatcse/2024/091322024

Matyas, J. (1965). Random optimization. Automation and Remote Control, 26(2), 244–251. https://www.semanticscholar.org

Mendigoria, C. H., Concepcion, R., Bandala, A., Bautista, M. G., Dadios, E., & Cuello, J. (2023). Optimization of the Hargreaves, HamonV1, and Penman potential evapotranspiration models using bio-inspired algorithms. International Journal on Electrical Engineering and Informatics, 15(2), 240–258. https://doi.org/10.15676/ijeei.2023.15.2.5

Mohamed, A. W., Hadi, A. A., & Jambi, K. M. (2019). Novel mutation strategy for enhancing SHADE and L-SHADE algorithms for global numerical optimization. Swarm and Evolutionary Computation, 50, 100455. https://doi.org/10.1016/j.swevo.2018.10.006

Montgomery, D. C. (2017). Design and Analysis of Experiments (9th ed.). https://bcs.wiley.com/he-bcs/Books?action=index

Mishra, S. K. (2006). Global optimization by differential evolution and particle swarm methods: Evaluation on some benchmark functions. North-Eastern Hill University. https://doi.org/10.2139/ssrn.933827

McGeoch, C. C. (2012). A guide to experimental algorithmics. Cambridge University Press.https://www.cambridge.org/core/books/guide-to-experimental-algorithmics/

CDB0CB718F6250E0806C909E1D3D1082

McKay, M. D., Beckman, R. J., & Conover, W. J. (1979). A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. Technometrics, 21(2), 239–245. https://doi.org/10.2307/1268522

Naser, M. Z., Al-Bashiti, M. K., Tapeh, A. T. G., Naser, A., Kodur, V., Hawileh, R., Abdalla, J., Khodadadi, N., Gandomi, A. H., & Eslamlou, A. D. (2025). A review of benchmark and test functions for global optimization algorithms and metaheuristics. WIREs Computational Statistics, 17, e70028. https://doi.org/10.1002/wics.70028

Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. The Computer Journal, 7(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308

Nomura, M., Akimoto, Y., & Ono, I. (2023). CMA-ES with learning rate adaptation: Can CMA-ES with default population size solve multimodal and noisy problems? Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2023). https://doi.org/10.1145/3583131.3590358

Nomura, M., Watanabe, S., Akimoto, Y., Ozaki, Y., & Onishi, M. (2021). Warm starting CMA-ES for hyperparameter optimization. Proceedings of the AAAI Conference on Artificial Intelligence, 35(10), 9188–9196. https://doi.org/10.1609/aaai.v35i10.17109

Rastrigin, L. A. (1974). Системы экстремального управления [Systems of extremal control]. Nauka. https://www.worldcat.org

Robledo, I., Li, Y., Cornejo Maceda, G. Y., & Castellanos, R. (2025). Fast and robust parametric and functional learning with Hybrid Genetic Optimisation (HyGO). arXiv preprint arXiv:2510.09391. https://doi.org/10.48550/arXiv.2510.09391

Roquel, K. I. D. Z., Abad, R. P. B., Fillone, A. M., & Dadios, E. P. (2015). Optimal scheduling of buses along Epifanio Delos Santos Avenue (EDSA), Metro Manila using genetic algorithm. Proceedings of the 2015 IEEE 8th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment and Management (HNICEM), 1–6. https://doi.org/10.1109/HNICEM.2015.7393262

Rosenbrock, H. H. (1960). An automatic method for finding the greatest or least value of a function. The Computer Journal, 3(3), 175–184. https://doi.org/10.1093/comjnl/3.3.175

Sa, G., et al. (2025). AI-driven design: Powered by large language model and intelligent computation. International Journal of Extreme Manufacturing, 7, 063001. https://doi.org/10.1088/2631-7990/adea23

Salgotra, R., & Gandomi, A. H. (2024). A novel multi-hybrid differential evolution algorithm for optimization of frame structures. Scientific Reports, 14, 4877. https://doi.org/10.1038/s41598-024-54384-3

Slowik, A., & Kwasnicka, H. (2020). Evolutionary algorithms and their applications to engineering problems. Neural Computing and Applications, 32(16), 12363–12379. https://doi.org/10.1007/s00521-020-04832-8

Standardized reporting of statistical results. (2025). Asian Journal of Advanced Research and Reports.

https://files.sdiarticle5.com/wp-content/uploads/2025/02/Ms_AJARR_130468.pdf

Simon, D. (2013). Evolutionary optimization algorithms. John Wiley & Sons. https://doi.org/10.1002/9781118659502

Skiena, S. S. (2020). The Algorithm Design Manual (3rd ed.). Springer. https://doi.org/10.1007/978-3-030-54256-6

Stanovov, V., Akhmedova, S., & Semenkin, E. (2021). NL-SHADE-RSP algorithm with adaptive archive and selective pressure for CEC 2021 numerical optimization. Proceedings of the 2021 IEEE Congress on Evolutionary Computation, 809–816. https://doi.org/10.1109/CEC45853.2021.9504959

Storn, R., & Price, K. (1997). Differential evolution — A simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization, 11(4), 341–359. https://doi.org/10.1023/A:1008202821328

United Nations. (2015). Transforming our world: The 2030 agenda for sustainable development. https://sdgs.un.org/2030agenda

Styblinski, M. A., & Tang, T. S. (1990). Experiments in function optimization: 2D function fitting and global optimization using a stochastic approximation approach. Neural Networks, 3(1), 13–21. https://doi.org/10.1016/0893-6080(90)90029-K

Wang, X., Wang, Z., & Ni, B. (2025). Quantum-classical hybrid genetic evolutionary

algorithm for topology optimization of continuum structures. International Journal for Numerical Methods in Engineering. https://doi.org/10.1002/nme.70073

Xu, H., Deng, Q., Zhang, Z., & Lin, S. (2025). A hybrid differential evolution particle swarm optimization algorithm based on dynamic strategies. Scientific Reports, 15, 4518. https://doi.org/10.1038/s41598-024-82648-5

Yang, X. S. (2020). Nature-inspired optimization algorithms: Challenges and open problems. Journal of Computational Science, 46, 101104. https://doi.org/10.1016/j.jocs.2020.101104

Yang, Z., Sun, Z., Yue, T. Z., Devanbu, P., & Lo, D. (2024). Robustness, Security, Privacy, Explainability, Efficiency, and Usability of Large Language Models for Code (Version 1). arXiv. https://doi.org/10.48550/ARXIV.2403.07506

Zhang, G. (2011). Quantum-inspired evolutionary algorithms: A survey and empirical study. Journal of Heuristics, 17(3), 303–351. https://doi.org/10.1007/s10732-010-9136-0

Zhao, Q., Duan, Q., Yan, B., Cheng, S., & Shi, Y. (2023). Automated Design of Metaheuristic Algorithms: A Survey. https://doi.org/10.48550/ARXIV.2303.06532

Appendix A

GANTT CHART

This section showcases the iterative process of planning, testing, and developing and refining of the study visually, through a gantt chart visualization shown below.

	JAN				FEB				MAR				APR				MAY					JUN				JUL					AUG				SEP			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	5	1	2	3	4	1	2	3	4	5	1	2	3	4	1	2	3	4
PLAN																																						
Formulate a Question	■	■			■				■																													
Examine gaps in the HyGO study	■	■			■				■																													
Define the problem mathematically	■	■			■				■																													
Establish performance metrics	■	■			■				■																													
Build Tools	■	■			■				■																													
Develop test algorithm in Python	■	■			■				■																													
Set up NumPy, SciPy, Matplotlib, Pandas	■	■			■				■																													
Test and verify all tools	■	■			■				■																													
Design Experiment		■				■				■																													
Define metrics, benchmarks, trials, and stopping criteria		■				■				■																													
Verify tools and inputs are ready		■				■				■																													
EXECUTE																																						
Run experiment/ Gather Data		■				■	■			■	■																												
Run algorithms on benchmark problems		■				■	■			■	■																												
Collect output data during experiment		■				■	■			■	■																												
Record and store all trial results		■				■	■			■	■																												
Analyze Data			■	■			■	■			■	■	■																										
Examine all collected data			■	■			■	■			■	■																											
Determine if results answer research questions			■	■			■	■			■	■																											
Return to planning if results are inconclusive			■	■			■	■			■	■																											
Publish													■	■	■	■	■	■												■	■	■	■	■				
Share validated results via research papers													■	■	■	■	■	■																				
Present clear conclusions on algorithm performance																														■	■	■	■	■				
Identify new questions for future research																														■	■	■	■	■				

LEGEND

Complete ■

Incomplete □

Appendix B

SOURCE CODE

Two-phase hybrid global optimizer for the road-network route-cost tuning stage of the SegRut system, combining a qubit-encoded Differential Evolution (DE) global search phase with an IPOP-CMA-ES local refinement phase that warm-starts from an archive of the best solutions found during Phase 1.

```
HyDE_qub — Hybrid Differential Evolution (Qubit Variant)

Two-Phase Optimization: DE Global Search + IPOP-CMA-ES
WHAT THIS FILE IS ABOUT
This implements a hybrid global optimizer for minimizing an
arbitrary black-box cost function J(x; theta) over a bounded
real-valued search domain.
It works in two phases:
  1) A Differential Evolution (DE) phase that encodes candidate
     solutions as "qubit" probability amplitudes in [0, 1] and
     performs DE/rand/1/bin mutation + crossover to explore the
     search space broadly. A "quantum tunnelling" mechanism kicks
     in if the population stagnates (no improvement for several
     generations), perturbing the weakest individuals to escape
     local optima.
```

2) An IPOP-CMA-ES phase (Increasing Population CMA Evolution Strategy) that warm-starts from the best solutions found by DE (kept in an "archive"), and refines the search locally by adapting a full covariance matrix. Each time CMA-ES stalls, it restarts with a doubled population size to try to escape the current basin of attraction.

The two phases are budgeted by function-evaluation counts (P1_budget for DE, remaining budget up to M for CMA-ES), so the algorithm balances broad exploration against focused exploitation.

```
import numpy as np

 HELPER STUBS
 These represent the encoding/decoding and fitness functions.
Replace with your actual implementations.

---

def qubit_encode(x):
    """
    WHAT: Maps a real-valued individual x into the "qubit" domain [0, 1]^D
    by min-max normalising its own components.
    WHY: DE mutation/crossover in this algorithm is performed in a
    normalised [0, 1] space rather than directly in the (possibly very
    different-scaled) real search domain, which keeps mutation step
    sizes consistent across dimensions.
    """
    return (x - x.min()) / (x.max() - x.min() + 1e-12)

def qubit_decode(z, lo, hi):
    """
    WHAT: Inverse of qubit_encode — maps a qubit-encoded vector z in
    [0, 1]^D back into the real search domain [lo, hi] (per-dimension).
    WHY: The fitness function is only ever evaluated in real coordinates,
```

```
    so every qubit-space individual must be decoded before evaluation.
    """
    return lo + z * (hi - lo)

def fitness(x, theta):
    """
    WHAT: The objective / cost function J(x; theta) being minimized.
    WHY: This is a placeholder — plug in your actual problem-specific
    cost function here. It must accept a real-valued vector x (shape D)
    and return a single scalar cost.
    """
    raise NotImplementedError("Define your objective function here.")

def quantum_tunnelling(individuals, lo, hi):
    """
    WHAT: Stagnation-recovery operator for qubit-encoded individuals.
    Randomly perturbs each individual's amplitudes within a small
    window and clips them back into valid [0, 1] bounds.
    WHY: When the DE population hasn't improved for s_trigger
    generations, this injects diversity into the weakest half of the
    population so the search can "tunnel" out of a local optimum
    instead of converging prematurely.
    """
    recovered = []
    for ind in individuals:
        perturbed = np.clip(ind + np.random.uniform(-0.1, 0.1, size=ind.shape), 0, 1)
        recovered.append(perturbed)
    return recovered

def lhs_init(N, D, lo, hi):
    """
    WHAT: Generates the initial population of N individuals across D
    dimensions within [lo, hi], intended to use Latin Hypercube
    Sampling with farthest-point reordering for good space coverage.
    WHY: A well-spread initial population improves the odds that DE's
    global search phase samples all regions of the search space rather
    than clustering by chance. (Currently a plain uniform-random
    placeholder — swap in true LHS + farthest-point reordering.)
    """
    samples = np.random.uniform(lo, hi, size=(N, D))  # Placeholder
    return samples
```

MAIN ALGORITHM: HyDE_qub

```
def HyDE_qub(
    theta,                # Problem parameters passed to fitness function
    N       = 30,         # Population size
    D       = 10,         # Problem dimensionality
    M       = 10_000,     # Total function evaluation budget
    P1_budget = 5_000,    # Budget allocated to Phase 1 (DE)
    N_arc   = 20,         # Archive size (top individuals kept)
    CR      = 0.9,        # Crossover rate for binomial crossover
    F_lo    = 0.4,        # Lower bound for adaptive scaling factor
    F_hi    = 0.9,        # Upper bound for adaptive scaling factor
    s_trigger = 15,       # Stagnation counter threshold for recovery
    lambda_base = 10,     # Initial CMA-ES offspring count
    k_max   = 10,         # Max IPOP restarts in Phase 2
    lo      = -5.0,       # Lower bound of search domain
    hi      =  5.0,       # Upper bound of search domain
):

    WHAT: Entry point that runs the full two-phase HyDE_qub optimizer
    and returns the best solution found.
    WHY / HOW: Phase 1 (qubit-encoded DE) does broad exploration and
    builds an archive of the best solutions seen so far. Phase 2
    (IPOP-CMA-ES) warm-starts its search distribution from that
    archive and exploits it to converge on a refined optimum, with
    population-doubling restarts (IPOP) to avoid getting stuck.

    Returns:
        x_best (np.ndarray): best real-valued solution vector found.
        J_best (float): the corresponding best (lowest) cost.
    """

    # ── STEP 1: Initialisation ──────────────────────────────────────
    lo, hi = np.full(D, lo), np.full(D, hi)
```

```
# Generate N individuals using Latin Hypercube Sampling
# with farthest-point reordering for better space coverage
population = lhs_init(N, D, lo, hi)                    # Shape: (N, D)

# Encode each individual into qubit representation [0, 1]^D
encoded = np.array([qubit_encode(x) for x in population])  # Shape: (N, D)

# Evaluate initial fitness for all individuals
costs = np.array([fitness(x, theta) for x in population])  # Shape: (N,)

# Initialise archive with the top N_arc individuals by cost
archive_idx = np.argsort(costs)[:N_arc]
archive = list(zip(population[archive_idx], costs[archive_idx]))

# Track the global best solution and counters
best_idx   = np.argmin(costs)
x_best     = population[best_idx].copy()
J_best     = costs[best_idx]
s          = 0             # Stagnation counter
N_eval     = N             # Evaluations used so far

print(f"[Init] Best cost: {J_best:.6f} | Evals used: {N_eval}")
```

```
#  PHASE 1: DIFFERENTIAL EVOLUTION — GLOBAL SEARCH
#  Explores the search space broadly using DE/rand/1/bin
#  with qubit-encoded individuals.
```

```
while N_eval < P1_budget and N_eval < M:

    for i in range(N):

        ── Mutation ──────────────────────────────────────────────
        # Select two distinct random indices r1, r2 ≠ i
        candidates = [r for r in range(N) if r != i]
        r1, r2 = np.random.choice(candidates, size=2, replace=False)

        Adaptive scaling factor drawn uniformly from [F_lo, F_hi]
        F = np.random.uniform(F_lo, F_hi)
```

```
 DE/rand/1 mutation in the qubit space
 mutant = base + F * (r1 - r2)
mutant = encoded[i] + F * (encoded[r1] - encoded[r2])

 Clip mutant to valid qubit bounds [0, 1]
mutant = np.clip(mutant, 0, 1)

── Crossover ──────────────────────────────────────────────
# Binomial crossover between current individual and mutant
# Ensures at least one dimension comes from the mutant
mask  = np.random.rand(D) < CR
mask[np.random.randint(D)] = True          # Guaranteed swap
trial_encoded = np.where(mask, mutant, encoded[i])

 ── Decode & Evaluate ──────────────────────────────────────
 Decode the trial vector back to the real domain
x_trial = qubit_decode(trial_encoded, lo, hi)
J_trial = fitness(x_trial, theta)

── Selection ──────────────────────────────────────────────
Greedy selection: keep trial if it is at least as good
if J_trial <= costs[i]:
    encoded[i]    = trial_encoded
    population[i] = x_trial
    costs[i]      = J_trial

___Update archive with this trial (keep top N_arc by cost)
archive.append((x_trial, J_trial))
archive.sort(key=lambda a: a[1])
archive = archive[:N_arc]

Count evaluations for this generation
 N_eval += N

── Stagnation Detection ──────────────────────────────────
Check if the global best has improved this generation
 gen_best = costs.min()
 if gen_best < J_best:
    J_best = gen_best
    x_best = population[np.argmin(costs)].copy()
    s = 0                                  Reset stagnation counter
 else:
```

```
        s += 1                          Increment stagnation counter

    ── Stagnation Recovery ──────────────────────────────────────────────
        If no improvement for s_trigger consecutive generations,
        apply quantum tunnelling to the worst half of the population
        if s >= s_trigger:
            worst_idx = np.argsort(costs)[::-1][:N // 2]
            recovered_encoded = quantum_tunnelling(
                [encoded[idx] for idx in worst_idx], lo, hi
            )

            Replace worst individuals with recovered ones and re-evaluate
            for j, idx in enumerate(worst_idx):
                encoded[idx]    = recovered_encoded[j]
                population[idx] = qubit_decode(recovered_encoded[j], lo, hi)
                costs[idx]      = fitness(population[idx], theta)

                Update archive with recovered individuals
                archive.append((population[idx], costs[idx]))
                archive.sort(key=lambda a: a[1])
                archive = archive[:N_arc]

            N_eval += len(worst_idx)
            s = 0                               Reset counter after recovery
            print(f"[Recovery] Quantum tunnelling applied at eval {N_eval}")

    print(f"[Phase 1 Done] Best cost: {J_best:.6f} | Evals used: {N_eval}")

      ____PHASE 2: IPOP-CMA-ES WITH WARM COVARIANCE

     Exploits the archive from Phase 1 to warm-start CMA-ES.
      IPOP doubles the population on each restart to escape
      local basins that smaller populations miss.


    # ── Warm-start covariance from archive scatter ──────────────
    archive_X = np.array([a[0] for a in archive])       Shape: (N_arc, D)
    C = np.cov(archive_X.T) + 1e-8 * np.eye(D)          Regularised covariance

    — Initialise CMA-ES state from the archive
    m     = archive[0][0].copy()                    Mean = best archive solution
    sigma = np.mean(np.std(archive_X, axis=0))          Step size from archive spread
```

```
lam   = lambda_base                    Current offspring count
k     = 0                              Restart counter

while N_eval < M and k < k_max:

    ── CMA-ES Parameter Setup ──────────────────────────────────────
    mu   = lam // 2                    Number of selected parents
    w    = np.log(mu + 0.5) - np.log(np.arange(1, mu + 1))
    w   /= w.sum()                     #Normalised recombination weights
    mu_eff = 1.0 / (w ** 2).sum()          # Effective selection mass

    # Adaptation constants (Hansen's standard formulas)
    c_s  = (mu_eff + 2) / (D + mu_eff + 5)
    d_s  = 1 + 2 * max(0, np.sqrt((mu_eff - 1) / (D + 1)) - 1) + c_s
    c_c  = (4 + mu_eff / D) / (D + 4 + 2 * mu_eff / D)
    c_1  = 2 / ((D + 1.3) ** 2 + mu_eff)
    c_mu = min(1 - c_1, 2 * (mu_eff - 2 + 1 / mu_eff) / ((D + 2) ** 2 + mu_eff))

    —Expected norm of a standard normal vector (chi approximation)
    chi  = np.sqrt(D) * (1 - 1 / (4 * D) + 1 / (21 * D ** 2))

    # Initialise evolution paths
    p_s  = np.zeros(D)                 # Step-size control path
    p_c  = np.zeros(D)                 # Covariance adaptation path

    ── Eigendecomposition of Covariance Matrix ──────────────
     C = B D² Bᵀ  where B = eigenvectors, D = sqrt(eigenvalues)
    eigvals, B = np.linalg.eigh(C)
    eigvals    = np.maximum(eigvals, 1e-20)      Numerical safety floor
    D_mat      = np.diag(np.sqrt(eigvals))       Diagonal sqrt eigenvalue matrix

     ── Inner Loop: Sample, Evaluate, Update ──────────────────
    while N_eval < M and (M - N_eval) >= lam:

        # Sample lam offspring from the current distribution
        Z = np.random.randn(lam, D)
        X = m + sigma * (Z @ D_mat @ B.T)        # Transform to search space
        X = np.clip(X, lo, hi)                 # Enforce domain bounds

        # Evaluate all offspring
        F_vals  = np.array([fitness(x, theta) for x in X])
        N_eval += lam
```

```
        —Select the top mu offspring (lowest cost)
        idx_sorted = np.argsort(F_vals)
        X_sel      = X[idx_sorted[:mu]]          # Selected parents
        Z_sel      = Z[idx_sorted[:mu]]          # Corresponding noise vectors

        # Update distribution mean as weighted sum of selected parents
        m_new = w @ X_sel

         ── Evolution Path Updates ──────────────────────────────────
        Step-size path (cumulative step-size adaptation)
        p_s = (1 - c_s) * p_s + np.sqrt(c_s * (2 - c_s) * mu_eff) * (B @ D_mat @ w @
Z_sel)

        # Covariance path (rank-one update path)
        h_sig = (np.linalg.norm(p_s) / np.sqrt(1 - (1 - c_s) ** (2 * N_eval / lam)) / chi
                < 1.4 + 2 / (D + 1))
        p_c   = (1 - c_c) * p_c + h_sig * np.sqrt(c_c * (2 - c_c) * mu_eff) * (m_new - m) /
sigma

        ── Covariance Matrix Update ──────────────────────────────
         Rank-one + rank-mu update
        C = ((1 - c_1 - c_mu) * C
            + c_1 * np.outer(p_c, p_c)
            + c_mu * sum(w[j] * np.outer((X_sel[j] - m) / sigma,
                                         (X_sel[j] - m) / sigma)
                         for j in range(mu)))

        m = m_new                              # Advance mean

        ── Step-Size Update (CSA) ──────────────────────────────────
        sigma = sigma * np.exp((c_s / d_s) * (np.linalg.norm(p_s) / chi - 1))

        # Update global best if a better solution was found
        best_in_gen = F_vals[idx_sorted[0]]
        if best_in_gen < J_best:
            J_best = best_in_gen
            x_best = X[idx_sorted[0]].copy()

         ── Numerical Safeguards ────────────────────────────────────
        if sigma < 1e-11 or not np.all(np.isfinite(C)):
            print(f"[CMA-ES] Numerical issue detected — breaking inner loop.")
```

```
            break
        IPOP Restart
```

```
Doubling the offspring count (IPOP) on each restart gives
    CMA-ES a wider search distribution, helping it escape a
        # basin that a smaller population had already converged into.
        k   += 1
        lam  = min(2 * lam, M - N_eval)             # Double offspring for next restart
        if lam < 4:
            break                                   # Budget too small to continue

        # Reset covariance and re-seed mean from archive
        C = np.eye(D)
        m = archive[np.random.randint(len(archive))][0].copy()

        print(f"[Phase 2] Restart {k} | lam={lam} | Best cost: {J_best:.6f} | Evals:
{N_eval}")
```

```
    TERMINATION
    Return the best solution found across both phases

    print(f"\n[Done] x* found with cost {J_best:.8f} after {N_eval} evaluations.")
    return x_best, J_bes
```

Full source code at: https://github.com/zorncbllr/hyde-hygo-benchmarking-suite

Appendix C

BENCHMARK RESULTS

Experiments were performed in 20 benchmark problems altogether for each of algorithms with 50 runs and 5,000 evaluations for all except for 5 scalable 25D problems with 50,000 evaluations. The complete results are provided in Chapter 3.

When speaking about mean final objective error, HyDE-QUB was the most accurate with the mean Friedman rank of 1.68, then went HyDE-CON (2.05), HyDE-BIN (2.40) and HyGO (3.93). HyDE was statistically significantly more accurate than HyGO, and there was no statistical difference between HyDE variants.

Concerning convergence rate for 50 runs, HyDE-QUB showed the highest result 73.1%, HyDE-BIN showed 70.0% and HyDE-CON showed 69.5%, HyGO – only 25.5%. With 50,000 evaluations, the convergence rate reached 88.4% for HyDE-QUB, 81.6% for HyDE-CON, 81.0% for HyDE-BIN, but HyGO still 27.0%.

Concerning time per run, HyDE-CON was the fastest (72 ms), HyDE-BIN (122 ms), but HyGO still the slowest (354 ms) within extended budget.

With respect to practically valuable margin of improvement, HyDE-QUB, HyDE-CON and HyDE-BIN exhibited significant effect sizes measured by Cliff's delta test in the comparative analysis of HyGO, specifically -0.954, -0.841 and -0.816 respectively.

As regards the scalability in 2D to 25D for 5 scalable functions, only HyDE-QUB did not exhibit any statistically significant decrease of performance at high dimensionality. HyGO demonstrated considerable degradation of performance at 4 out of 5 scalable functions, and the extended budget did not contribute to performance improvement substantially.

In overall result, HyDE-QUB proved to be the most convergent and scalable variant among the other variants algorithms for the benchmarks, HyDE-BIN turned out to be the best for smooth low dimensional problems, and HyDE-CON demonstrated the fastest wall clock time if it was the criterion.

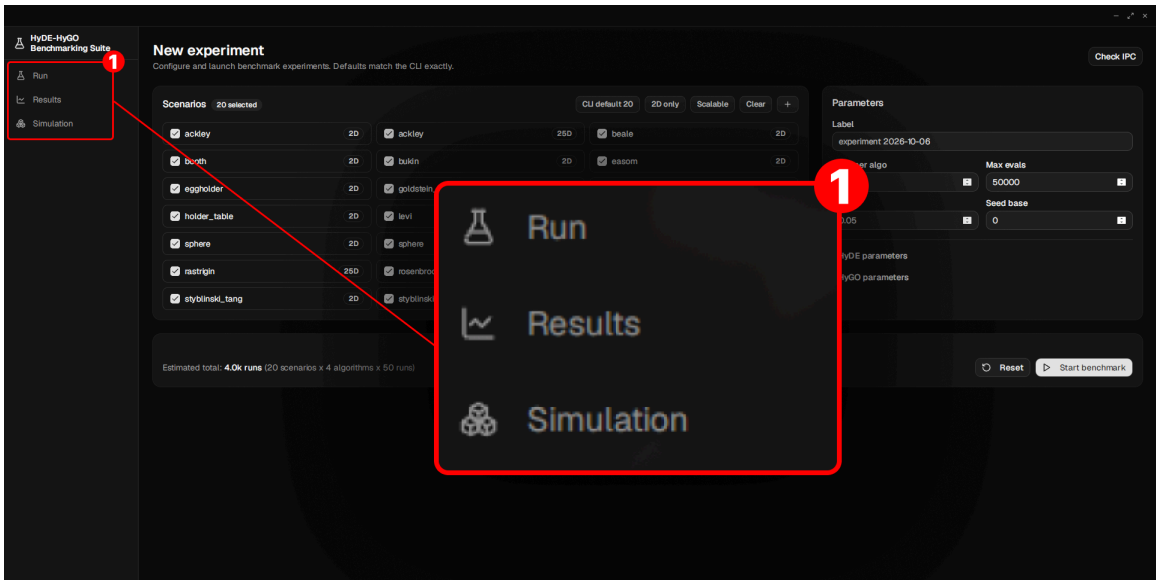
Appendix D

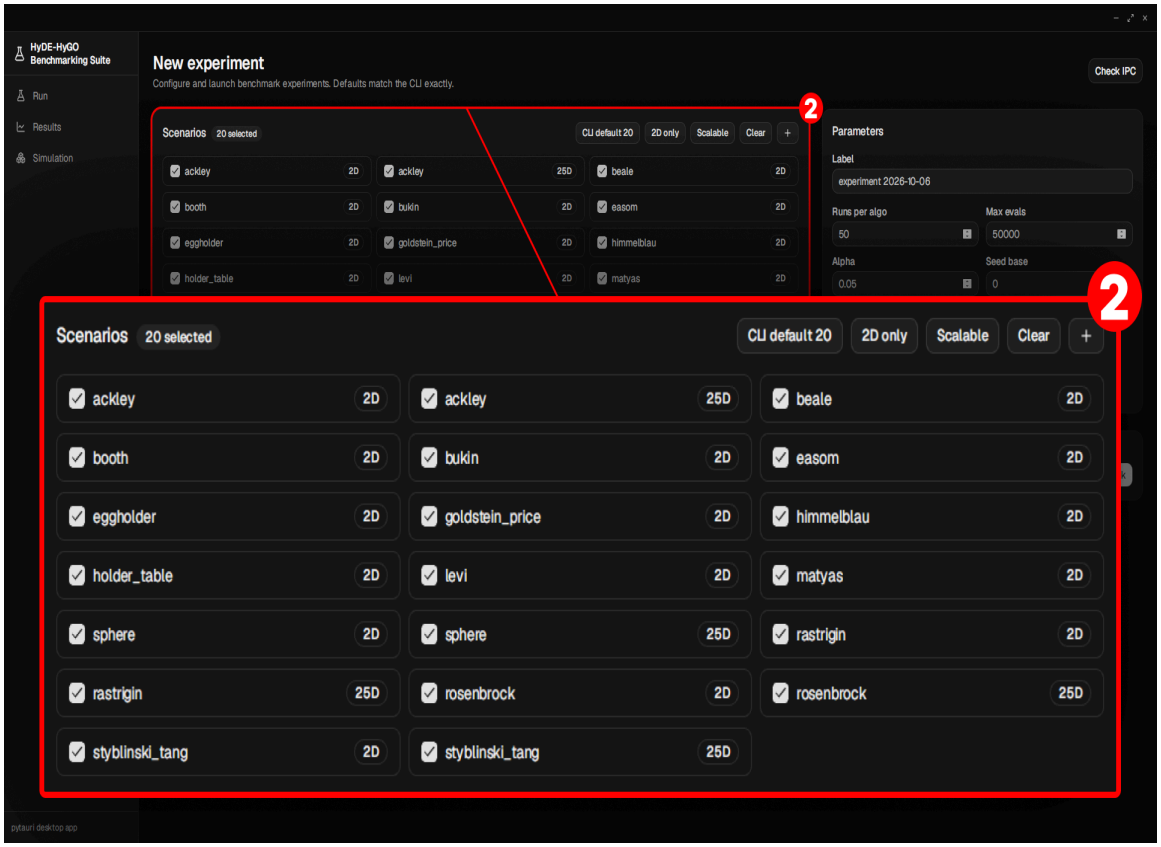
SIMULATIONS

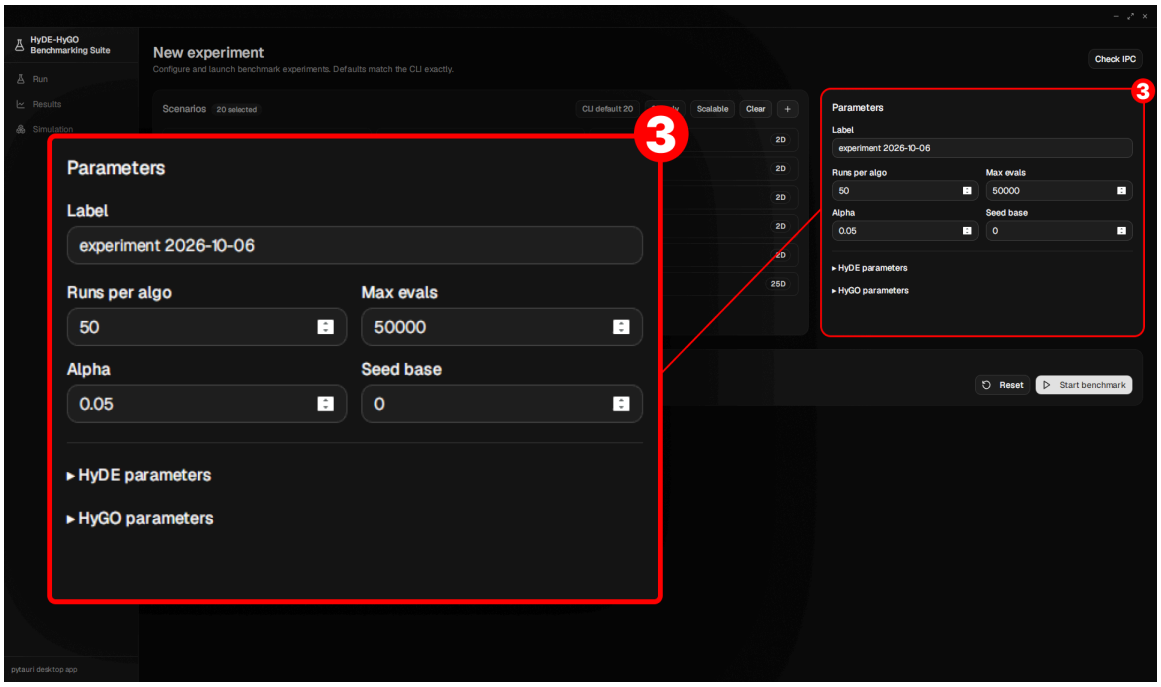
ACCESS THE NEW EXPERIMENT SCREEN

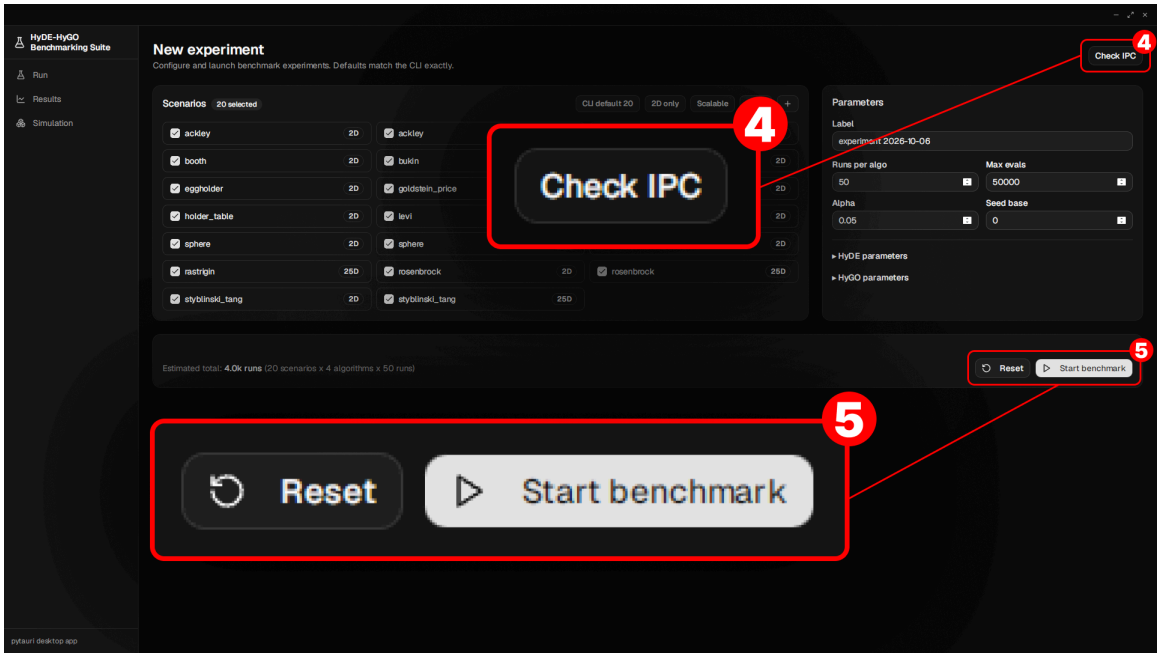
Once users open the **BenchSuite**, they will be directed by default to the **Run Page**, where users can:

1. Move between the major application areas using **Sidebar Menu** items (**Run**, **Results**, or **Simulation**);
2. Select **Scenarios** checkboxes or use the shortcut buttons (**CLI Default 20**, **2D Only**, **Scalable**, **Clear**, and +) to choose which test functions (standard problems for algorithm comparison) will be benchmarked in 2D or 25D;
3. Set up the **Parameters Panel** items: **Label** (name of experiment), **Runs per Algo** (number of repetitions per algorithm), **Max Evals** (number of function evaluations per repetition), **Alpha**, and **Seed Base** (the initial value of the random seed), or open advanced settings in **HyDE Parameters** and **HyGO Parameters**;
4. Use **Check IPC** to check the connection between the application interface and the Python backend before running; and
5. Use **Reset** to reset the settings or **Start Benchmark** to start the experiment, with the number of runs shown as **Estimated Total Runs**.




HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
New experiment
Configure and launch benchmark experiments. Defaults match the CLI exactly.
Check IPC
Scenarios 20 selected
CLI default 20
2D only
Scalable
Clear
ackley 2D
ackley 25D
beale 2D
booth 2D
bukin 2D
easom 2D
eggholder 2D
goldstein_price 2D
himmelblau 2D
holder_table 2D
levi 2D
matyas 2D
sphere 2D
sphere 25D
rastrigin 2D
rastrigin 25D
rosenbrock 2D
rosenbrock 25D
styblinski_tang 2D
styblinski_tang 25D
Parameters
Label
experiment 2026-10-06
Runs per algo
50
Max evals
50000
Alpha
0.05
Seed base
0
2


HyDE-HyGO Benchmarking Suite
New experiment
Configure and launch benchmark experiments. Defaults match the CLI exactly.
Check IPC
Parameters
Label
experiment 2026-10-06
Runs per algo
50
Max evals
50000
Alpha
0.05
Seed base
0
▸ HyDE parameters
▸ HyGO parameters
Reset
Start benchmark
3

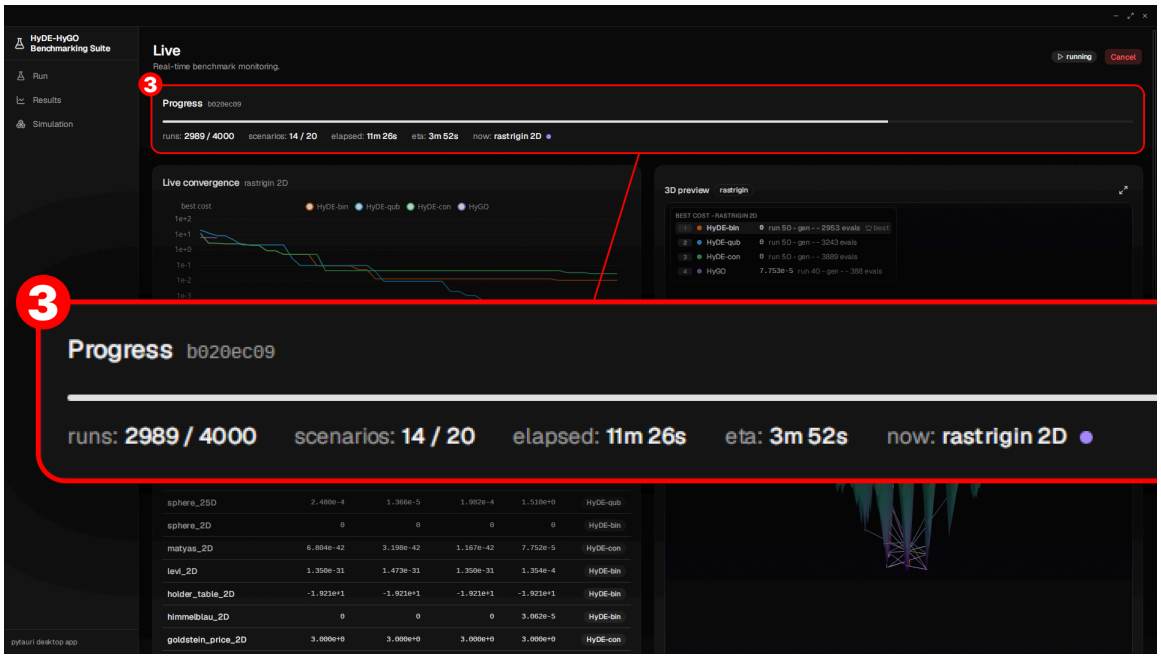


MONITOR A RUNNING BENCHMARK

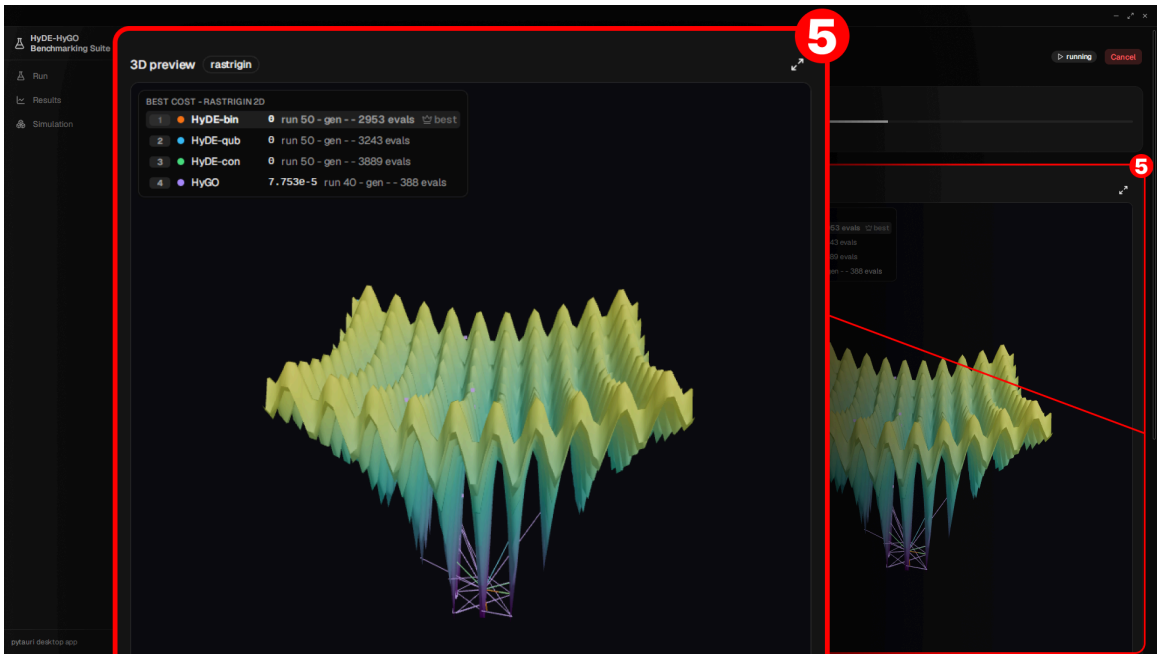
After clicking the **Start Benchmark** button, users can use the Live screen to monitor the experiment they just configured, where they can:

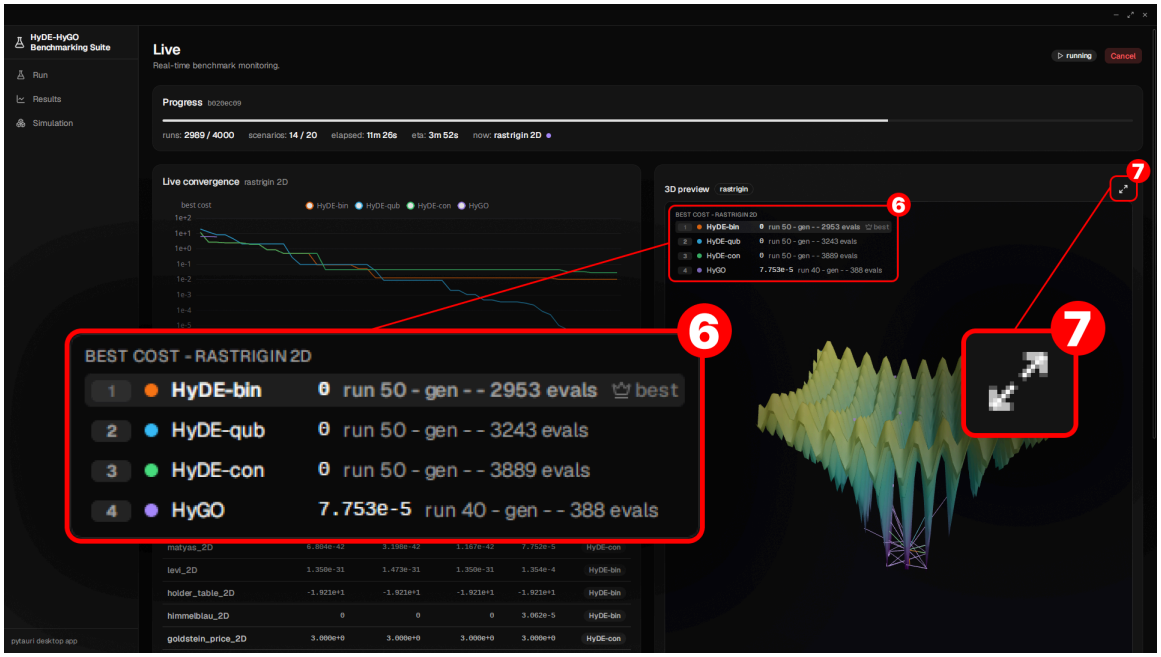
1. View the **Running Status** to ensure that the benchmark process is running;
2. Click **Cancel** to end the benchmarking process prior to its completion;
3. View **Progress** to check the number of runs, number of completed scenarios, time elapsed, estimated time of arrival, and current scenario;
4. View **Live Convergence** to analyze the best cost of each algorithm on the current scenario; the lower the best cost, the better;
5. View **3D Preview** to analyze the surface of the current scenario;
6. View **Best Cost List** to compare the best cost of each algorithm, number of runs, number of generations, and evaluations, with the best one being labeled as **Best**;
7. Click **Expand** to enlarge the 3D preview; and
8. View **Completed Scenarios** with the results of all completed scenarios for each algorithm, with the best one being labeled as **Best**.

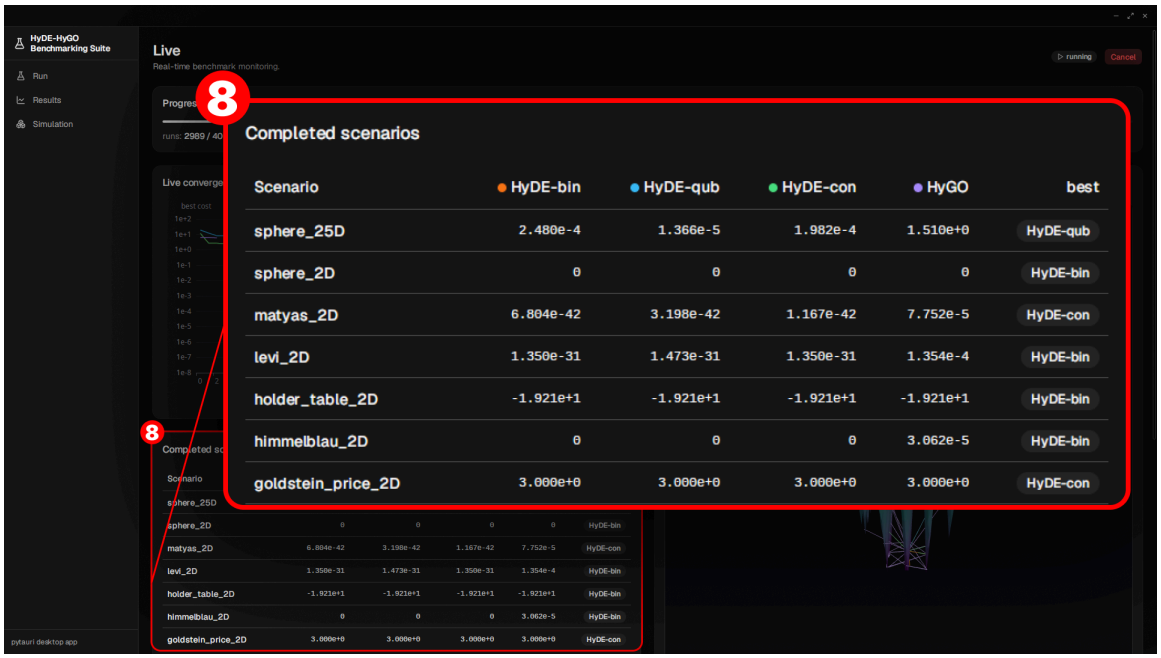
HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Live
Real-time benchmark monitoring.
running
Cancel
Progress b020ec09
runs: 2989 / 4000 scenarios: 14 / 20 elapsed: 11m 26s eta: 3m 52s now: rastrigin 2D
Live convergence rastrigin 2D
3D preview rastrigin
1
2
running
Cancel


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Live
Real-time benchmark monitoring.
3
Progress b020ec09
runs: 2989 / 4000 scenarios: 14 / 20 elapsed: 11m 26s eta: 3m 52s now: rastrigin 2D
Live convergence rastrigin 2D
3D preview rastrigin

HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Live
Real-time benchmark monitoring.
running
Cancel
Progress
runs: 2989 / 4000
scenarios: 14 / 20
elapsed: 11m 26s
eta: 3m 52s
now: rastrigin 2D
Live convergence rastrigin 2D
best cost
HyDE-bin
HyDE-qub
HyDE-con
HyGO
3D preview rastrigin
Completed scenarios
Scenario
sphere_25D
sphere_2D
matyas_2D
levi_2D
holder_table_2D
himmelblau_2D
goldstein_price_2D
4


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
3D preview rastrigin
BEST COST - RASTRIGIN 2D
1 HyDE-bin run 50 - gen - - 2953 evals best
2 HyDE-qub run 50 - gen - - 3243 evals
3 HyDE-con run 50 - gen - - 3889 evals
4 HyGO 7.753e-5 run 40 - gen - - 388 evals
5

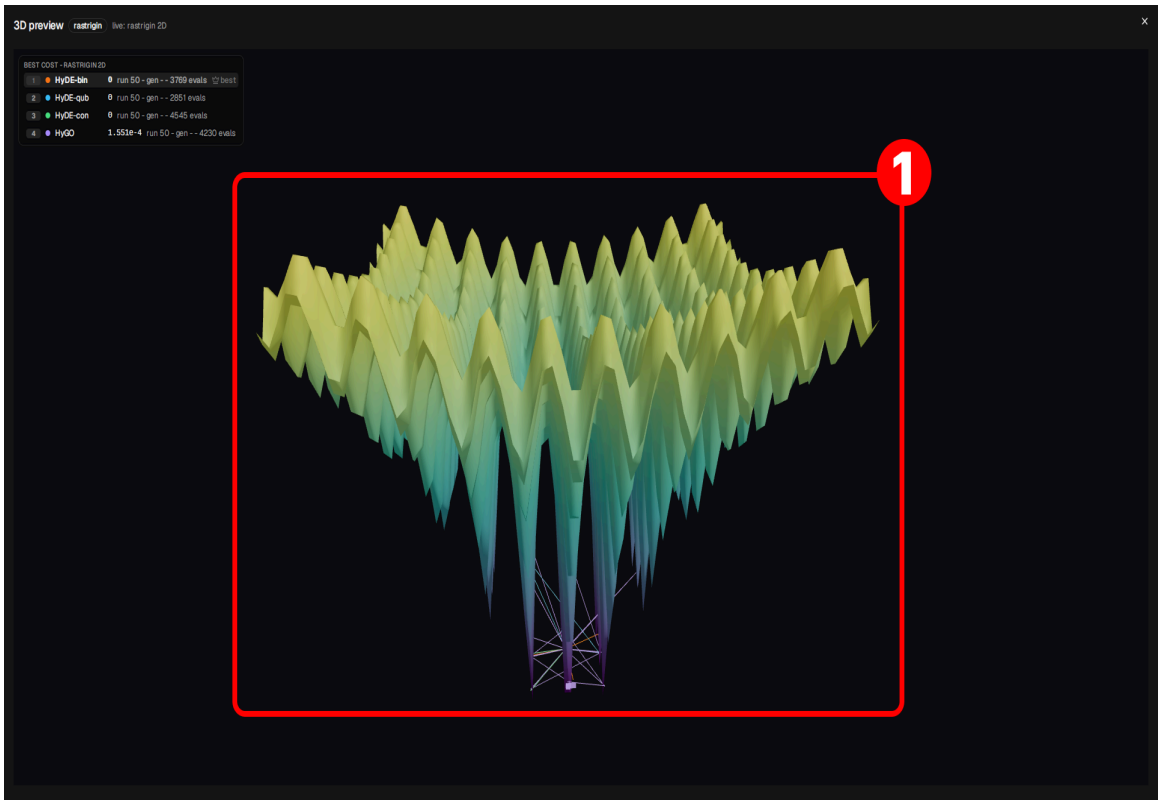

HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Live
Real-time benchmark monitoring.
running
Cancel
Progress
runs: 2989 / 4000
scenarios: 14 / 20
elapsed: 11m 26s
eta: 3m 52s
now: rastrigin 2D
Live convergence rastrigin 2D
3D preview rastrigin
6
7
BEST COST - RASTRIGIN 2D
1 HyDE-bin 0 run 50 - gen - - 2953 evals best
2 HyDE-qub 0 run 50 - gen - - 3243 evals
3 HyDE-con 0 run 50 - gen - - 3889 evals
4 HyGO 7.753e-5 run 40 - gen - - 388 evals


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Live
Real-time benchmark monitoring.
8
Completed scenarios
Scenario
HyDE-bin
HyDE-qub
HyDE-con
HyGO
best
sphere_25D 2.480e-4 1.366e-5 1.982e-4 1.510e+0 HyDE-qub
sphere_2D 0 0 0 0 HyDE-bin
matyas_2D 6.804e-42 3.198e-42 1.167e-42 7.752e-5 HyDE-con
levi_2D 1.350e-31 1.473e-31 1.350e-31 1.354e-4 HyDE-bin
holder_table_2D -1.921e+1 -1.921e+1 -1.921e+1 -1.921e+1 HyDE-bin
himmelblau_2D 0 0 0 3.062e-5 HyDE-bin
goldstein_price_2D 3.000e+0 3.000e+0 3.000e+0 3.000e+0 HyDE-con

ENLARGE THE 3D PREVIEW

When the **Expand** button is clicked, the **Live 3D preview** is magnified and becomes the point of focus, where users can:

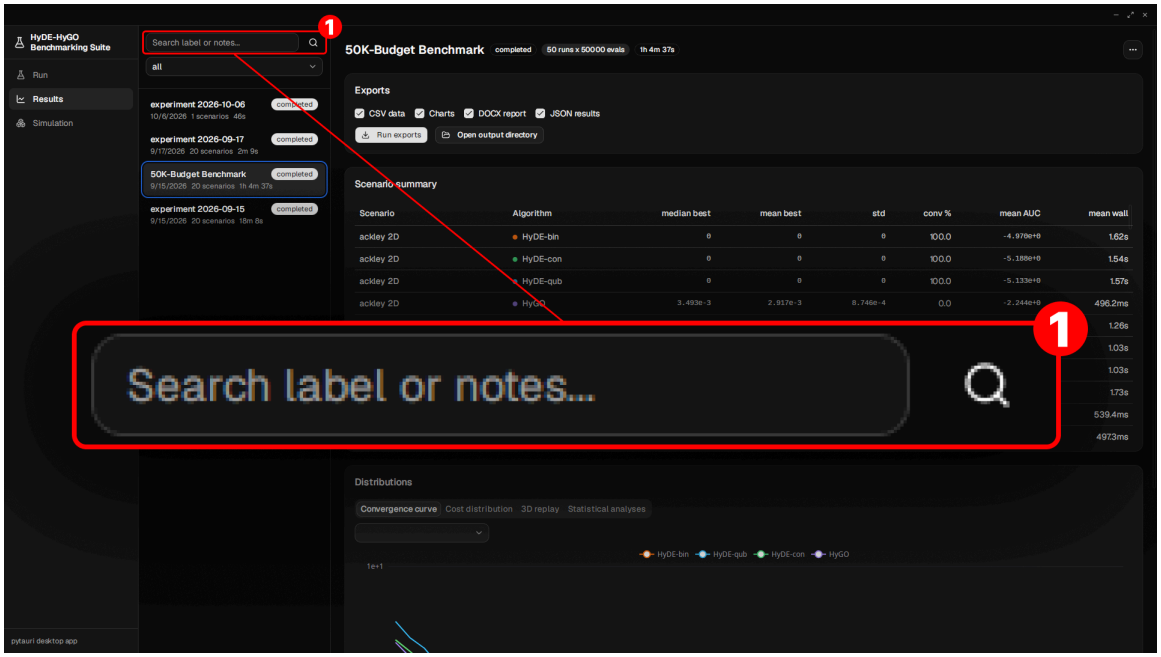
1. **Manipulate** the orientation and position of the magnified 3D live preview of the ongoing experiment;
2. Look at the **Scenario Tag** to verify which test function is being viewed;
3. Go to the **Best Cost List** to monitor the progression of each algorithm in the scenario; and
4. Click **Close** to go back to the **Live** page.

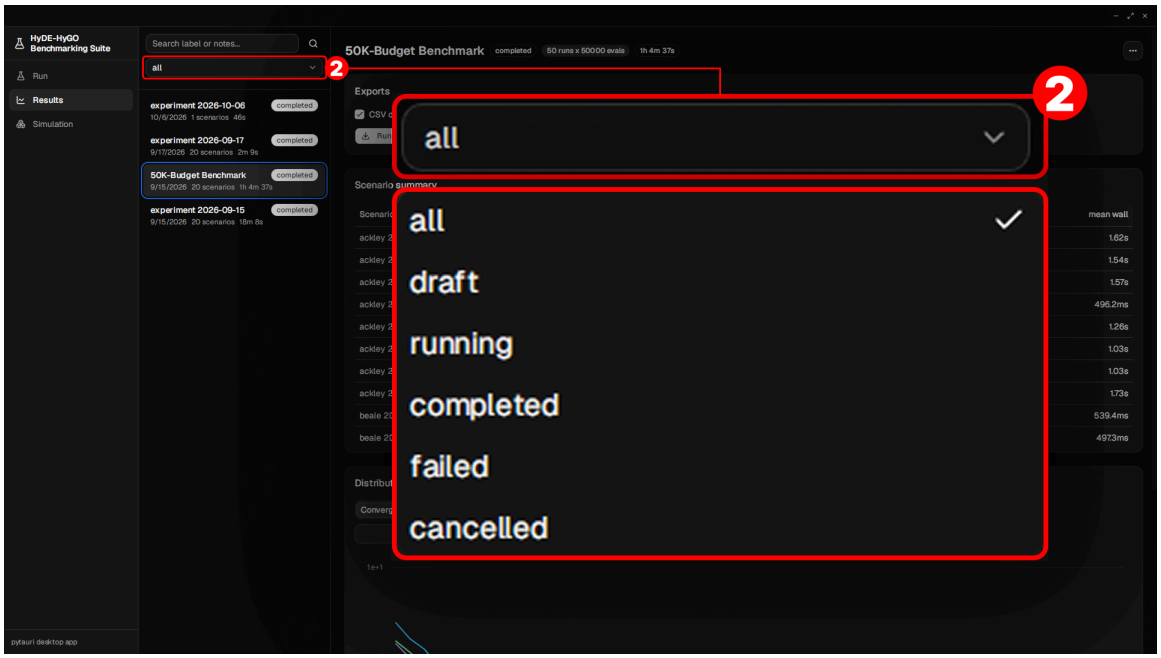


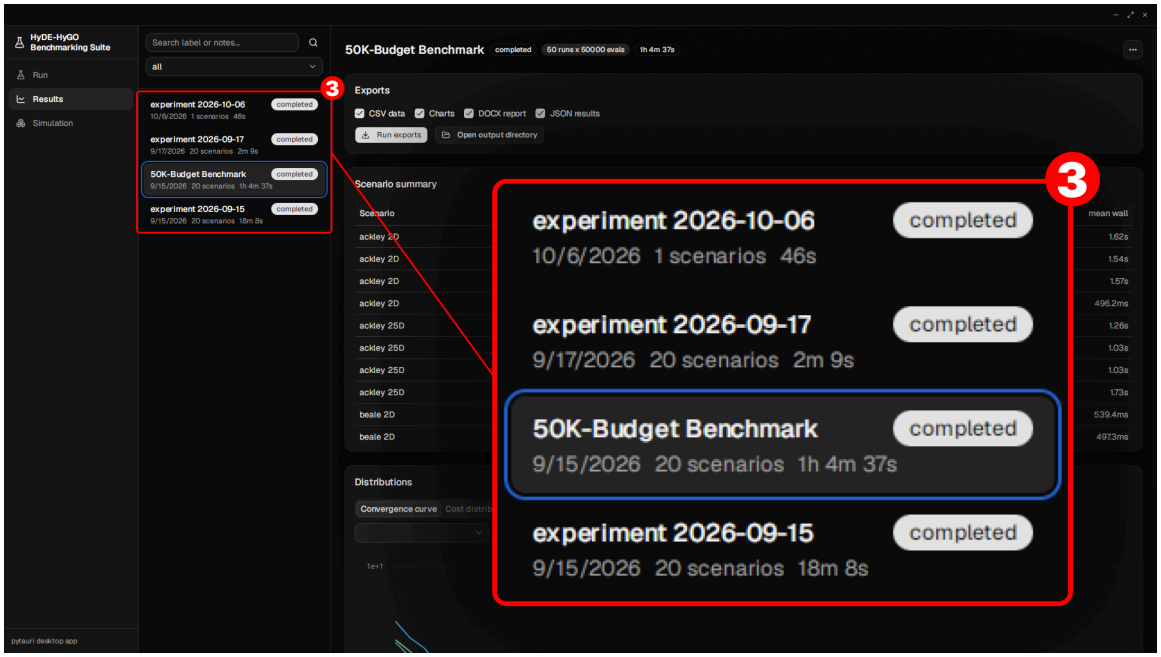
REVIEW SAVED RESULTS

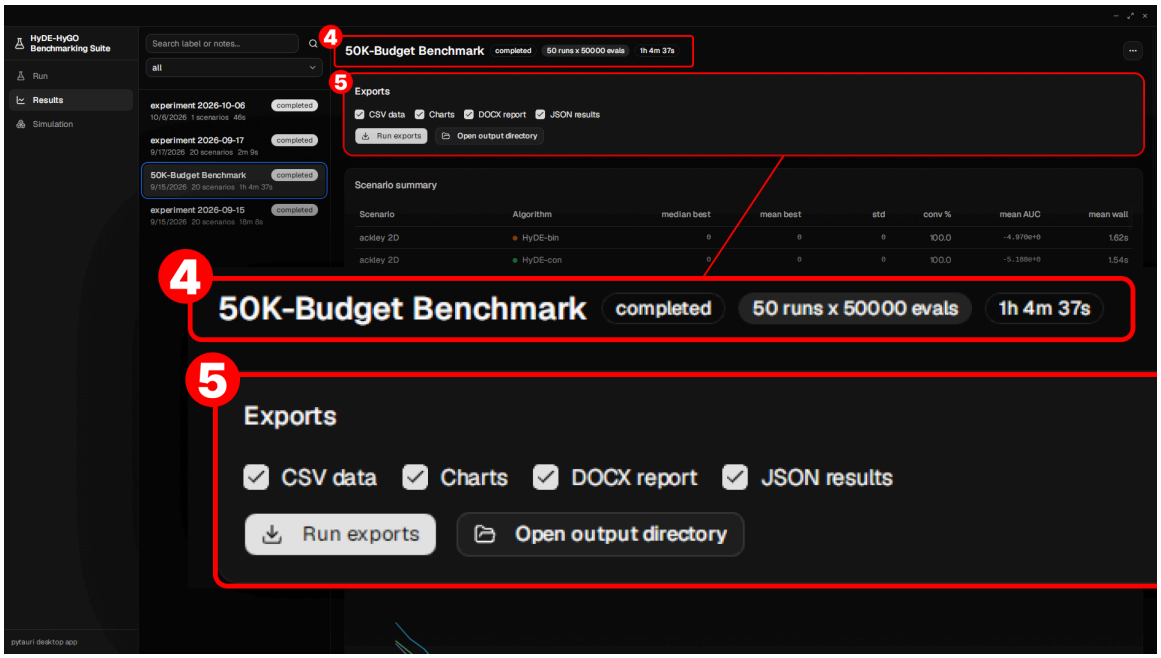
When users move to the **Results** page, users will see the previously conducted experiments and their results, where they can:

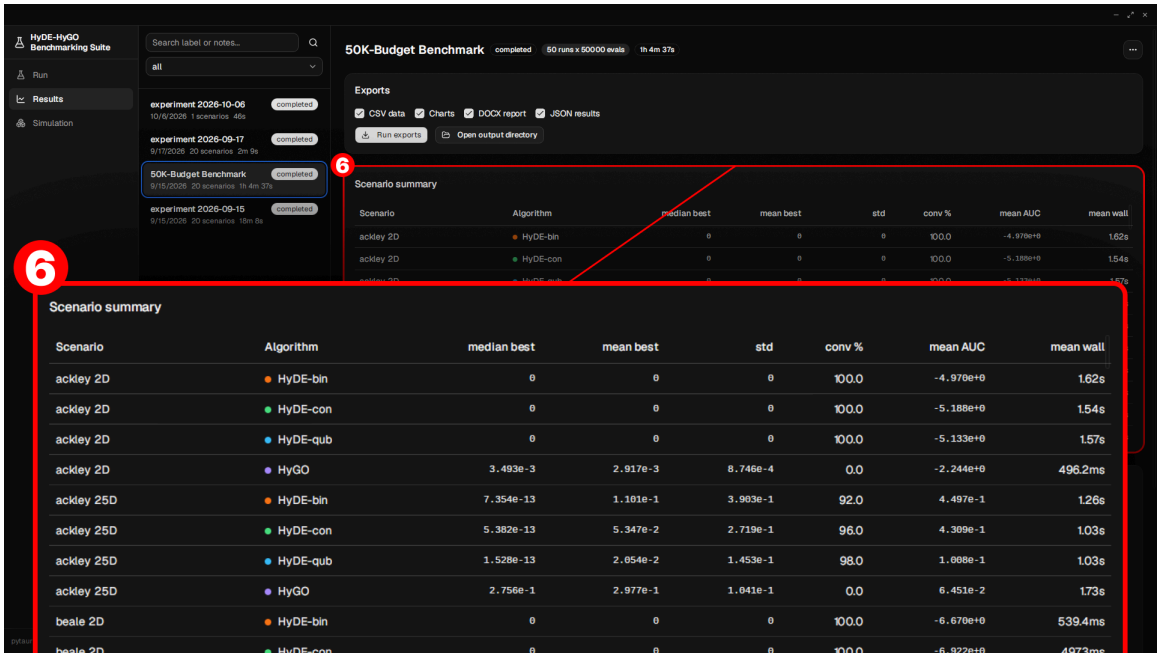
1. Type in the **Search Box** to search for an experiment using its label or notes;
2. Select from the **Filter Dropdown** to limit the list of saved experiments (currently set to all);
3. Select an experiment from the **Experiment List** to view its results;
4. See the **Experiment Header** that displays the experiment title, status, settings (number of runs and evaluations per run), and duration;
5. Use the **Exports** options to choose which file types (**CSV Data**, **Charts**, **DOCX Report**, and **JSON Results**) to export, click **Run Exports** to export them, or click **Open Output Directory** to open the output directory;
6. See the **Scenario Summary** that compares the algorithms used for each scenario in terms of best cost, standard deviation, convergence rate, and average time;


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Search label or notes...
all
experiment 2026-10-06
experiment 2026-09-17
50K-Budget Benchmark
experiment 2026-09-15
50K-Budget Benchmark completed 50 runs x 50000 evals 1h 4m 37s
Exports
CSV data
Charts
DOCX report
JSON results
Run exports
Open output directory
Scenario summary
Scenario
Algorithm
median best
mean best
std
conv %
mean AUC
mean wall
ackley 2D
HyDE-bin
HyDE-con
HyDE-qub
HyGO
Distributions
Convergence curve
Cost distribution
3D replay
Statistical analyses
1
Search label or notes...


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Search label or notes...
all
experiment 2026-10-06
experiment 2026-09-17
50K-Budget Benchmark
experiment 2026-09-15
50K-Budget Benchmark completed 50 runs x 50000 evals 1h 4m 37s
Exports
Scenario summary
2
all
all
draft
running
completed
failed
cancelled


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Search label or notes...
all
experiment 2026-10-06 completed
10/6/2026 1 scenarios 46s
experiment 2026-09-17 completed
9/17/2026 20 scenarios 2m 9s
50K-Budget Benchmark completed
9/15/2026 20 scenarios 1h 4m 37s
experiment 2026-09-15 completed
9/15/2026 20 scenarios 18m 8s
50K-Budget Benchmark completed 50 runs x 50000 evals 1h 4m 37s
Exports
CSV data Charts DOCX report JSON results
Run exports Open output directory
Scenario summary
Distributions
Convergence curve
3

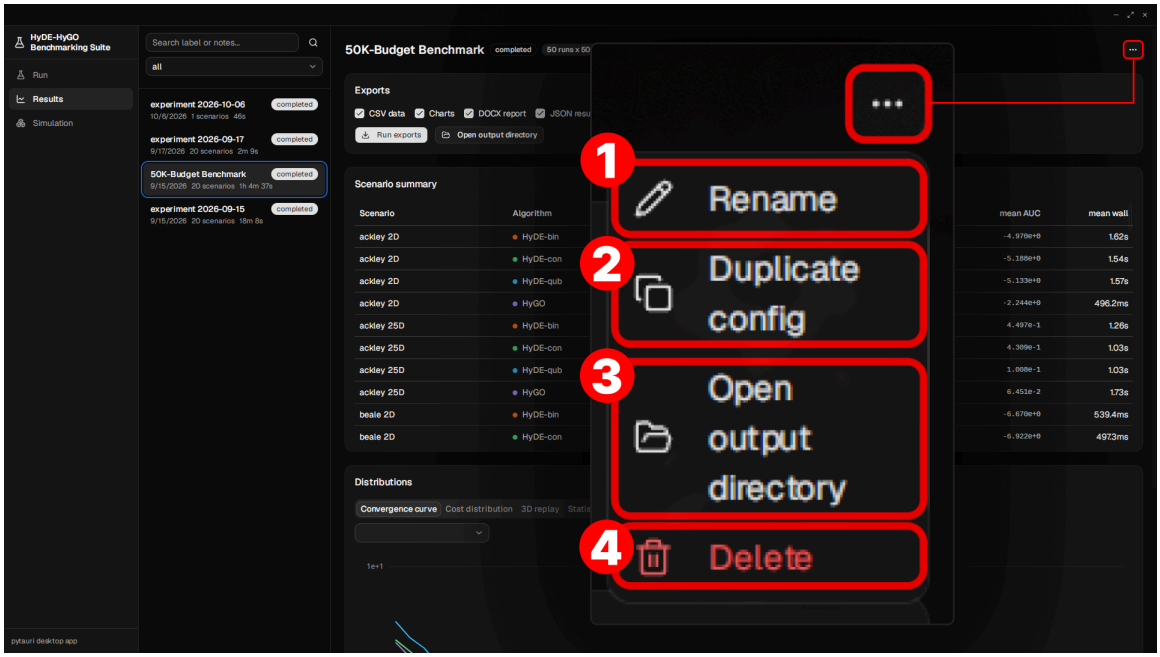

HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Search label or notes...
all
experiment 2026-10-06 completed
experiment 2026-09-17 completed
50K-Budget Benchmark completed
experiment 2026-09-15 completed
4
50K-Budget Benchmark completed 50 runs x 50000 evals 1h 4m 37s
5
Exports
CSV data Charts DOCX report JSON results
Run exports Open output directory
Scenario summary
Scenario Algorithm median best mean best std conv % mean AUC mean wall
ackley 2D HyDE-bin 100.0 1.62s
ackley 2D HyDE-con 100.0 1.54s
4
50K-Budget Benchmark completed 50 runs x 50000 evals 1h 4m 37s
5
Exports
CSV data Charts DOCX report JSON results
Run exports Open output directory



MANAGE AN EXPERIMENT

In the **Results** page, when users click the (**...**) **Menu** button, they can:

1. Choose **Rename** to rename the experiment;
2. Choose **Duplicate Config** to duplicate the configuration of the experiment;
3. Choose **Open Output Directory** to access the directory that contains the experiment files; and
4. Choose **Delete** to delete the experiment.


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Search label or notes...
all
experiment 2026-10-06
10/6/2026 1 scenarios 46s
completed
experiment 2026-09-17
9/17/2026 20 scenarios 2m 9s
completed
50K-Budget Benchmark
9/15/2026 20 scenarios 1h 4m 37s
completed
experiment 2026-09-15
9/15/2026 20 scenarios 18m 8s
completed
pyfauri desktop app
50K-Budget Benchmark
completed
50 runs x 50
Exports
CSV data
Charts
DOCX report
Run exports
Open output directory
Scenario summary
Scenario
Algorithm
mean AUC
mean wall
ackley 2D
HyDE-bin
-4.970e+0
1.62s
ackley 2D
HyDE-con
-5.188e+0
1.54s
ackley 2D
HyDE-qub
-5.133e+0
1.57s
ackley 2D
HyGO
-2.244e+0
496.2ms
ackley 25D
HyDE-bin
4.497e-1
1.26s
ackley 25D
HyDE-con
4.309e-1
1.03s
ackley 25D
HyDE-qub
1.008e-1
1.03s
ackley 25D
HyGO
6.451e-2
1.73s
beale 2D
HyDE-bin
-6.670e+0
539.4ms
beale 2D
HyDE-con
-6.922e+0
497.3ms
Distributions
Convergence curve
Cost distribution
3D replay
1e+1
1
Rename
2
Duplicate config
3
Open output directory
4
Delete

NAVIGATE IN DISTRIBUTIONS SECTION TABS

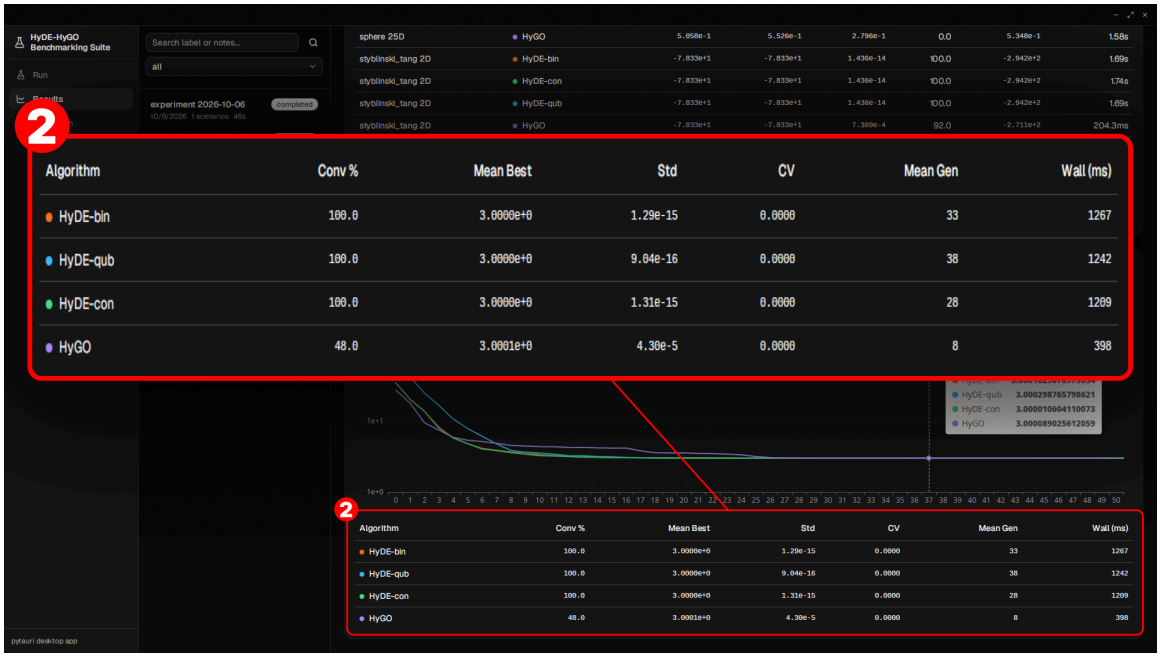
In the **Results** page, when users navigate to the **Distributions panel**, users can choose from the following tabs:

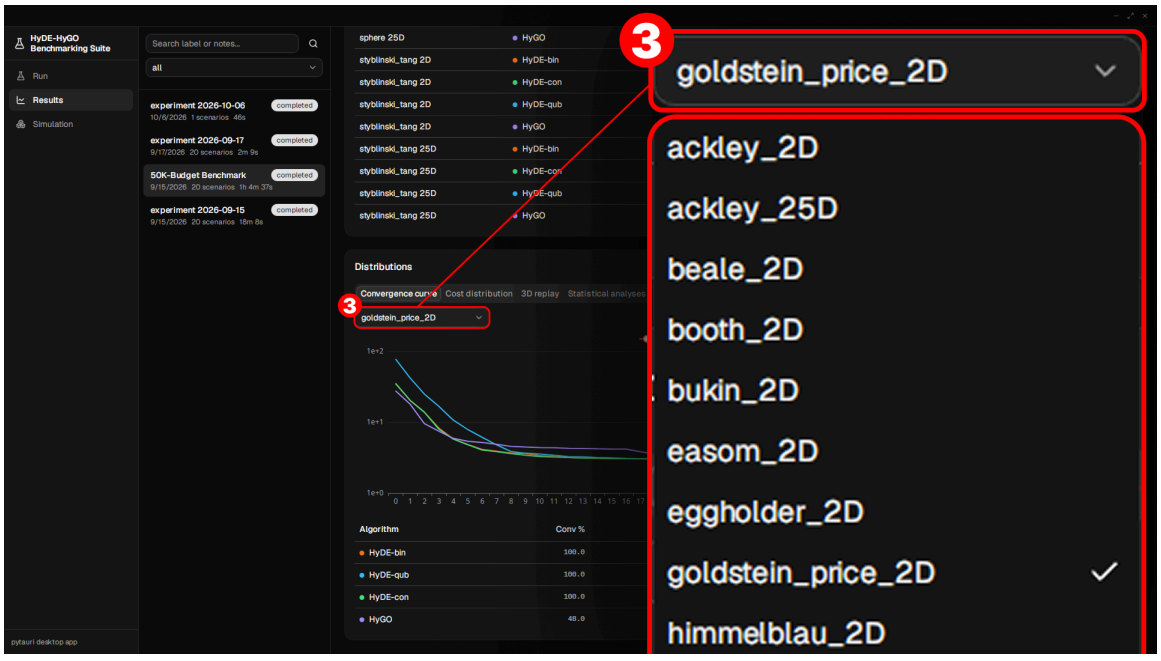
1. Click **Convergence Curve** to see how the best cost of each algorithm decreases across generations;
2. Click **Cost Distribution** to see how the final results of each algorithm are spread;
3. Click **3D Replay** to replay a recorded search step by step; and
4. Click **Statistical Analyses** to see the tests that show whether the differences between the algorithms are significant.

VIEW THE CONVERGENCE CURVE

In the **Results** page, once users click the **Convergence Curve** tab, they can see how the best cost of each algorithm changes across generations, where they can:

1. Hover over the chart to see the **Tooltip** showing the actual best cost of each algorithm at the selected generation;
2. Read the **Statistics Table** below the chart to compare each algorithm's Conv %, Mean Best, Std, CV, Mean Gen, and Wall (ms); and
3. Select the required scenario using the **Scenario Dropdown** to view its convergence curve.


HyDE-HyGO Benchmarking Suite
Search label or notes...
all
experiment 2026-10-06
completed
2
Algorithm
Conv %
Mean Best
Std
CV
Mean Gen
Wall (ms)
HyDE-bin
100.0
3.0000e+0
1.29e-15
0.0000
33
1267
HyDE-qub
100.0
3.0000e+0
9.04e-16
0.0000
38
1242
HyDE-con
100.0
3.0000e+0
1.31e-15
0.0000
28
1209
HyGO
48.0
3.0001e+0
4.30e-5
0.0000
8
398
pytauri desktop app


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
experiment 2026-10-06
experiment 2026-09-17
50K-Budget Benchmark
experiment 2026-09-15
completed
Distributions
Convergence curve
Cost distribution
3D replay
Statistical analyses
3
goldstein_price_2D
ackley_2D
ackley_25D
beale_2D
booth_2D
bukin_2D
easom_2D
eggholder_2D
goldstein_price_2D
himmelblau_2D
pytauri desktop app

VIEW THE COST DISTRIBUTION

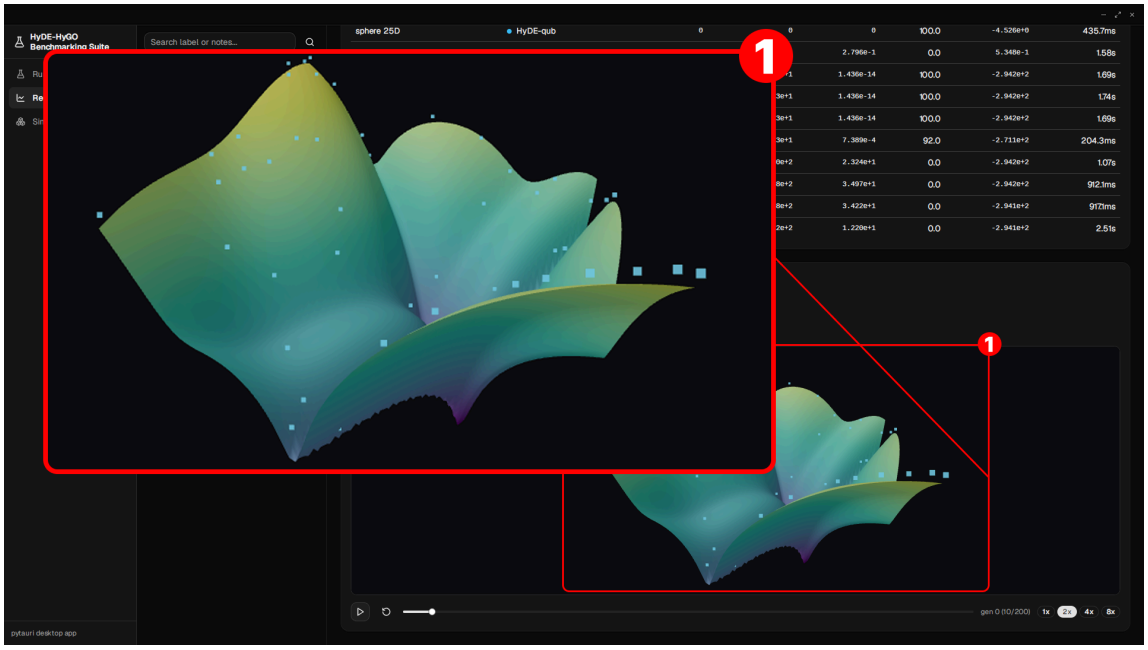
In the **Results** page, once users click the **Cost Distribution** tab, users can see how the final results of each algorithm are spread, where they can:

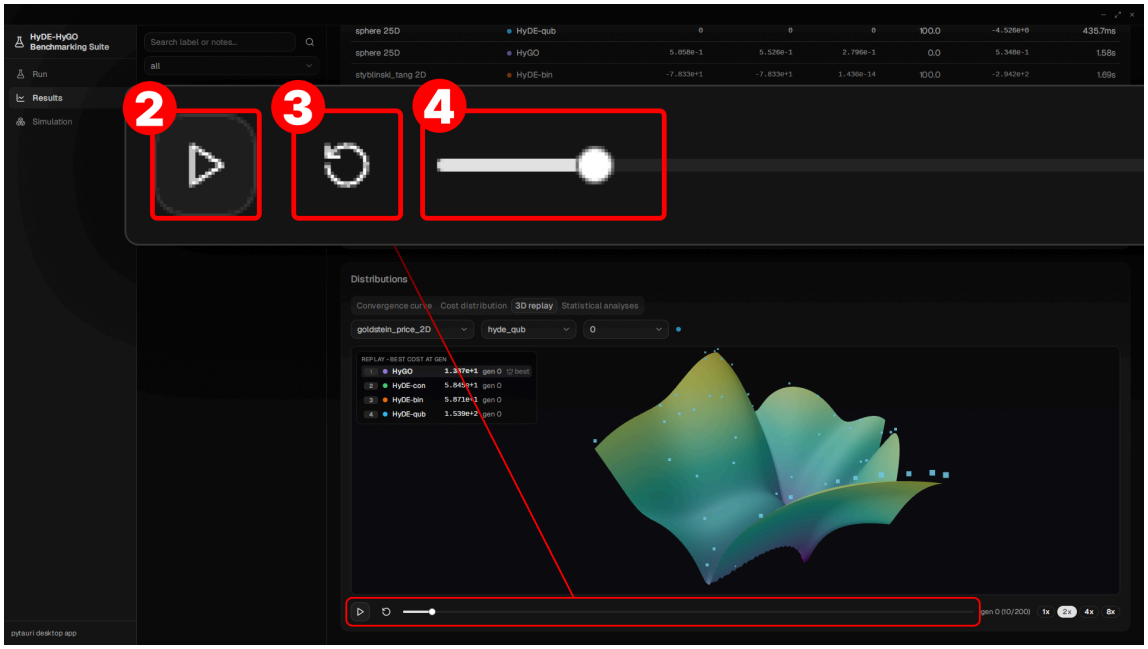
1. View the **Cost Distribution Chart**, which shows a box plot of the final cost for each algorithm; a lower, thinner box means better results.

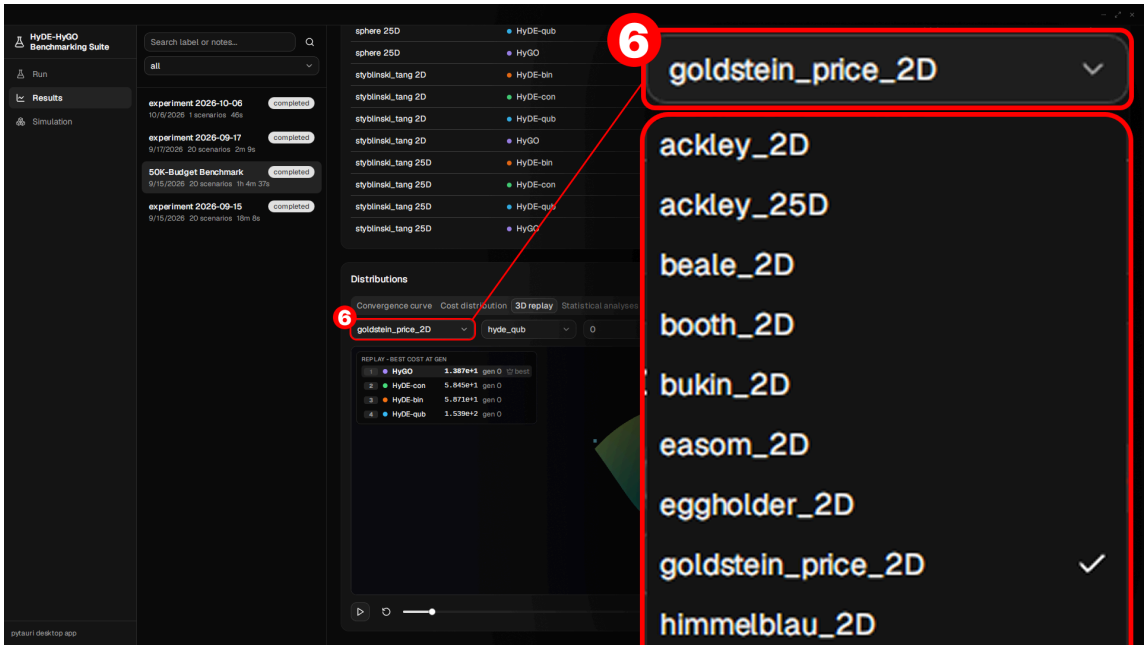
REPLAY THE SEARCH IN 3D

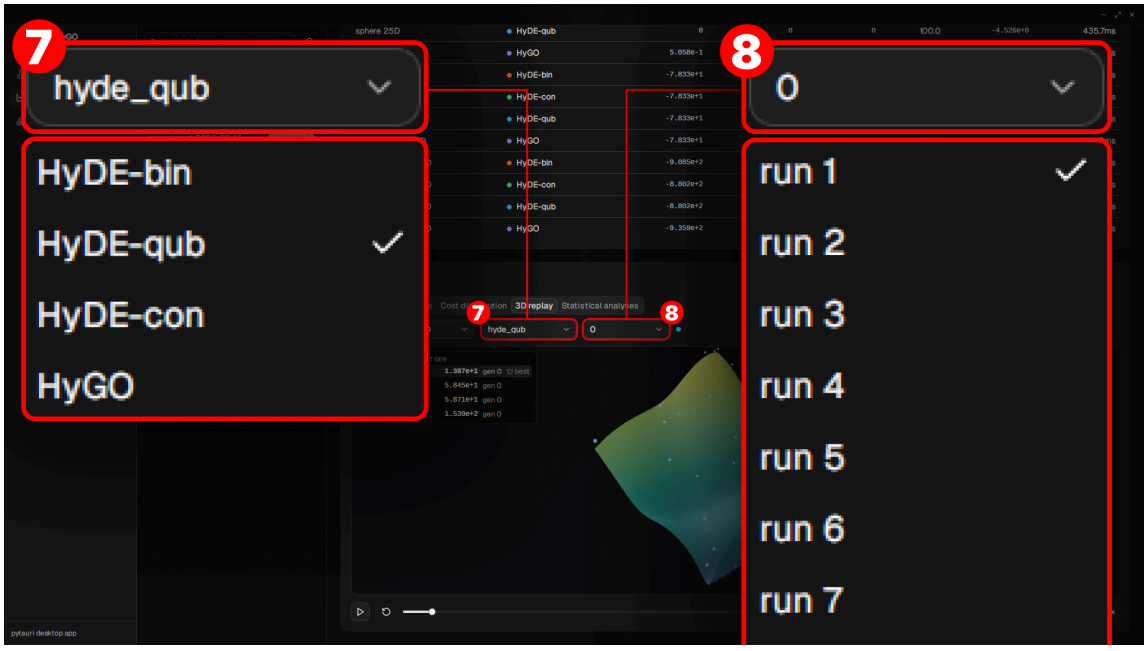
In the **Results** page, when users click the **3D Replay** tab, they can replay a recorded search step by step, where they can:

1. See the **3D Surface** with the dots, which represent the candidate solutions generated by the algorithm in the current generation, and the **Replay Best Cost List** ranking the algorithms from best to worst for that generation;
2. Click **Play** to start the replay;
3. Click **Reset** to return the replay to its initial state;
4. Drag the **Progress Slider** to move to a specific generation of the replay;
5. Click **Speed** (1x, 2x, 4x, or 8x) to change how fast the replay runs;
6. Click the **Scenario Dropdown** to select the scenario to replay;
7. Click the **Algorithm Dropdown** to select the algorithm to replay; and
8. Click the **Run Dropdown** to select the run of the chosen algorithm to replay.




HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
2
3
4
Distributions
Convergence curve
Cost distribution
3D replay
Statistical analyses
goldstein_price_2D
hyde_qub
HyGO
HyDE-con
HyDE-bin
HyDE-qub
1x
2x
4x
8x
5

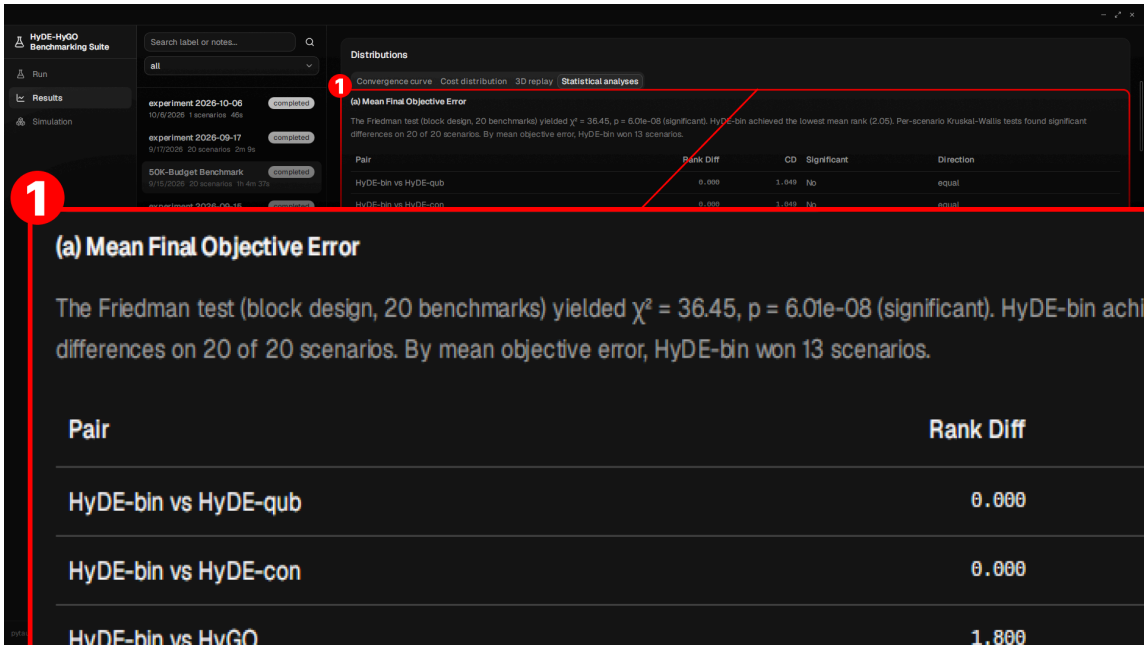

6
goldstein_price_2D
ackley_2D
ackley_25D
beale_2D
booth_2D
bukin_2D
easom_2D
eggholder_2D
goldstein_price_2D
himmelblau_2D


7
hyde_qub
HyDE-bin
HyDE-qub
HyDE-con
HyGO
8
0
run 1
run 2
run 3
run 4
run 5
run 6
run 7

READ THE STATISTICAL ANALYSIS

In the **Results** page, when users click the **Statistical Analyses** tab, users can see the tests that indicate the significance of the differences between the algorithms, as well as charts of how consistently each algorithm performs, where they can:

1. Analyze the **Statistical Tables** to learn which algorithm wins in each pair (Rank Diff, CD, Significant, Direction) and in each scenario (H-stat, p-value, Sig., Best), where a pair is considered significant if its Rank Diff exceeds the CD; and
2. View the **Result Consistency** chart (coefficient of variation per benchmark at 25D) and the **Normalized Degradation** chart (from 2D to 25D; a lower value means the algorithm scales better).

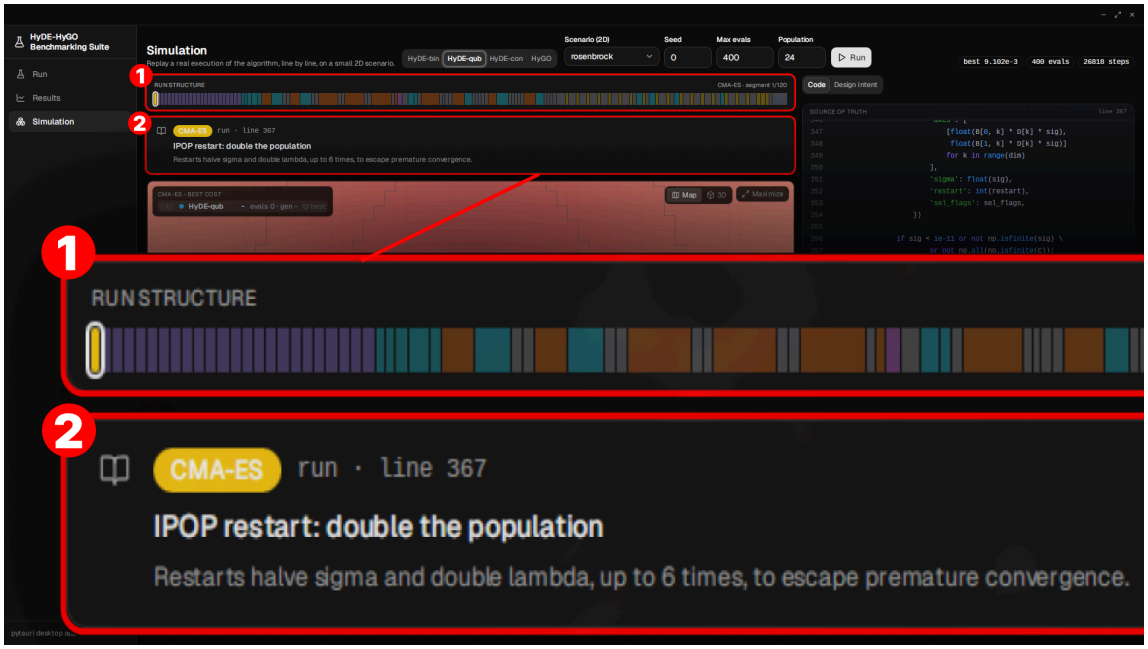


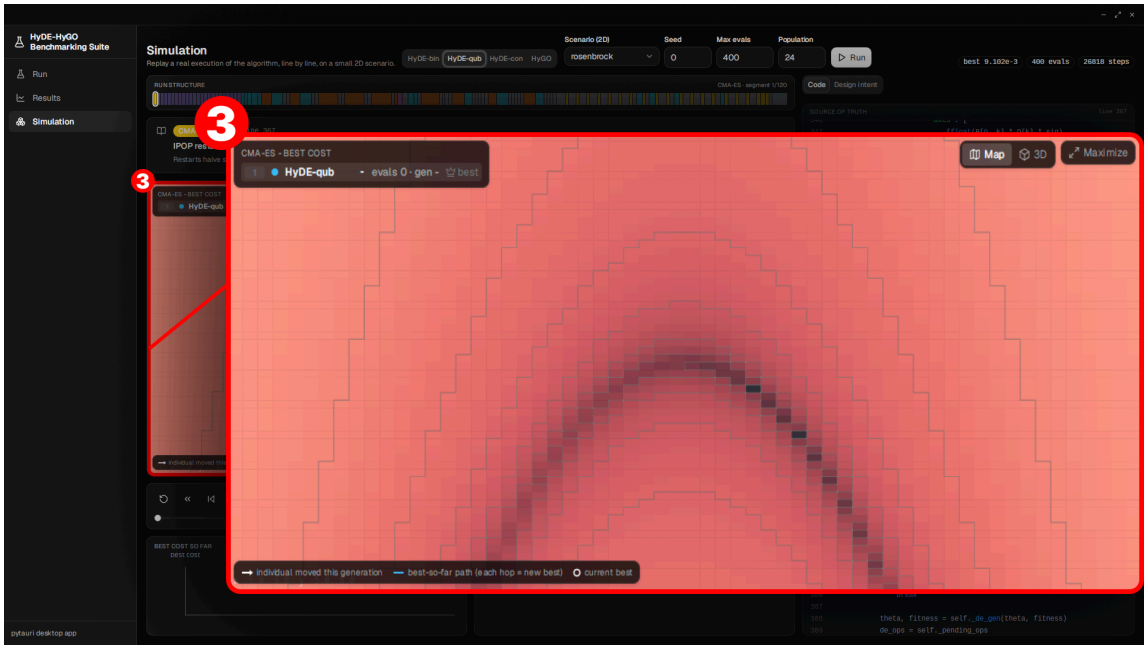
2
Result consistency at 25D - coefficient of variation per benchmark
HyDE-bin
HyDE-qub
HyDE-con
HyGO
8
7
6
5
4
3
2
1
0
ackley
sphere
rastrigin
rosenbrock
2
Normalized degradation 2D to 25D - (mean 25D - mean 2D) / max(mean 2D, 1e-12); lower means it scales better

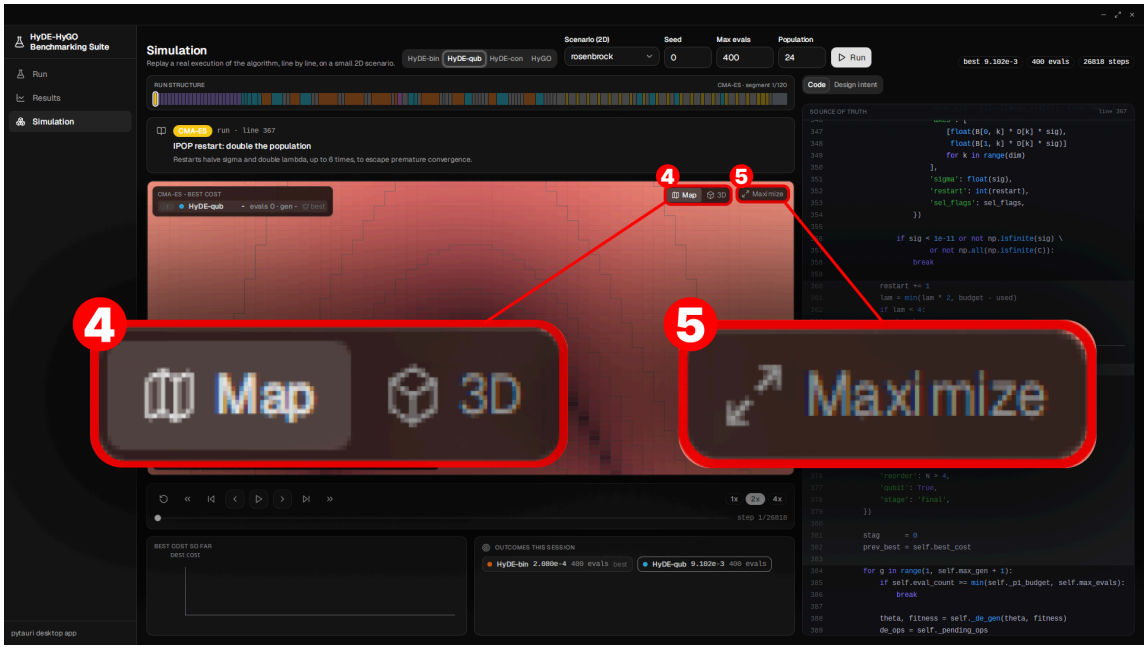
ACCESS THE SIMULATION SCREEN

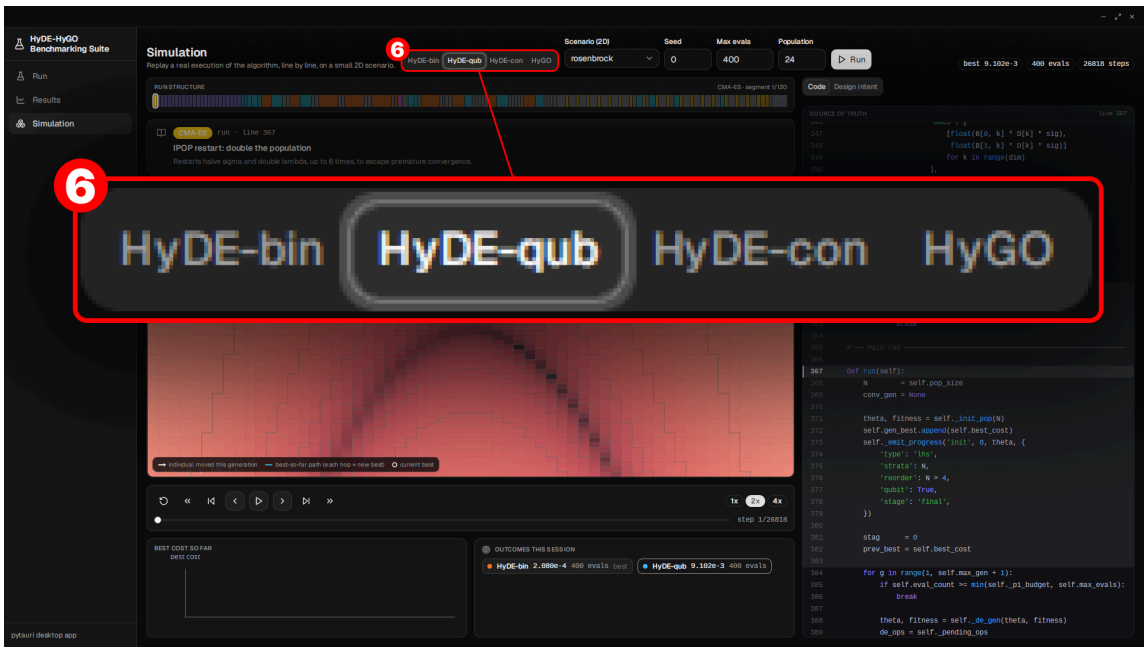
In the **Simulation page**, when users run an algorithm, they can follow its execution line by line on a small 2D scenario, where they can:

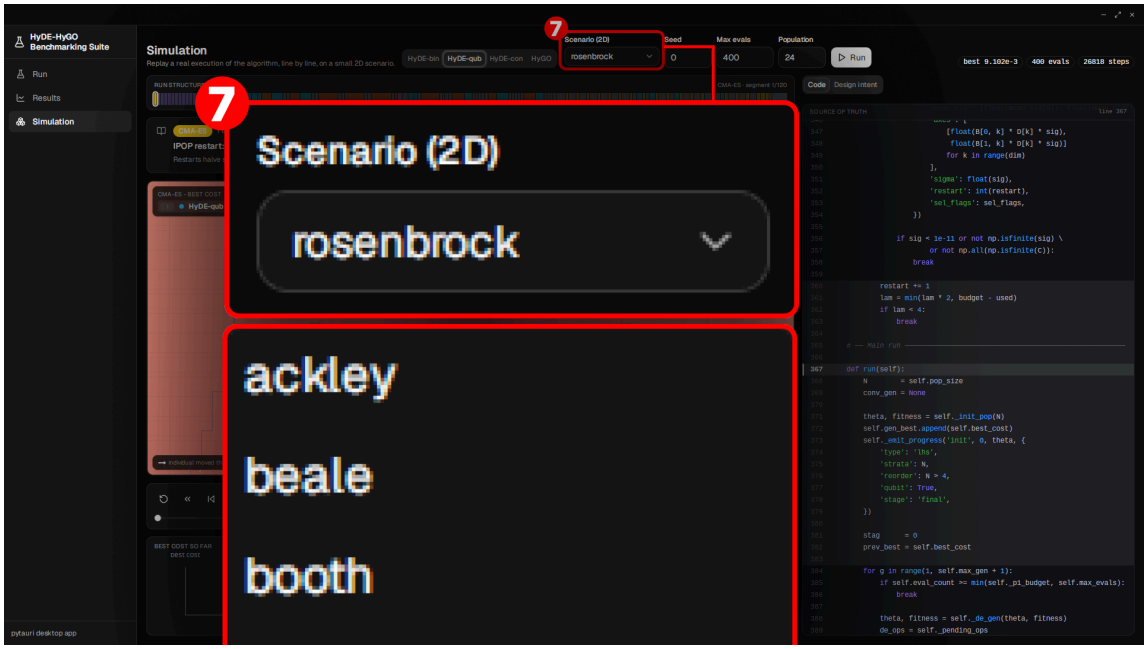
1. See the **Run Structure**, a color-coded timeline of the whole run, indicating the current segment and stage;
2. See the **Stage Card**, which shows the current stage of the algorithm, the name of the function and the line of code being run, and an explanation of it;
3. See the **Simulation Display** with the search process in the selected scenario, together with the Best Cost Label showing the current stage, best cost, evaluation count, and generation;
4. Click the **Map** or **3D** buttons to switch between the top-down map and the 3D surface;
5. Click the **Maximize** button to enlarge the simulation display;
6. Click the **Algorithm Tabs** (HyDE-bin, HyDE-qub, HyDE-con, or HyGO) to select the algorithm you want to simulate;
7. Click the **Scenario** dropdown to select a 2D test function for the simulation;
8. Type the **Seed** to set the initial value of the random seed;
9. Type the **Max Evals** to set the evaluation limit;
10. Type the **Population** to set the number of candidates;
11. Click **Run** to execute the algorithm with the specified parameters and get the **Run Summary** with the best cost, evaluations, and total steps;
12. Control the simulation using the **Playback Controls** (Play, Pause, Step Forward/Backward, and Restart) and choose one of three speeds (1x, 2x, or 4x);
13. See the **Source of Truth**, the actual code of the algorithm with the currently running line highlighted;
14. See **Best Cost So Far**, the chart showing the best cost achieved in the course of the simulation; and
15. View **Outcomes This Session** to compare the best costs and evaluations of the simulated algorithms in the current session.

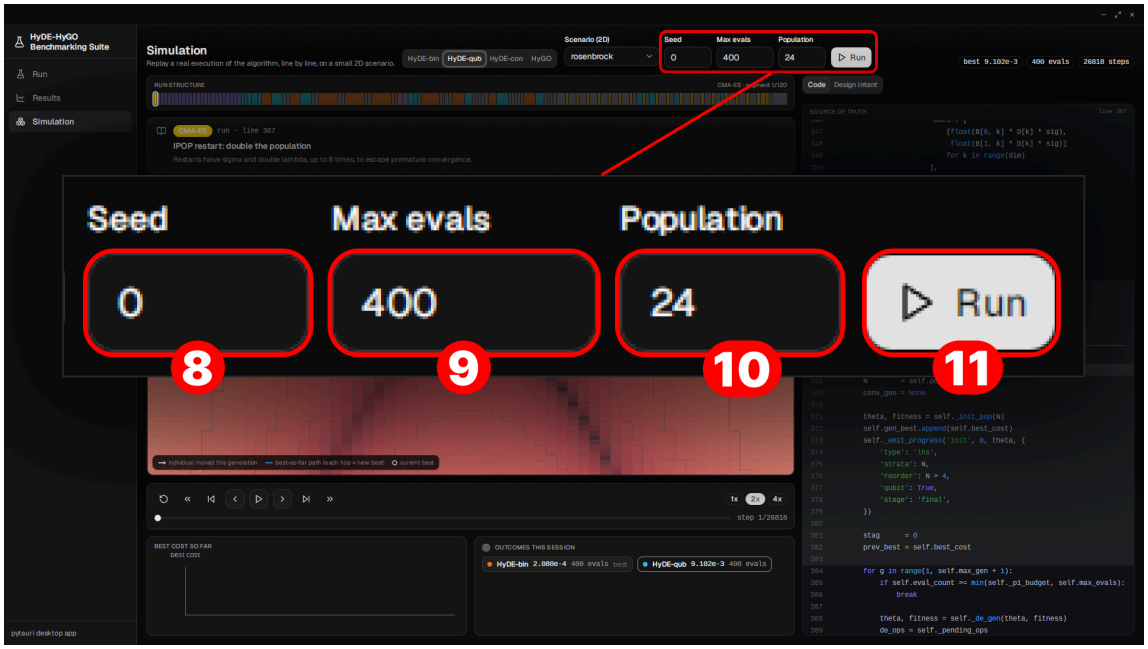

HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Simulation
Replay a real execution of the algorithm, line by line, on a small 2D scenario.
HyDE-bin
HyDE-qub
HyDE-con
HyGO
Scenario (2D)
rosenbrock
Seed
0
Max evals
400
Population
24
Run
best 9.102e-3
400 evals
26818 steps
RUN STRUCTURE
Code
Design intent
CMA-ES
run · line 367
IPOP restart: double the population
Restarts halve sigma and double lambda, up to 6 times, to escape premature convergence.
CMA-ES - BEST COST
HyDE-qub
evals 0 · gen -
best
Map
3D
Maximize
1
2
RUN STRUCTURE
CMA-ES
run · line 367
IPOP restart: double the population
Restarts halve sigma and double lambda, up to 6 times, to escape premature convergence.


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Simulation
Replay a real execution of the algorithm, line by line, on a small 2D scenario.
HyDE-bin
HyDE-qub
HyDE-con
HyGO
Scenario (2D)
rosenbrock
Seed
0
Max evals
400
Population
24
Run
best 9.102e-3
400 evals
26818 steps
RUN STRUCTURE
Code
Design intent
3
CMA-ES - BEST COST
HyDE-qub
evals 0 · gen -
best
Map
3D
Maximize
individual moved this generation
best-so-far path (each hop = new best)
current best
pytauri desktop app

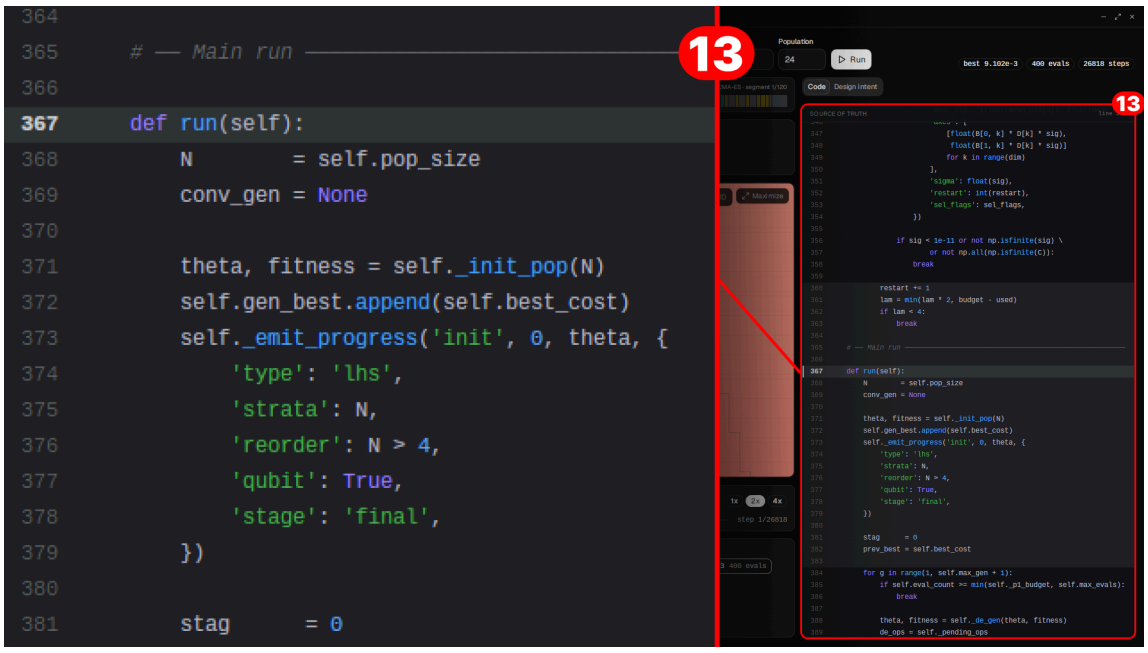

HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Simulation
Replay a real execution of the algorithm, line by line, on a small 2D scenario.
HyDE-bin
HyDE-qub
HyDE-con
HyGO
Scenario (2D)
rosenbrock
Seed
0
Max evals
400
Population
24
Run
best 9.102e-3
400 evals
26818 steps
RUN STRUCTURE
Code
Design intent
IPOP restart: double the population
Restarts halve sigma and double lambda, up to 6 times, to escape premature convergence.
4
5
Map
3D
Maximize
pytauri desktop app


HyDE-HyGO Benchmarking Suite
Run
Results
Simulation
Simulation
Replay a real execution of the algorithm, line by line, on a small 2D scenario.
6
HyDE-bin
HyDE-qub
HyDE-con
HyGO
Scenario (2D)
rosenbrock
Seed
0
Max evals
400
Population
24
Run
best 9.102e-3
400 evals
26818 steps
RUN STRUCTURE
Code
Design intent
IPOP restart: double the population
Restarts halve sigma and double lambda, up to 6 times, to escape premature convergence.
HyDE-bin
HyDE-qub
HyDE-con
HyGO
pytauri desktop app

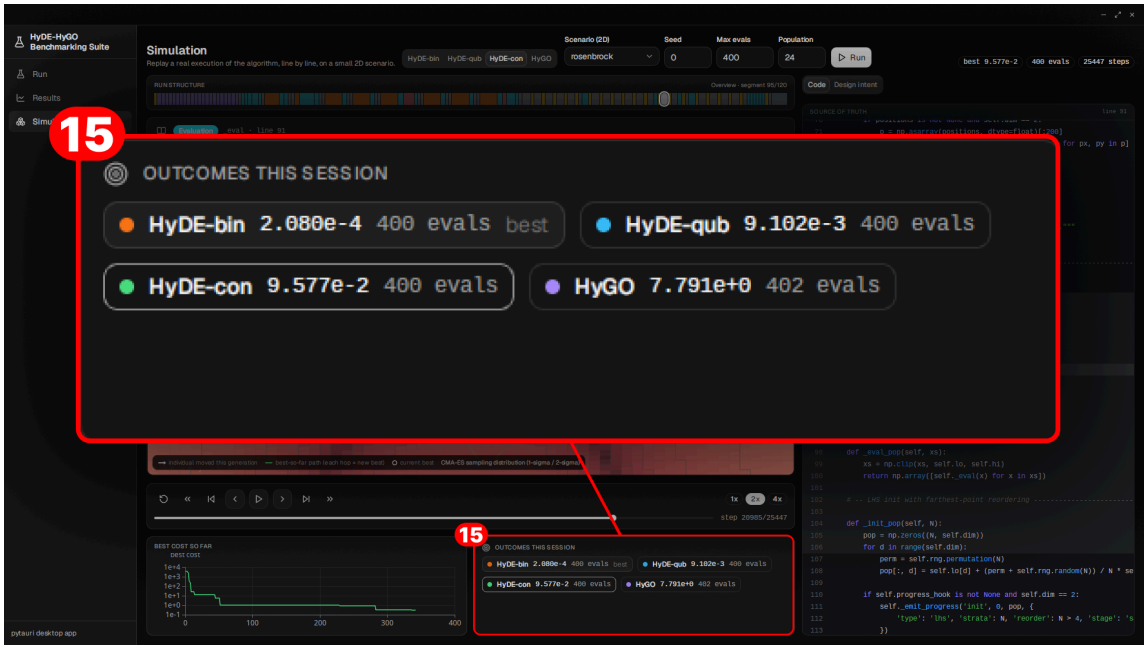

7
Scenario (2D)
rosenbrock
ackley
beale
booth


Seed
Max evals
Population
0
400
24
Run
8
9
10
11

HyDE-HyGO Benchmarking Suite
Simulation
Run
Results
Scenario (2D)
Seed
Max evals
Population
rosenbrock
400
24
Run
best 9.102e-3
400 evals
26818 steps
12
11
1x
2x
4x


364
365 # ── Main run ──
366
367 def run(self):
368 N = self.pop_size
369 conv_gen = None
370
371 theta, fitness = self._init_pop(N)
372 self.gen_best.append(self.best_cost)
373 self._emit_progress('init', 0, theta, {
374 'type': 'lhs',
375 'strata': N,
376 'reorder': N > 4,
377 'qubit': True,
378 'stage': 'final',
379 })
380
381 stag = 0
13

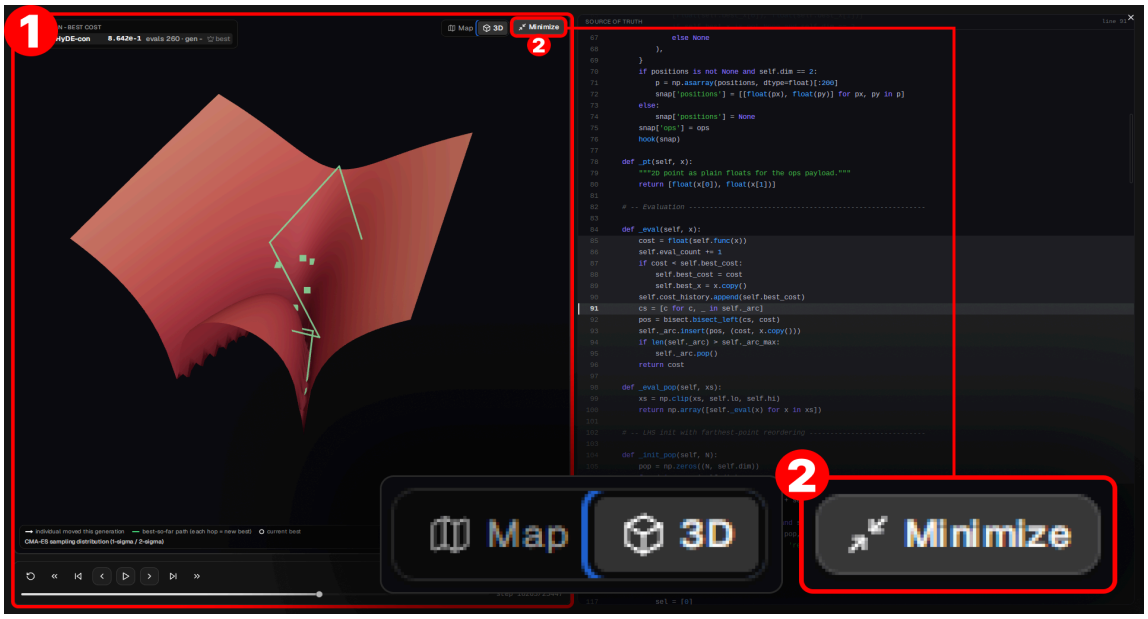
14
BEST COST SO FAR
best cost
1e+4
1e+3
1e+2
1e+1
1e+0
1e-1
0
100
200
300
400

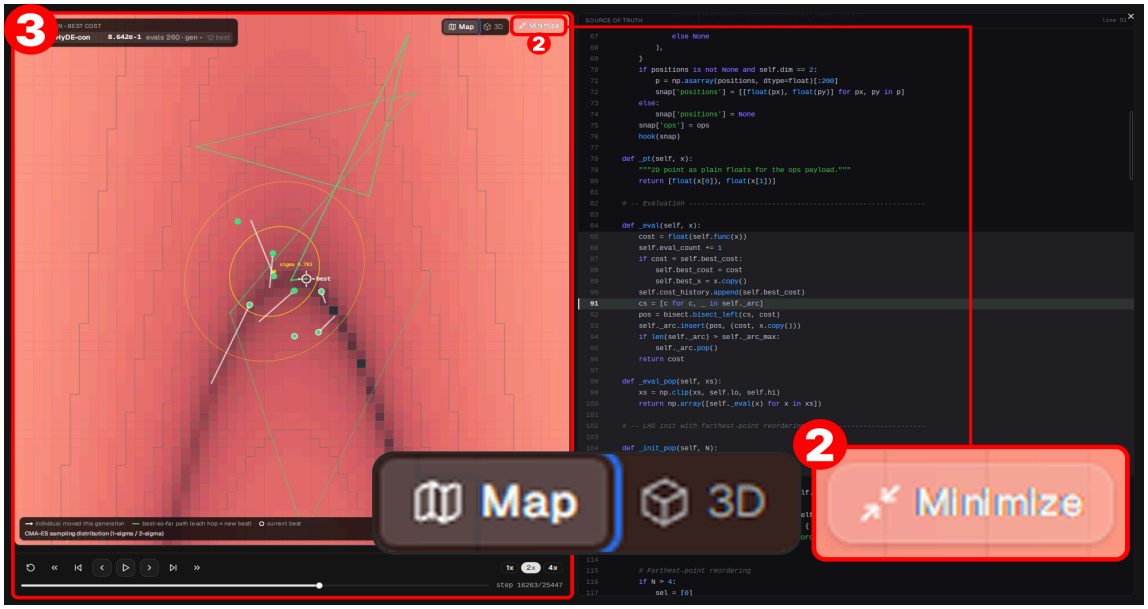

15
OUTCOMES THIS SESSION
HyDE-bin 2.080e-4 400 evals best
HyDE-qub 9.102e-3 400 evals
HyDE-con 9.577e-2 400 evals
HyGO 7.791e+0 402 evals

SWITCH AND ENLARGE THE SIMULATION DISPLAY

In the **Simulation** page, users can change or maximize the view to examine the search more closely, where they can:

1. See the **Maximized 3D Surface**, the enlarged 3D view with the playback controls beneath it;
2. Click **Minimize** to restore the original size, or click the Map or 3D button to switch between the top-down map and the 3D surface; and
3. See the **Maximized Map**, the enlarged top-down map with the playback controls beneath it.




3
Map
3D
2
2
Minimize

CURRICULUM VITAE

Manuel II M. Caballero
Purok Mangga, Puro, Aroroy, Masbate
09636989489
07307477@dwc-legazpi.edu

PERSONAL INFORMATION

Birthdate	:	September 13, 2005
Status	:	Single
Gender	:	Male
Religious affiliation	:	Roman Catholic

EDUCATIONAL ATTAINMENT

Tertiary	:	Divine Word College of Legazpi Course: BS Computer Science
Secondary	:	Aroroy National High School Year Graduated: 2023
Elementary	:	Rufo C. Pajes Sr. Central School Year Graduated: 2017

ACHIEVEMENTS

2023-2026	:	Consistent Academic Achiever
2023-2024	:	4th Placer, CSITS Hackathon
2024-2025	:	1st Placer, CSITS Hackathon
2025 -2026	:	Internal Vice President, CSITS

CURRICULUM VITAE

Denzel Jay B. Benjamin
Purok 3 Tres Marias, Donsol, Sorsogon
09060583631
07304749@dwc-legazpi.edu

PERSONAL INFORMATION

Birthdate	:	June 8, 2005
Status	:	Single
Gender	:	Male
Religious affiliation	:	Roman Catholic

EDUCATIONAL ATTAINMENT

Tertiary	:	Divine Word College of Legazpi Course: BS Computer Science
Secondary	:	Donsol National Comprehensive High School Year Graduated: 2023
Elementary	:	Busdac Elementary School Year graduate: 2017

ACHIEVEMENTS

2023-2026	:	Consistent Academic Achiever
2023-2024	:	External Vice President, CSITS
2024-2025	:	1st Placer, CSITS Hackathon

CURRICULUM VITAE

Christian Nickhos A. Divina
Seabreeeze Cabid-An Homes, Sorsogon City, Sorsogon
09455566097
07207243@dwc-legazpi.edu

PERSONAL INFORMATION

Birthdate	:	November 15, 2003
Status	:	Single
Gender	:	Male
Religious affiliation	:	Roman Catholic

EDUCATIONAL ATTAINMENT

Tertiary	:	Divine Word College of Legazpi Course: BS Computer Science
Secondary	:	Sorsogon National Comprehensive High School Year Graduated: 2022
Elementary	:	The Lewis College Year graduate: 2016